

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ



ĐỒ ÁN
TỐT NGHIỆP ĐẠI HỌC

Đề tài:

**Triển khai bộ tăng tốc tính toán mạng EfficientNet-V2
trên FPGA**

CAO NGỌC THÀNH ĐÔ
do.cnt210184@sis.hust.edu.vn

NGUYỄN MẠNH HÙNG
hung.nm210394@sis.hust.edu.vn

Lớp: Điện tử 09 - K66

Lớp: Điện tử 10 - K66

Khoa: Điện tử

Khoa: Điện tử

Trường: Điện - Điện tử

Trường: Điện - Điện tử

Giảng viên hướng dẫn: PGS. TS Nguyễn Đức Minh
TS Hoàng Phương Chi
TS Vương Hoàng Nam

Hà Nội, ngày 26 tháng 06 Năm 2025

ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP (DÀNH CHO CÁN BỘ HƯỚNG DẪN)

TS Vương Hoàng Nam

(Ký và ghi rõ họ tên)

Điểm từng tiêu chí cho lẻ đến 0,5. Nếu Điểm tổng các tiêu chí > 10 thì Điểm hướng dẫn làm tròn thành 10

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP
(DÀNH CHO CÁN BỘ PHẢN BIỆN)

Tên đề tài: Triển khai bộ tăng tốc tính toán mạng EfficientNet-V2 trên FPGA

Họ tên SV: Cao Ngọc Thành Đô MSSV: 20210184

Cán bộ phản biện:

STT	Tiêu chí (Điểm tối đa)	Hướng dẫn đánh giá tiêu chí	Điểm tiêu chí
1	Trình bày quyển ĐATN (4 điểm)	Đồ án trình bày đúng mẫu quy định, bố cục các chương logic và hợp lý: Bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy, có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn, v.v. Kỹ năng diễn đạt, phân tích, giải thích, lập luận: cấu trúc câu rõ ràng, văn phong khoa học, lập luận logic và có cơ sở, thuật ngữ chuyên ngành phù hợp, v.v.	
2	Nội dung và kết quả đạt được (5,5 điểm)	Nêu rõ tính cấp thiết, ý nghĩa khoa học và thực tiễn của đề tài, các vấn đề và các giả thuyết, phạm vi ứng dụng của đề tài. Thực hiện đầy đủ quy trình nghiên cứu: Đặt vấn đề, mục tiêu đề ra, phương pháp nghiên cứu/ giải quyết vấn đề, kết quả đạt được, đánh giá và kết luận. Nội dung và kết quả được trình bày một cách logic và hợp lý, được phân tích và đánh giá thỏa đáng. Biện luận phân tích kết quả mô phỏng/ phần mềm/ thực nghiệm, so sánh kết quả đạt được với kết quả trước đó có liên quan. Chỉ rõ phù hợp giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận đề đề xuất hướng giải quyết có thể thực hiện trong tương lai. Hàm lượng khoa học/ độ phức tạp cao, có tính mới/tính sáng tạo trong nội dung và kết quả đồ án.	
4	Điểm thành tích (1 điểm)	Có bài báo KH được đăng hoặc chấp nhận đăng/ đạt giải SV NCKH giải 3 cấp Trường trở lên/ Các giải thưởng khoa học trong nước, quốc tế từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế. (1 điểm) Được báo cáo tại hội đồng cấp Trường trong hội nghị SV NCKH nhưng không đạt giải từ giải 3 trở lên/ Đạt giải khuyến khích trong cuộc thi khoa học trong nước, quốc tế/ Kết quả đồ án là sản phẩm ứng dụng có tính hoàn thiện cao, yêu cầu khối lượng thực hiện lớn. (0,5 điểm)	
Điểm tổng các tiêu chí:			
Điểm hướng dẫn:			

Cán bộ hướng dẫn
(Ký và ghi rõ họ tên)

Điểm từng tiêu chí cho lẻ đến 0,5. Nếu Điểm tổng các tiêu chí > 10 thì Điểm hướng dẫn làm tròn thành 10

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP
(DÀNH CHO CÁN BỘ THÀNH VIÊN HỘI ĐỒNG)

Hội đồng số:

Họ tên SV: Cao Ngọc Thành Đô MSSV: 20210184

Cán bộ thành viên HĐ:

STT	Tiêu chí (Điểm tối đa)	Hướng dẫn đánh giá tiêu chí	Điểm tiêu chí
1	Chất lượng slides/Bản vẽ kỹ thuật (1,5 điểm)	Sử dụng các minh họa hỗ trợ: Hình ảnh, biểu đồ rõ nét và phù hợp, dễ hiểu	
		Không quá nhiều từ, biết sử dụng từ khoá; bố cục logic, có đánh số trang	
2	Kỹ năng thuyết trình (1,5 điểm)	Tự tin, làm chủ nội dung trình bày, đúng thời gian quy định	
		Dễ hiểu, dễ theo dõi, lô-gic, lời cuốn.	
3	Nội dung và kết quả đạt được (5 điểm)	Nêu rõ tính cấp thiết, ý nghĩa khoa học và thực tiễn của đề tài, các vấn đề và các giả thuyết, phạm vi ứng dụng của đề tài. Thực hiện đầy đủ quy trình nghiên cứu: Đặt vấn đề, mục tiêu đề ra, phương pháp nghiên cứu/ giải quyết vấn đề, kết quả đạt được, đánh giá và kết luận.	
		Nội dung và kết quả được trình bày một cách logic và hợp lý, được phân tích và đánh giá thỏa đáng. Biện luận phân tích kết quả mô phỏng/ phần mềm/ thực nghiệm, so sánh kết quả đạt được với kết quả trước đó có liên quan.	
		Chỉ rõ phù hợp giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận đề đề xuất hướng giải quyết có thể thực hiện trong tương lai. Hàm lượng khoa học/ độ phức tạp cao, có tính mới/tính sáng tạo trong nội dung và kết quả đồ án.	
4	Trả lời câu hỏi (2,5 điểm)	Trả lời ngắn gọn, chính xác, đi thẳng vào vấn đề của câu hỏi.	
		Nắm vững kiến thức cơ bản liên quan đến lĩnh vực nghiên cứu/ công việc của đồ án.	
5	Điểm thành tích (1 điểm)	Có bài báo KH được đăng hoặc chấp nhận đăng/ đạt giải SV NCKH giải 3 cấp Trường trở lên/ Các giải thưởng khoa học trong nước, quốc tế từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế. (1 điểm)	
		Được báo cáo tại hội đồng cấp Trường trong hội nghị SV NCKH nhưng không đạt giải từ giải 3 trở lên/ Đạt giải khuyến khích trong cuộc thi khoa học trong nước, quốc tế/ Kết quả đồ án là sản phẩm ứng dụng có tính hoàn thiện cao, yêu cầu khối lượng thực hiện lớn. (0,5 điểm)	
		Điểm tổng các tiêu chí:	
		Điểm hướng dẫn:	

Cán bộ hướng dẫn
(Ký và ghi rõ họ tên)

Điểm từng tiêu chí cho lẻ đến 0,5. Nếu Điểm tổng các tiêu chí > 10 thì Điểm hướng dẫn làm tròn thành 10

LỜI NÓI ĐẦU

Trong bối cảnh trí tuệ nhân tạo (AI) ngày càng khẳng định vai trò quan trọng trong nhiều lĩnh vực như thị giác máy tính, y tế, xe tự hành và các thiết bị thông minh, việc tối ưu hóa mô hình học sâu để triển khai trên phần cứng chuyên dụng trở nên cấp thiết hơn bao giờ hết. Các mô hình mạng nơ-ron tích chập (CNN), đặc biệt là các loại mạng EfficientNet – một loại mô hình mạng có kiến trúc nổi bật nhờ khả năng cân bằng giữa độ chính xác và hiệu năng – đang được ứng dụng rộng rãi trong các hệ thống nhúng và thiết bị biên (edge devices).

Tuy nhiên, với hạn chế về tài nguyên và yêu cầu xử lý thời gian thực của các nền tảng như FPGA, việc triển khai hiệu quả các mô hình phức tạp như EfficientNet-V2 đòi hỏi phải có các kỹ thuật thiết kế phần cứng tối ưu. Trong xu hướng đó, bộ gia tốc phần cứng (CNN Accelerator) cho phép đẩy nhanh quá trình suy luận (inference), giảm tiêu thụ điện năng và tăng khả năng tích hợp vào hệ thống SoC.

Xuất phát từ nhu cầu thực tiễn và tiềm năng ứng dụng rộng lớn, đề án tốt nghiệp với đề tài "Triển khai bộ tăng tốc tính toán mạng EfficientNet-V2 trên FPGA" được thực hiện nhằm thiết kế, tối ưu và kiểm thử một kiến trúc phần cứng cho phép xử lý mô hình EfficientNet-V2 trực tiếp trên nền tảng FPGA. Đề án không chỉ giúp làm rõ quy trình thiết kế một hệ thống tăng tốc CNN hiệu quả mà còn mở ra những hướng nghiên cứu mới trong lĩnh vực phần cứng AI trên các thiết bị nhúng.

Hy vọng rằng kết quả của đề án sẽ góp phần vào xu hướng hiện đại hóa các hệ thống trí tuệ nhân tạo nhúng, đáp ứng được yêu cầu về tốc độ, năng lượng và khả năng triển khai thực tế trong các ứng dụng công nghiệp.

Em xin gửi lời cảm ơn chân thành đến thầy **Nguyễn Đức Minh** và cô **Hoàng Phương Chi** đã hỗ trợ, hướng dẫn, và cung cấp những định hướng quý báu, cùng với trang thiết bị cần thiết giúp em thực hiện đề tài này. Em cũng xin gửi lời cảm ơn sâu sắc đến toàn thể đội **CNN (EDABK)** và **lab EDABK** đã tạo điều kiện thuận lợi để em hoàn thiện đề án này. Em xin gửi lời tri ân đến thầy cô, bạn bè và gia đình, những người đã luôn đồng hành và ủng hộ em trong suốt quá trình thực hiện đề án này.

Với hạn chế về kiến thức và kinh nghiệm trong quá trình thực hiện, em nhận thức rằng đề tài vẫn còn những thiếu sót. Tuy nhiên, em đã luôn cố gắng để cải thiện và hoàn thiện đề án. Em mong nhận được các đánh giá và nhận xét từ thầy cô và các bạn để đề án có thể được hoàn thiện hơn. Những ý kiến đóng góp và phản hồi chân thành từ thầy cô là những nguồn động lực quý báu giúp em hoàn thiện đề tài một cách tốt nhất. Với sự hỗ trợ này, em hy vọng đề án sẽ đạt được mục tiêu nghiên cứu và có giá trị thực tiễn cao hơn.

LỜI CAM ĐOAN

Tôi tên là **Cao Ngọc Thành Đô**, mã số sinh viên **20210184**, sinh viên lớp **Điện tử 09**, khóa **K66**. Tôi đã hoàn thành đồ án "**Triển khai bộ tăng tốc tính toán mạng EfficientNet-V2 trên FPGA**" dưới sự hướng dẫn của **PGS. TS NGUYỄN ĐỨC MINH, TS. HOÀNG PHƯƠNG CHI, TS. VƯƠNG HOÀNG NAM**. Tôi cam kết rằng tất cả các nghiên cứu trong đồ án này là kết quả của quá trình nghiên cứu và tìm hiểu của riêng tôi. Mọi thông tin trích dẫn trong đồ án đều tuân thủ các quy định về quyền sở hữu trí tuệ, và tôi đã chỉ rõ các tài liệu tham khảo một cách minh bạch. Tôi xin chịu trách nhiệm toàn bộ về nội dung đã viết trong đồ án này.

Hà Nội, ngày 26 tháng 6 năm 2025

Người cam đoan



Cao Ngọc Thành Đô

MỤC LỤC

DANH MỤC CHỮ VIẾT TẮT	i
DANH MỤC HÌNH VẼ	ii
DANH MỤC BẢNG BIỂU	iii
TÓM TẮT ĐỒ ÁN	iv
ABSTRACT	v
CHƯƠNG 1. ĐẶT VẤN ĐỀ	1
1.1 Phân tích bài toán và đề xuất hệ thống	1
1.2 Tổng quan tình hình nghiên cứu hiện nay	3
1.3 Lý do chọn đề tài	5
1.4 Mục tiêu đề tài	7
1.4.1 Chỉ tiêu kỹ thuật	7
1.4.2 Yêu cầu chức năng	8
1.5 Kế hoạch thực hiện	9
1.6 Kết luận chương	9
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	10
2.1 Mạng Neural tích chập	10
2.1.1 Tổng quan về mạng Neural tích chập	10
2.1.2 Kiến trúc CNN cơ bản	16
2.1.3 Ưu điểm và nhược điểm của CNN	18
2.2 MobileNet và EfficientNet	20
2.2.1 Sự ra đời của mạng MobileNet và EfficientNet	20
2.2.2 Lớp tích chập Depth wise đặc trưng của mạng MobileNet và Efficient Net	23

2.2.3	Hàm kích hoạt (Activation)	24
2.2.4	Lớp Ép và Kích thích (Squeeze-and-Excitation) (SE) trong MobileNet và EfficientNet	26
2.2.5	Ưu điểm và nhược điểm của mạng nơ-ron tích chập (CNN) nói chung và EfficientNet nói riêng	26
2.2.6	Ứng dụng của Mạng EfficientNet	27
2.3	CNN Accelerator trên FPGA	28
2.3.1	Tổng quan về FPGA	28
2.3.2	Khó khăn khi triển khai CNN trên CPU và GPU tại thiết bị biên	29
2.3.3	Sự cần thiết của bộ tăng tốc CNN	30
2.3.4	Kiến trúc tổng thể của một CNN Accelerator trên FPGA	31
2.3.5	Hệ thống bộ nhớ	32
2.3.6	Giao tiếp nội bộ và ngoại vi	33
2.4	Công cụ và phần mềm thiết kế	35
2.4.1	Vivado	35
2.5	Thông số đánh giá	36
2.6	Kết luận chương	38
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG		40
3.1	Kiến trúc tổng thể hệ thống	40
3.2	Thiết kế bộ tăng tốc tính toán mạng EfficientNet-V2	41
3.2.1	Luồng dữ liệu trong hệ thống	41
3.2.2	Triển khai khối tính xử lý chính Fused FU	42
3.2.3	Triển khai khối tính xử lý chính MB FU	43
3.2.4	Triển khai khối tính toán tích chập - Processing Elements	45
3.2.5	Triển khai khối tính toán hàm kích hoạt - Active Block	46
3.2.6	Triển khai khối tính toán - Pooling	47
3.2.7	AXI Master	48
3.3	Kết luận chương	51
CHƯƠNG 4. THỰC NGHIỆM VÀ KẾT QUẢ		52

4.1	Xây dựng môi trường thực nghiệm	52
4.1.1	Xây dựng block design	52
4.1.2	Phân vùng địa chỉ trong block design	53
4.1.3	Kiểm thử chức năng và đánh giá độ chính xác	54
4.1.4	Đánh giá Throughput và FPS	56
4.1.5	Tài nguyên và năng lượng tiêu thụ	56
4.2	Kết quả thực nghiệm	57
4.2.1	Quá trình kiểm thử chức năng và đánh giá độ chính xác	57
4.2.2	Tài nguyên sử dụng	58
4.2.3	Throughput và FPS	59
4.3	Kết quả và nhận xét	59
4.4	Kết luận chương	60
	KẾT LUẬN	61
	Kết luận chung	61
	Hướng phát triển	61
	PHỤ LỤC	64

DANH MỤC CHỮ VIẾT TẮT

Viết tắt	Giải thích
AI	Artificial Intelligence (Trí tuệ nhân tạo)
SoC	System on Chip (Hệ thống trên chip)
CNN	Convolutional Neural Network (Mạng nơ-ron tích chập)
CPU	Central Processing Unit (Bộ xử lý trung tâm)
FPGA	Field Programmable Gate Array
FSM	Finite State Machine (Máy trạng thái hữu hạn)
FPS	Frames Per Second
PWC	Pointwise Convolution (Tích chập điểm)
DWC	Depthwise Convolution (Tích chập theo chiều sâu)
MB Conv	Mobile Inverted Bottleneck Convolution
Fused-MB Conv	Fused MB layer Convolution
PE	Processing Element (Phần tử xử lý)
PE Cluster	Processing Element Cluster (Cụm phần tử xử lý)
FU	Functional Unit (Khối chức năng)
IFM	Input Feature Map (Ma trận đặc trưng đầu vào)
OFM	Output Feature Map (Ma trận đặc trưng đầu ra)
BRAM	Block RAM (Bộ nhớ RAM khối)
AXI	Advanced eXtensible Interface (Chuẩn giao tiếp AXI)
UART	Universal Asynchronous Receiver–Transmitter
DMA	Direct Memory Access (Truy cập bộ nhớ trực tiếp)
DSP	Digital Signal Processing (Xử lý tín hiệu số)
LUT	Look-Up Table (Bảng tra cứu)
FF	Flip-Flop (Chốt nhớ)
BN	Batch Normalization (Chuẩn hóa theo lô)
SE	Squeeze-and-Excitation (Khối nén và kích thích)

DANH MỤC HÌNH VẼ

Hình 1.1	Tổng quan hệ thống triển khai bộ gia tốc CNN trên FPGA	2
Hình 2.1	Lớp Tích chập	11
Hình 2.2	Kernel	12
Hình 2.3	Bước nhảy stride	13
Hình 2.4	Đệm dữ liệu biên	14
Hình 2.5	Độ sâu	15
Hình 2.6	Luồng dữ liệu của mạng CNN	17
Hình 2.7	Lớp Pooling	18
Hình 2.8	Kiến trúc mạng EfficientNet-V2.	21
Hình 2.9	Luồng dữ liệu tính toán cho lớp depthwise	23
Hình 2.10	Kiến trúc của FPGA	29
Hình 2.11	Sơ đồ khối bộ gia tốc CNN thông thường hiện nay	31
Hình 2.12	Hệ thống bộ nhớ	33
Hình 2.13	AXI bus	34
Hình 3.1	Sơ đồ khối hệ thống	40
Hình 3.2	Luồng dữ liệu giữa bộ nhớ trong chip và bộ nhớ ngoài chip	42
Hình 3.3	Khối Fused	43
Hình 3.4	Khối MB	44
Hình 3.5	Cấu trúc của khối tính toán và hoạt động của khối	46
Hình 3.6	Cấu trúc và hoạt động của khối tính toán Average Pooling	48
Hình 3.7	Mô hình FSM của Module AXI Master	49
Hình 3.8	Quy trình đọc dữ liệu của IP AXI bus trên vivado	50
Hình 3.9	Quy trình ghi dữ liệu của IP AXI bus trên vivado	51
Hình 4.1	Hệ thống block design trên Vivado	53
Hình 4.2	Phân vùng địa chỉ trong block design	54
Hình 4.3	Môi trường kiểm thử	55

DANH MỤC BẢNG BIỂU

Bảng 1.1	Tổng hợp kết quả các nghiên cứu tiêu biểu	4
Bảng 1.2	Thông số kĩ thuật của hệ thống	7
Bảng 1.3	Kế hoạch thực hiện đề tài	9
Bảng 4.1	Kết quả các bài kiểm tra	57
Bảng 4.2	Kết quả sau khi tổng hợp bằng Vivado	58
Bảng 4.3	Kết quả đo hiệu năng	59
Bảng 4.4	Tổng hợp và so sánh với các nghiên cứu tiêu biểu	59

TÓM TẮT ĐỒ ÁN

Sự xuất hiện của các mạng nơ-ron tích chập gọn nhẹ (CNN) như MobileNet và Efficient Net, vốn kết hợp phép tích chập phân tách theo chiều sâu (depthwise separable convolution) với tích chập chuẩn, đã giúp giảm đáng kể kích thước mô hình và độ phức tạp tính toán. Những tiến bộ này thúc đẩy mạnh mẽ việc phát triển các kiến trúc CNN hiệu quả, phù hợp triển khai trên thiết bị biên, đồng thời kéo theo sự ra đời của các bộ tăng tốc phần cứng chuyên dụng. Tuy nhiên, việc hiện thực hóa hiệu quả các phép tích chập này trên phần cứng (FPGA hoặc ASIC) vẫn gặp nhiều thách thức như sử dụng tài nguyên chưa tối ưu, các đơn vị tính toán bị nhàn rỗi, hạn chế trong truy cập bộ nhớ và tối ưu luồng dữ liệu, dẫn đến nghẽn băng thông và tăng độ trễ hệ thống.

Trong nghiên cứu này, chúng tôi đề xuất triển khai bộ tăng tốc tính toán cho mạng EfficientNetV2_B0 trên FPGA theo kiến trúc hai lõi với hai khối tính toán chuyên biệt tương ứng với các khối chính của mô hình: Fused FU và MB FU. Kiến trúc đề xuất được triển khai và đánh giá trên nền tảng Zynq ZCU104 FPGA, hoạt động ổn định ở tần số 250 MHz. Hệ thống đạt tổng hiệu năng 62,7 GOPS, trong đó Fused FU đạt 34 GOPS và MB FU đạt 28,7 GOPS. Thiết kế tiêu thụ 42,586 LUT (18.48%), 43,600 FFs (9.46%), 222 BRAMs (71.15%).

Nội dung của đồ án được trình bày qua bốn chương chính sau đây:

Chương 1: Đặt Vấn Đề

Chương 2: Cơ Sở Lý Thuyết

Chương 3: Thiết Kế Hệ Thống

Chương 4: Thực Nghiệm Và Kết Quả

ABSTRACT

The emergence of lightweight convolutional neural network (CNN) such as Mobile Net and Efficient Net, which combine depthwise-separable convolution with standard convolution, has significantly reduced model size and computational complexity. These advances have strongly driven the development of efficient CNN architectures suitable for edge deployment, leading to the advent of dedicated hardware accelerators. However, effectively realizing these convolution operations on hardware (FPGA or ASIC) still faces numerous challenges, including sub-optimal resource utilization, idle compute units, limitations in memory access and data-flow optimization, which ultimately result in bandwidth bottlenecks and increased system latency.

In this study, we propose an FPGA-based computing accelerator for the Efficient Net V2 B0 network using a dual-core architecture that incorporates two specialized compute engines corresponding to the model's key blocks: Fused FU and MB FU. The proposed design is implemented and evaluated on the Zynq ZCU104 FPGA platform, operating stably at 250 MHz. The system attains an aggregate performance of 62.7 GOPS, with the Fused FU contributing 34 GOPS and the MB FU reaching 28.7 GOPS. The implementation consumes 42,586 LUTs (18.48%), 43,600 FFs (9.46%), and 222 BRAMs (71.15%).

The content of the project is organized into four main chapters as follows:

Chapter 1: Introduction

Chapter 2: Theoretical Background

Chapter 3: System Design

Chapter 4: Experimentation and Results

CHƯƠNG 1. ĐẶT VẤN ĐỀ

Chương này nhằm giới thiệu tổng quan về bài toán nghiên cứu, trình bày hiện trạng các phương pháp tiếp cận hiện có, từ đó phân tích ưu – nhược điểm, xác định các yêu cầu chức năng cũng như đề xuất hướng giải quyết hợp lý. Đồng thời, chương cũng đưa ra các chỉ tiêu kỹ thuật cụ thể đóng vai trò làm cơ sở để xây dựng và đánh giá hệ thống trong các chương tiếp theo. Bên cạnh đó, một kế hoạch thực hiện chi tiết cũng được đề ra, nhằm đảm bảo tiến độ và định hướng triển khai hiệu quả cho toàn bộ dự án.

1.1 Phân tích bài toán và đề xuất hệ thống

Trong bối cảnh về công nghệ trí tuệ nhân tạo (AI) phát triển mạnh mẽ từng ngày, các mô hình về học sâu đã và đang hằng ngày được ứng dụng phổ biến và rộng rãi trong nhiều lĩnh vực của đời sống từ nhận diện hình ảnh, nhận diện giọng nói con người, đến phân tích văn bản. Đặc biệt, các mạng nơ ron tích chập (CNN) như mạng EfficientNet hay là MobileNet đã chứng minh được khả năng vượt trội trong việc giải quyết các bài toán phân loại hình ảnh với độ chính xác cao, đồng thời tối ưu hóa được sự phức tạp của mô hình và tài nguyên sử dụng cho tính toán và giảm thiểu tiêu thụ năng lượng đối với các thiết bị biên.

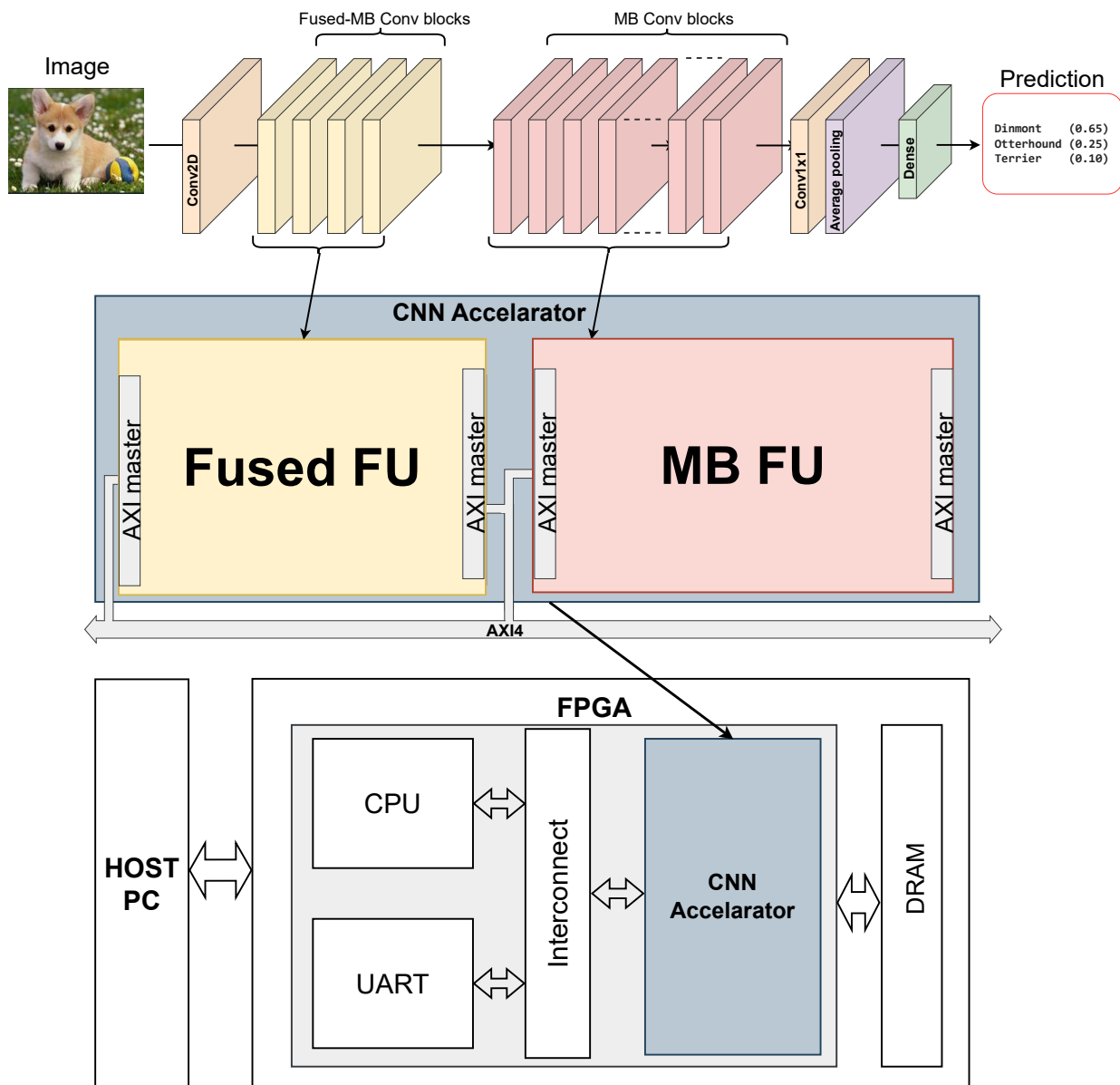
EfficientNet hay MobileNet là cấu trúc mạng nhẹ, phù hợp với các thiết bị biên vốn bị giới hạn về tài nguyên phần cứng và khả năng tính toán. Tuy nhiên, việc triển khai mạng này trên các thiết bị nhúng, thiết bị di động vẫn gặp nhiều thách thức, đặc biệt là về mặt hiệu suất tính toán và tiêu thụ năng lượng. Vì vậy, việc sử dụng các kiến trúc phần cứng tối ưu cho các mô hình mạng CNN nói chung và mạng EfficientNet nói riêng sẽ giúp giải quyết vấn đề này, từ đó đạt được hiệu suất tính toán hiệu quả và giảm lượng năng lượng tiêu thụ.

Vì vậy, chúng tôi đề xuất triển khai một bộ tăng tốc tính toán cho mạng CNN được tích hợp trong hệ thống SoC trên nền tảng FPGA, như minh họa trong Hình 1.1. Phía trên cùng của hình thể hiện sơ đồ kiến trúc của mạng EfficientNetV2, trong đó dữ liệu đầu vào là ảnh sẽ được mã hóa thành các giá trị điểm ảnh và đầu ra của hệ thống là kết quả dự đoán đối tượng. Mạng EfficientNetV2 bao gồm hai khối tính toán chính là Fused-MB Conv và MB Conv. Tương ứng với hai khối này, phần cứng sẽ có hai đơn vị xử lý riêng biệt là Fused FU và MB FU.

Bộ gia tốc tính toán cho mạng CNN giao tiếp với hệ thống thông qua giao diện AXI, sử dụng các khối AXI Master để truy xuất dữ liệu cần thiết từ bộ nhớ. Cuối cùng, toàn bộ CNN Accelerator sẽ được tích hợp vào hệ thống phần cứng trên FPGA.

Dữ liệu ảnh được thu thập từ môi trường bên ngoài sẽ được máy tính chủ (Host PC) truyền vào hệ thống thông qua giao thức UART. Khối xử lý trung tâm (CPU) đảm nhiệm việc điều khiển và khởi tạo các tham số, dữ liệu tính toán, bao gồm dữ liệu ảnh và trọng số, được

lưu trữ trong bộ nhớ ngoài (DRAM). Toàn bộ quá trình truyền nhận dữ liệu giữa các khối chức năng trong hệ thống sẽ do khối Interconnect đảm nhận, đảm bảo kết nối giữa các thành phần trong hệ thống SoC.



Hình 1.1 Tổng quan hệ thống triển khai bộ gia tốc CNN trên FPGA

1.2 Tổng quan tình hình nghiên cứu hiện nay

Trong những năm gần đây, với sự phát triển mạnh mẽ của trí tuệ nhân tạo, đặc biệt là các mạng nơ-ron tích chập (CNN), việc tối ưu và triển khai các mô hình CNN trên nền tảng phần cứng như FPGA đã trở thành một hướng nghiên cứu tiềm năng. Nhu cầu thiết kế bộ gia tốc (accelerator) hiệu quả cho các mạng như MobileNet, EfficientNet nhằm phục vụ cho các thiết bị nhúng, thiết bị biên (edge devices) có tài nguyên hạn chế ngày càng tăng. Nhiều công trình đã tập trung khai thác đặc điểm nhẹ, tốc độ cao của các mô hình CNN hiện đại để tối ưu hóa hiệu suất xử lý, giảm tiêu thụ tài nguyên và năng lượng khi triển khai trên FPGA.

Chúng tôi trình bày một số hướng tiếp cận tiêu biểu trong việc xây dựng và tối ưu bộ gia tốc cho mạng EfficientNet và MobileNet trên FPGA, qua đó làm cơ sở tham khảo cho thiết kế trong đề tài.

Nghiên cứu [1] trình bày một bộ gia tốc nhẹ và có thể cấu hình lại cho EfficientNet trên FPGA. Dự án giải quyết vấn đề hiệu quả khi xử lý các phép toán depthwise convolution, module squeeze-and-excitation (SE) và các hàm kích hoạt như swish và sigmoid. Thiết kế của LETA sử dụng datapath có thể thay đổi để cân bằng tải công việc giữa các engine convolution, giảm thiểu tình trạng nhàn rỗi. LETA được triển khai trên FPGA Xilinx XCVU37P và đạt được thông lượng gấp 2 lần trên mỗi DSP với tổn thất độ chính xác tối thiểu (1.6% top-1 và 0.7% top-5) khi chạy EfficientNet-B3.

Bên cạnh đó bài báo [2] đề xuất FILM-QNN, một phương pháp lượng tử hóa nhiều lớp với độ chính xác pha trộn, nhằm tối ưu hóa và tăng tốc các mô hình DNN trên FPGA. Thuật toán này gán độ chính xác khác nhau cho các bộ lọc trong mỗi lớp, giúp giảm độ chính xác mất mát và cải thiện khả năng song song hóa phần cứng. Bài báo cũng sử dụng các kỹ thuật tối ưu hóa như đóng gói DSP và sắp xếp lại trọng số, giúp nâng cao thông lượng và sử dụng hiệu quả tài nguyên FPGA. Các kết quả thử nghiệm cho thấy phương pháp này đạt độ chính xác tương đương với các phiên bản 8-bit và 32-bit, đồng thời cải thiện thông lượng tính toán trên các mô hình ResNet-18, ResNet-50 và MobileNet-V2.

Tại nghiên cứu [3] đã giới thiệu một kiến trúc phần cứng tái cấu hình để tăng tốc mạng nơ-ron tích chập (CNN) nhẹ EfficientNet. Kiến trúc này bao gồm một mảng thanh ghi và lõi tính toán tái cấu hình để tăng tốc tính toán độ chập chiều sâu, một đơn vị vector để thực thi các hàm kích hoạt phi tuyến và phép toán tỉ lệ, cùng với một kiến trúc lõi tính toán song song để cải thiện hiệu suất và khả năng sử dụng tài nguyên. Các kiến trúc bộ nhớ cũng được thiết kế để hỗ trợ việc tăng tốc các phép toán này. Đánh giá hiệu suất cho thấy accelerator đạt 69,50 FPS và 255,22 GOPS với tần số hệ thống chính là 300 MHz và DSP kernel chạy ở 600 MHz. So với accelerator EfficientNet-B3 hiện tại, thiết kế này cải thiện hiệu suất lõi đơn lên 1.28 lần và hiệu suất của mỗi DSP lên 1.38 lần.

Cuối cùng trong bài báo [4] giới thiệu một thiết kế accelerator dành riêng cho MobileNet,

tối ưu hóa việc tính toán Depthwise Separable Convolution (DSC) và các phép toán liên quan. Các tác giả đề xuất sử dụng các đơn vị xử lý (PE) chuyên dụng cho Depthwise Convolution (DWC) và Pointwise Convolution (PWC), cùng với một mô-đun tái sắp xếp cho lớp convolution đầu tiên. Họ cũng giới thiệu các kỹ thuật xử lý DSC theo đường ống gọi là Pipelining Separable Convolution (PSC) và Tiled-Convolution (TC), đặc biệt xem xét tải tính toán của PWC. Việc sử dụng quantization 8-bit trong accelerator chỉ gây giảm độ chính xác không đáng kể (0.68%) so với độ chính xác đầy đủ, trong khi vẫn đảm bảo các phép toán phần cứng đơn giản và hiệu quả. Trên nền tảng ZCU-102, accelerator này đạt được 190.9 FPS và 108.3 GOPS với tài nguyên phần cứng tối thiểu, đồng thời đạt hiệu suất năng lượng 18.20 GOPS/W, cao gấp 3.7 lần so với GPU A-100.

Trong các nghiên cứu tiêu biểu nêu trên thì ta có thể tóm lược được các thông số tiêu biểu cần chú ý đến khi thực hiện việc triển khai mạng nhân tích chập EfficientNet, MobileNet như bảng 1.1:

Bảng 1.1 Tổng hợp kết quả các nghiên cứu tiêu biểu

	[1]	[2]	[3]	[4]
Mô hình	EfficientNet B0	MobileNetV2	EfficientNet B3	MobileNetV1
FPGA	Xilinx XCVU37P	ZCU102	Xilinx XCVU37P	Xilinx XCZU9EG
Tần số	300MHz	100MHz	300MHz	150MHz
Throughput(GOPS)	95.1	29.3	255.2	108.3
LUTs	71,689	39,100	138,519	214,900
FFs	113,415	N/A	197,799	166,000
BRAMs	85	126.5	220.5	506
Năng lượng	7.8 W	3 W	22,885 W	5,95 W
FPS (ảnh/s)	243.8	49.2	69.5	190
GOPS/W	12.2	9.7	11.15	18.2

Những nghiên cứu trước đây chủ yếu tập trung vào việc tăng tốc tính toán và cải thiện thông lượng cho các mạng thuộc họ MobileNet và EfficientNetV1. Các dự án này đã đạt được những kết quả đáng khích lệ, giải quyết thành công các bài toán đề ra trong nhiều ứng dụng khác nhau. Tuy nhiên, khi áp dụng những kiến trúc này vào việc tính toán cho mạng EfficientNetV2, các mô hình và phương pháp trước đây đã gặp phải một số hạn chế rõ rệt, đặc biệt là về khả năng xử lý thông lượng cao và hiệu quả sử dụng tài nguyên phần cứng. Với sự phát triển và yêu cầu ngày càng cao về hiệu suất tính toán trong các ứng dụng AI phức tạp, việc sử dụng các kiến trúc cũ không còn đáp ứng được các tiêu chí về hiệu suất và tối ưu tài nguyên phần cứng trong việc triển khai EfficientNetV2.

1.3 Lý do chọn đề tài

Với sự phát triển mạnh mẽ của trí tuệ nhân tạo (AI) và học sâu, các mô hình mạng nơ-ron tích chập (CNN) đã chứng minh hiệu quả vượt trội trong việc xử lý hình ảnh, video và các tín hiệu không gian. Tuy nhiên, việc triển khai các mô hình CNN trên các hệ thống có tài nguyên hạn chế, như thiết bị biên (edge devices) hay các thiết bị nhúng, là một thách thức lớn. Các thiết bị này thường có hạn chế về bộ xử lý, bộ nhớ và năng lượng, do đó yêu cầu các giải pháp tính toán tối ưu, giúp duy trì hiệu suất cao trong khi giảm thiểu tiêu thụ tài nguyên.

Một xu hướng nổi bật hiện nay là việc thiết kế và triển khai các bộ gia tốc phần cứng chuyên dụng cho việc tính toán các mạng CNN, đặc biệt là trên các thiết bị biên. Các bộ gia tốc này, bao gồm FPGA, ASIC, hay các kiến trúc phần cứng khác, cho phép tăng tốc quá trình tính toán của CNN, giảm độ trễ và tối ưu hóa sử dụng tài nguyên phần cứng. Việc tích hợp bộ gia tốc CNN vào các thiết bị biên không chỉ giúp tăng cường khả năng xử lý AI ngay tại nơi thu thập dữ liệu mà còn giảm tải cho các hệ thống đám mây, cải thiện hiệu quả và tiết kiệm băng thông.

FPGA đã trở thành công nghệ tiên tiến trong thiết kế vi mạch. Việc sử dụng FPGA để triển khai các mạng nơ-ron không chỉ nâng cao hiệu suất tính toán mà còn hỗ trợ xử lý song song, giúp tăng tốc độ thực thi cho các ứng dụng trí tuệ nhân tạo, đồng thời đáp ứng hiệu quả các yêu cầu thực tiễn trong nghiên cứu và phát triển. Hiện nay, để giải quyết các bài toán như nhận dạng hình ảnh, xử lý tín hiệu và robot tự động hóa, việc sử dụng FPGA làm nền tảng triển khai không chỉ tối ưu hóa hiệu suất mà còn giúp tiết kiệm chi phí và giảm thiểu năng lượng tiêu thụ.

Với xu hướng phát triển này, việc thiết kế bộ gia tốc phần cứng cho các mô hình CNN, như EfficientNet, trên các nền tảng biên là một hướng nghiên cứu tiềm năng và quan trọng. Điều này không chỉ giúp nâng cao hiệu suất tính toán của các mô hình học sâu mà còn góp phần tối ưu hóa việc triển khai AI trên các thiết bị biên, đáp ứng yêu cầu về hiệu suất, tiêu thụ năng lượng và khả năng mở rộng trong các ứng dụng thực tế.

Chính vì vậy, đề tài "Triển khai bộ tăng tốc tính toán mạng EfficientNet-V2 trên FPGA" được chọn nhằm nghiên cứu, thiết kế và thử nghiệm một bộ gia tốc phần cứng cho mô hình EfficientNet, tối ưu hóa quá trình tính toán và mang lại hiệu quả cao trong việc triển khai trên các thiết bị biên.

Đã có nhiều nghiên cứu và triển khai bộ gia tốc phần cứng cho mạng CNN, đặc biệt là các ứng dụng trên các thiết bị di động và mạng EfficientNetV1 trên nền tảng FPGA. Những bộ gia tốc này đã mang lại hiệu quả đáng kể trong việc tối ưu hóa hiệu suất tính toán, giảm độ trễ và tiết kiệm tài nguyên cho các ứng dụng AI trong nhiều lĩnh vực như nhận dạng hình ảnh, phân tích video và các ứng dụng trí tuệ nhân tạo thời gian thực. Tuy nhiên, khi đặt trong bối cảnh triển khai cho mạng EfficientNetV2, những bộ gia tốc hiện tại vẫn còn một số hạn chế

đáng kể:

1. *Khó khăn trong việc xử lý các phép toán phức tạp.* Các bộ gia tốc hiện tại không thể xử lý hiệu quả các phép toán phức tạp trong EfficientNetV2, dẫn đến việc giảm hiệu suất tính toán.
2. *Hạn chế về thông lượng.* Mặc dù EfficientNetV2 yêu cầu thông lượng tính toán cao, nhưng các bộ gia tốc cũ chưa thể đáp ứng được yêu cầu này, khiến việc triển khai mạng gặp phải độ trễ lớn.
3. *Hiệu quả sử dụng tài nguyên phần cứng chưa tối ưu.* Các kiến trúc phần cứng hiện tại không thể tận dụng tối đa khả năng song song hóa và các đặc điểm tối ưu của EfficientNetV2, đặc biệt là khi triển khai trên FPGA.
4. *Khó khăn trong việc tối ưu hóa sử dụng năng lượng.* Việc tối ưu hóa hiệu quả năng lượng cho các ứng dụng AI trên FPGA vẫn là một thách thức, đặc biệt là khi triển khai các mô hình CNN phức tạp như EfficientNetV2.

Vậy nên, để giải quyết các hạn chế và bất lợi nêu trên, chúng ta cần tìm ra các phương pháp và công nghệ mới. Một trong những giải pháp quan trọng để cải thiện hiệu suất và tối ưu hóa việc triển khai các mô hình CNN như EfficientNetV2 trên FPGA là lựa chọn một FPGA phù hợp, có đủ tài nguyên tính toán cần thiết như LUT, FF và BRAM.

Xét trong bối cảnh cơ sở vật chất của phòng thí nghiệm, ta có thể lựa chọn giữa hai loại FPGA là ZCU104 và KC705, cả hai đều có thể đáp ứng được các tiêu chí về tài nguyên phần cứng và khả năng mở rộng. Sau đây là một số so sánh giữa hai loại FPGA này để giúp ta đưa ra lựa chọn phù hợp:

- *Tích hợp hệ thống đa lõi:* ZCU104 sử dụng hai lõi là ARM Cortex-A9 và Cortex-A53 từ đó có khả năng xử lý đa nhiệm và nhiều luồng, giúp tăng hiệu suất xử lý cho các mtính toán tích chập.
- *Có nhiều IP hỗ trợ:* Zynq ZCU104 tích hợp nhiều IP như MIG, Microblazer và USB, có thể kết nối với nhiều thiết bị cùng lúc như camera và mạng. Còn đối với KC705 thì nó không hỗ trợ kết nối camera và có ít những cổng chức năng chung như GPIO
- *Tính linh hoạt:* KC705 rất linh hoạt có khả năng tùy biến cao, có thể tinh chỉnh cấu trúc phần cứng để phù hợp với kiến trúc triển khai. Còn ZCU104 thì bị giới hạn hơn, các kiến trúc phần cứng mới được thêm vào và giao tiếp với cổng chức năng đều phải thông qua các module khác như MPSoC.

Dựa trên phân phân tích trên giữa hai loại FPGA, ta có thể thấy KC705 có ít sự hỗ trợ từ IP hơn và không hỗ trợ camera. Vì vậy với đề tài này ta sẽ sử dụng FPGA board là Zynq UltraScale+ MPSoC ZCU104.

Đề tài này giúp tôi nâng cao các kiến thức chuyên ngành về điện tử, vi mạch và trí tuệ nhân tạo, đồng thời phát triển kỹ năng nghiên cứu và thực hành trong việc thiết kế các bộ gia tốc phần cứng. Việc thiết kế bộ gia tốc cho mạng nơ ron tích chập EfficientNet và triển khai trên nền tảng FPGA yêu cầu tôi phải áp dụng các kỹ năng chuyên môn, từ lập trình phần mềm, tối ưu hóa phần cứng cho đến kết nối mạch điện tử. Qua đó, tôi sẽ phát triển khả năng giải quyết các bài toán thực tế trong lĩnh vực AI và điện tử, đồng thời trau dồi kỹ năng thiết kế vi mạch và tối ưu hóa hệ thống phần cứng.

1.4 Mục tiêu đề tài

"Triển khai bộ tăng tốc tính toán mạng EfficientNet-V2 trên FPGA" là nghiên cứu và phát triển một bộ gia tốc phần cứng chuyên dụng cho việc triển khai mạng nơ ron tích chập EfficientNet trên nền tảng FPGA, với mục đích tối ưu hóa hiệu suất tính toán và giảm thiểu tài nguyên sử dụng của bộ gia tốc CNN hiện tại. Với những chỉ tiêu kỹ thuật và yêu cầu chức năng dưới đây dưới đây.

1.4.1 Chỉ tiêu kỹ thuật

Đối với chỉ tiêu kỹ thuật của hệ thống ta có thể dựa vào bảng 1.2 để đưa ra những chỉ tiêu như sau:

Bảng 1.2 Thông số kỹ thuật của hệ thống

	[1]	[2]	[3]	[4]	Đề tài này
Mô hình	EfficientNet B0	MobileNetV2	EfficientNet B3	MobileNetV1	EfficientNetV2 B0
FPGA	Xilinx XCVU37P	ZCU102	Xilinx XCVU37P	Xilinx XCZU9EG	ZCU104
Tần số	300MHz	100MHz	300MHz	150MHz	$\geq 200\text{MHz}$
Throughput(GOPS)	95.1	29.3	255.2	108.3	≥ 50
LUTs	71,689	39,100	138,519	214,900	$< 100,000$
FFs	113,415	N/A	197,799	166,000	$< 100,000$
BRAMs	85	126.5	220.5	506	< 300
Năng lượng	7.8W	3W	22.885W	5,95 W	$< 5\text{W}$
FPS	243.8	49.2	69.5	190	≥ 30
GOPS/W	12.2	9.7	11.15	18.2	≥ 15

Dựa vào các số liệu trong bảng trên, chúng tôi sẽ triển khai hệ thống theo hướng tối ưu để đáp ứng đầy đủ các yêu cầu kỹ thuật đã đề ra.

1.4.2 Yêu cầu chức năng

Trong đồ án này, tôi thực hiện thiết kế bộ gia tốc cho mạng nơ ron tích chập EfficientNet và triển khai trên nền tảng FPGA với các yêu cầu chức năng sau:

- Hệ thống tiếp nhận đầu vào là các hình ảnh từ tập dữ liệu ImageNet [5]. Mỗi ảnh sẽ được chuyển đổi thành các giá trị số tương ứng với từng điểm ảnh. Các giá trị số này sau đó được truyền vào bộ tăng tốc tính toán tích chập để thực hiện quá trình suy luận, đầu ra của hệ thống là dự đoán về đối tượng có trong ảnh.
- Sau khi tích hợp bộ gia tốc CNN dành cho mô hình EfficientNet, hệ thống có khả năng tăng tốc đáng kể quá trình tính toán, đồng thời đảm bảo thực hiện chính xác các phép toán đặc trưng trong mạng nơ-ron tích chập như phép tích chập (convolution), kích hoạt phi tuyến (activation) và gộp đặc trưng (pooling) trong từng lớp của EfficientNet.

1.5 Kế hoạch thực hiện

Phần này trình bày kế hoạch triển khai đề tài theo từng giai đoạn cụ thể, bao gồm nội dung công việc và thời gian thực hiện dự kiến.

Bảng 1.3 Kế hoạch thực hiện đề tài

STT	Công việc thực hiện	Thời gian
1	Khảo sát tổng quan công nghệ AI, mạng CNN và nhu cầu triển khai trên thiết bị biên	Tuần 1–2
2	Nghiên cứu công trình EfficientNet, MobileNet và CNN Accelerator trên FPGA	Tuần 1–2
3	Xác định yêu cầu chức năng và chỉ tiêu kỹ thuật cho hệ thống đề xuất	Tuần 1–2
4	Tìm hiểu chi tiết mô hình MobileNet, EfficientNet và lớp đặc trưng của EfficientNetV2	Tuần 1–2
5	Tìm hiểu cách triển khai CNN trên FPGA, đề xuất kiến trúc phần cứng hệ thống	Tuần 3–4
6	Phân tích thông số đánh giá: FPS, GOPS, LUT, FF, BRAM, Power	Tuần 3–4
7	Nghiên cứu giao tiếp AXI4: AXI4-Lite, AXI4-Full	Tuần 5
8	Đề xuất thiết kế tổng thể bộ tăng tốc CNN tích hợp SoC	Tuần 6
9	Thiết kế khối tích chập, Pooling	Tuần 7–8
10	Thiết kế khối Fused Function Unit, MB Function Unit	Tuần 9–14
11	Thiết kế khối AXI master giao tiếp bộ nhớ ngoài	Tuần 14
12	Tạo sơ đồ khối (Block Design) trong Vivado	Tuần 12–15
13	Mô phỏng, kiểm thử chức năng hệ thống	Tuần 7–15
14	Đo thông lượng, tài nguyên, năng lượng tiêu thụ	Tuần 16
15	Tổng hợp, phân tích kết quả	Tuần 17
16	Viết báo cáo, hoàn thiện mô hình minh họa	Tuần 16–17

1.6 Kết luận chương

Chương 1 đã trình bày tổng quan đề tài, từ phạm vi nghiên cứu, lý do lựa chọn đến các yêu cầu kỹ thuật. Qua phân tích hiện trạng và phương án triển khai, nhóm đã xác định được hướng phát triển phù hợp và đề ra kế hoạch thực hiện làm nền tảng cho các chương tiếp theo.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương 2 trình bày các vấn đề liên quan đến nền tảng lý thuyết trong việc thiết kế bộ gia tốc cho mạng neural tích chập (Convolutional Neural Network – CNN). Nội dung chương bao gồm phần giới thiệu tổng quan về CNN, kiến trúc cơ bản, nguyên lý hoạt động, cũng như phân tích các ưu điểm và nhược điểm của mạng CNN trong các ứng dụng thực tế. Tiếp theo, chương này trình bày lý do ra đời, đặc điểm kiến trúc và đánh giá ưu nhược điểm của hai mô hình CNN hiệu quả hiện nay là EfficientNet và MobileNet. Bên cạnh đó, chương còn đề cập đến quy trình tích hợp bộ gia tốc CNN vào hệ thống trên chip (SoC), bao gồm các thành phần như nền tảng FPGA, giao tiếp AXI Bus, hệ thống bộ nhớ, cũng như các giao tiếp nội bộ và ngoại vi. Ngoài ra, chương này cũng giới thiệu phần mềm thiết kế phần cứng được sử dụng trong quá trình triển khai là Xilinx Vivado.

2.1 Mạng Neural tích chập

2.1.1 Tổng quan về mạng Neural tích chập

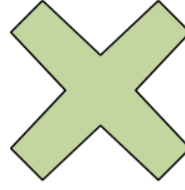
Mạng Neural tích chập (CNN) là một trong những mô hình nổi bật trong lĩnh vực học sâu (deep learning), đặc biệt là trong các bài toán xử lý ảnh và thị giác máy tính. CNN được thiết kế để khai thác cấu trúc không gian của dữ liệu đầu vào như hình ảnh hai chiều hoặc chuỗi dữ liệu một chiều bằng cách sử dụng các phép tích chập (convolution) và các tầng học đặc trưng tự động.

Khác với mạng neural truyền thống (fully connected network), nơi mỗi neuron kết nối với toàn bộ đầu vào, CNN sử dụng các bộ lọc (filter hoặc kernel) có kích thước nhỏ và trượt qua dữ liệu đầu vào để trích xuất đặc trưng cục bộ. Điều này giúp giảm đáng kể số lượng tham số của mô hình và gia tăng khả năng tổng quát hóa.

Cấu trúc tổng quát của một CNN bao gồm nhiều lớp được xếp chồng lên nhau theo thứ tự nhất định. Trong đó, các lớp tích chập (convolutional layers) (Hình 2.1) chịu trách nhiệm học các đặc trưng từ cơ bản đến phức tạp. Ban đầu là các đặc trưng đơn giản như cạnh, góc, sau đó là các thành phần hình học phức tạp hơn.

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

5 x 5 - Image Matrix



1	0	1
0	1	1
0	1	0

3 x 3 - Kernal Matrix

Hình 2.1 Lớp Tích chập

Để hiểu rõ hơn về cơ chế hoạt động của CNN, cần làm rõ một số thuật ngữ quan trọng sau:

Công thức tính toán của lớp tích chập (Convolution):

Giả sử X là đầu vào có kích thước $H \times W \times C$ (H là chiều cao, W là chiều rộng và C là số kênh), K là kernel có kích thước $f \times f \times C$ (f là kích thước của kernel) và Y là đầu ra có kích thước $H' \times W' \times D$ (D là số lượng feature maps).

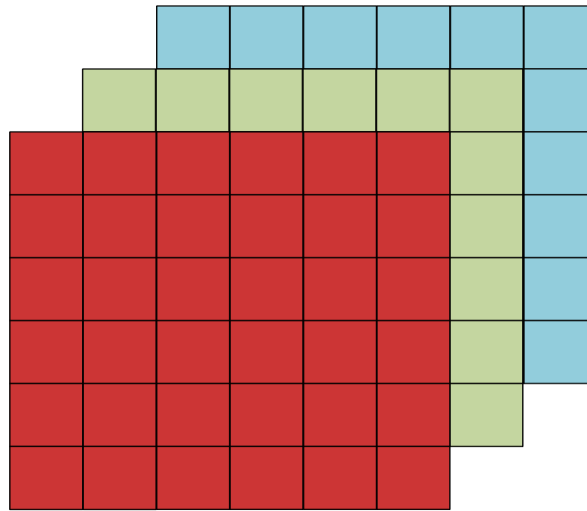
Công thức của phép toán tích chập có thể được viết như sau:

$$Y(i, j) = \sum_{m=1}^f \sum_{n=1}^f \sum_{c=1}^C X(i+m-1, j+n-1, c) \cdot W(m, n, c) \quad (2.1)$$

Trong đó:

- $X(i, j, c)$ là giá trị của pixel tại vị trí (i, j) trong ảnh đầu vào tại kênh c ; với i và j có giá trị từ 1 đến H và W (chiều cao và chiều rộng của ảnh đầu vào) và c là kênh của ảnh đầu vào, có giá trị từ 1 đến C (số lượng kênh của ảnh đầu vào).
- $W(m, n, c)$ là trọng số trong kernel tại vị trí (m, n) và kênh c . với m và n có giá trị chạy từ 1 đến f (kích thước của kernel) và c là kênh của kernel tương ứng, giá trị chạy từ 1 đến C .
- $Y(i, j)$ là giá trị tại điểm ảnh (i, j) của feature map đầu ra. i và j có giá trị từ 1 đến H' và W' (chiều cao và chiều rộng của feature map đầu ra). $Y(i, j)$ đại diện cho điểm ảnh tại vị trí (i, j) trong output của phép tính tích chập.
- f là kích thước của kernel, tức là chiều dài hoặc chiều rộng của kernel.

- **Kernel (Bộ lọc) (Hình 2.2):** Là ma trận trọng số nhỏ được sử dụng để quét qua ảnh đầu vào nhằm trích xuất các đặc trưng. Kích thước phổ biến là 3×3 , 5×5 , ...



6x6x3

Hình 2.2 Kernel

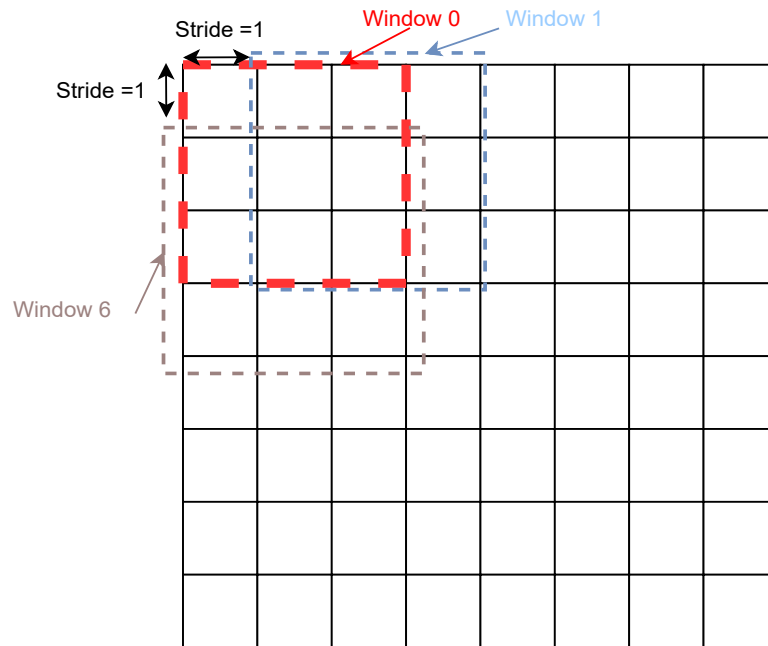
- **Stride (Bước nhảy) (Hình 2.3):** Chỉ số bước dịch chuyển của kernel khi trượt trên ảnh đầu vào. Stride càng lớn thì kích thước đầu ra càng nhỏ. Hình ảnh 2.3 minh họa về stride trong mạng neuron tích chập (CNN). Stride là bước nhảy của cửa sổ bộ lọc (kernel) khi di chuyển qua ảnh đầu vào.

Window là những phần nhỏ được bộ lọc quét qua. Mỗi "window" ứng với một lần tính toán tích chập.

Trong hình, stride = 1 có nghĩa là bộ lọc sẽ di chuyển qua ảnh đầu vào một ô mỗi lần, tức là mỗi lần di chuyển theo hàng ngang, bộ lọc sẽ quét qua một phần tử ảnh mới ngay cạnh phần tử trước đó (window 0, window 1) và khi xuống dòng bộ lọc cũng sẽ di chuyển xuống dòng một ô (window 0 và window 6). Điều này có thể thấy qua các cửa sổ được đánh dấu màu đỏ (Window 0, Window 1) với độ lệch chỉ 1 ô.

Khi bước nhảy (stride) lớn hơn 1, bộ lọc sẽ nhảy qua nhiều ô, tức là bỏ qua một số phần tử đầu vào trong quá trình tính toán, làm giảm độ phân giải của ảnh đầu ra và có thể giúp giảm chi phí tính toán.

Với stride = 1, ta đảm bảo tính toán hết tất cả các phần tử trong ảnh đầu vào, không bỏ sót phần tử nào trong ảnh. Điều này có thể làm cho ảnh đầu ra có độ phân giải cao hơn.



Hình 2.3 Bước nhảy stride

- **Padding (Đệm biên):** Kỹ thuật thêm pixel (thường là 0) vào biên của ảnh đầu vào để giữ nguyên kích thước đầu ra hoặc kiểm soát kích thước kết quả tích chập. Hình 2.4 trên minh họa quá trình đệm biên dữ liệu (Padding) trong mạng neuron tích chập (CNN). Đệm biên là một kỹ thuật được sử dụng để thêm các giá trị vào biên của ảnh đầu vào, giúp điều chỉnh kích thước của ảnh đầu vào khi qua các lớp tích chập.

Trong hình, các ô bên ngoài của ma trận (biên) được điền bằng giá trị 0. Đây là quá trình đệm biên (Padding), trong đó các giá trị tại biên được thêm vào để làm tăng kích thước của ảnh đầu vào mà không thay đổi độ phân giải.

Mục đích của đệm biên là để đảm bảo rằng các pixel ở biên của ảnh đầu vào cũng được xử lý bởi bộ lọc trong quá trình tích chập. Nếu không có đệm, các phần tử ở biên có thể bị bỏ qua, dẫn đến mất thông tin quan trọng.

Trong trường hợp này, ảnh đầu vào có kích thước 4x4 và các giá trị ở biên (được đệm bằng 0) làm tăng kích thước ảnh lên 6x6. Quá trình này đảm bảo rằng bộ lọc có thể áp dụng lên tất cả các pixel trong ảnh, bao gồm cả các pixel ở biên, mà không cần phải giảm độ phân giải của ảnh sau mỗi phép tích chập.

0	0	0	0	0	0	0	0
0							0
0							0
0							0
0							0
0							0
0							0
0	0	0	0	0	0	0	0

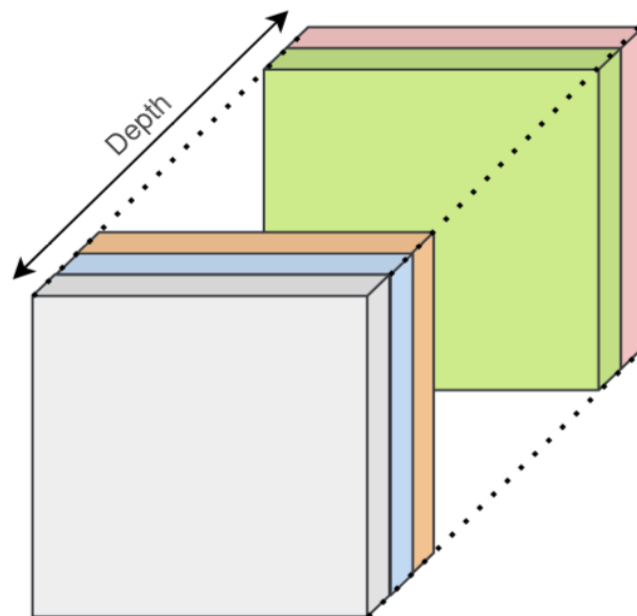
Hình 2.4 Đệm dữ liệu biên

- **Feature Map (Bản đồ đặc trưng):** là kết quả đầu ra của lớp tích chập trong mạng nơ-ron tích chập (CNN), thể hiện các đặc trưng không gian trích xuất từ dữ liệu đầu vào. Sau khi dữ liệu đầu vào (như ảnh) được quét qua bộ lọc (kernel), mỗi lớp tích chập sẽ trích xuất các đặc trưng cục bộ, như cạnh hoặc góc, từ ảnh. Feature Maps thể hiện các đặc trưng này tại mỗi điểm trong ảnh, giúp mạng nhận diện các mẫu cấu trúc đơn giản hoặc phức tạp tùy vào lớp và độ sâu của mạng. Chúng giúp giảm số lượng tham số cần học, tăng khả năng tổng quát hóa và giảm độ phức tạp của mô hình
- **Pooling (Lớp gộp):** Lớp pooling là một thành phần quan trọng trong mạng nơ-ron tích chập (CNN), giúp giảm kích thước của bản đồ đặc trưng (feature map) và giảm độ phức tạp tính toán, từ đó giảm thiểu chi phí tính toán và số lượng tham số trong mô hình. Pooling giúp giữ lại các đặc trưng quan trọng trong dữ liệu đầu vào mà không cần phải học thêm các tham số. Hai phương pháp pooling phổ biến nhất là:
 - **Max Pooling (Gộp giá trị lớn nhất):** Lựa chọn giá trị lớn nhất trong một vùng nhỏ của feature map. Max pooling giúp mô hình học các đặc trưng mạnh mẽ và giữ lại các thông tin quan trọng nhất từ dữ liệu đầu vào.
 - **Average Pooling (Gộp giá trị trung bình):** Tính giá trị trung bình trong một vùng nhỏ của feature map. Average pooling giúp giữ lại thông tin tổng thể của dữ liệu mà không quá tập trung vào các đặc trưng mạnh mẽ.

Tác dụng của lớp pooling là giảm kích thước của bản đồ đặc trưng, giảm độ phức tạp tính toán và tăng tính bất biến đối với sự thay đổi vị trí của các đặc trưng trong ảnh đầu vào.

- **Depth (Độ sâu):** Depth (Độ sâu) trong mạng nơ-ron tích chập (CNN) đề cập đến số lượng bản đồ đặc trưng (feature maps) được tạo ra bởi các bộ lọc (kernel) khác nhau tại mỗi lớp tích chập. Mỗi bộ lọc sẽ tạo ra một feature map riêng biệt và tổng số feature maps này quyết định độ sâu của lớp tích chập.

Trong hình 2.5 minh họa, mỗi màu khác nhau đại diện cho một bản đồ đặc trưng (feature map) riêng biệt, được sinh ra bởi một kernel khác nhau trong quá trình tính toán. Độ sâu này giúp mạng học được nhiều đặc trưng khác nhau của dữ liệu đầu vào, từ các đặc trưng đơn giản như cạnh cho đến các đặc trưng phức tạp hơn ở các lớp sâu hơn của mạng.



Hình 2.5 Độ sâu

- **Hàm kích hoạt** là thành phần quan trọng trong mạng nơ-ron tích chập (CNN), giúp đưa tính phi tuyến vào mô hình, từ đó cải thiện khả năng học và biểu diễn các đặc trưng phức tạp trong dữ liệu. Nếu không có hàm kích hoạt, mạng chỉ có thể học các hàm tuyến tính, hạn chế khả năng xử lý các mối quan hệ phức tạp.

Các hàm kích hoạt phổ biến trong CNN bao gồm:

- **ReLU (Rectified Linear Unit):** Hàm ReLU giúp giảm vấn đề biến mất gradient, với công thức:

$$f(x) = \max(0, x) \quad (2.2)$$

- **Sigmoid:** Hàm sigmoid có giá trị đầu ra trong khoảng (0, 1), dùng trong các mạng phân loại nhị phân:

$$f(x) = \frac{1}{1 + e^{-x}} \quad (2.3)$$

- **Tanh (Hyperbolic Tangent):** Hàm tanh có giá trị đầu ra trong khoảng (-1, 1), công thức tính:

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2.4)$$

- **Swish:** Hàm Swish giúp cải thiện hiệu suất huấn luyện và kết quả phân loại, công thức:

$$f(x) = x \cdot \sigma(x) \quad (2.5)$$

với $\sigma(x)$ là hàm sigmoid.

Hàm kích hoạt được chọn có ảnh hưởng lớn đến hiệu suất của mạng nơ-ron. ReLU, với ưu điểm đơn giản và hiệu quả, được sử dụng phổ biến trong hầu hết các mạng CNN. Tuy nhiên, đối với các mô hình phức tạp hơn như EfficientNet hoặc các mô hình mới hơn, hàm Swish thường được sử dụng để cải thiện hiệu suất.

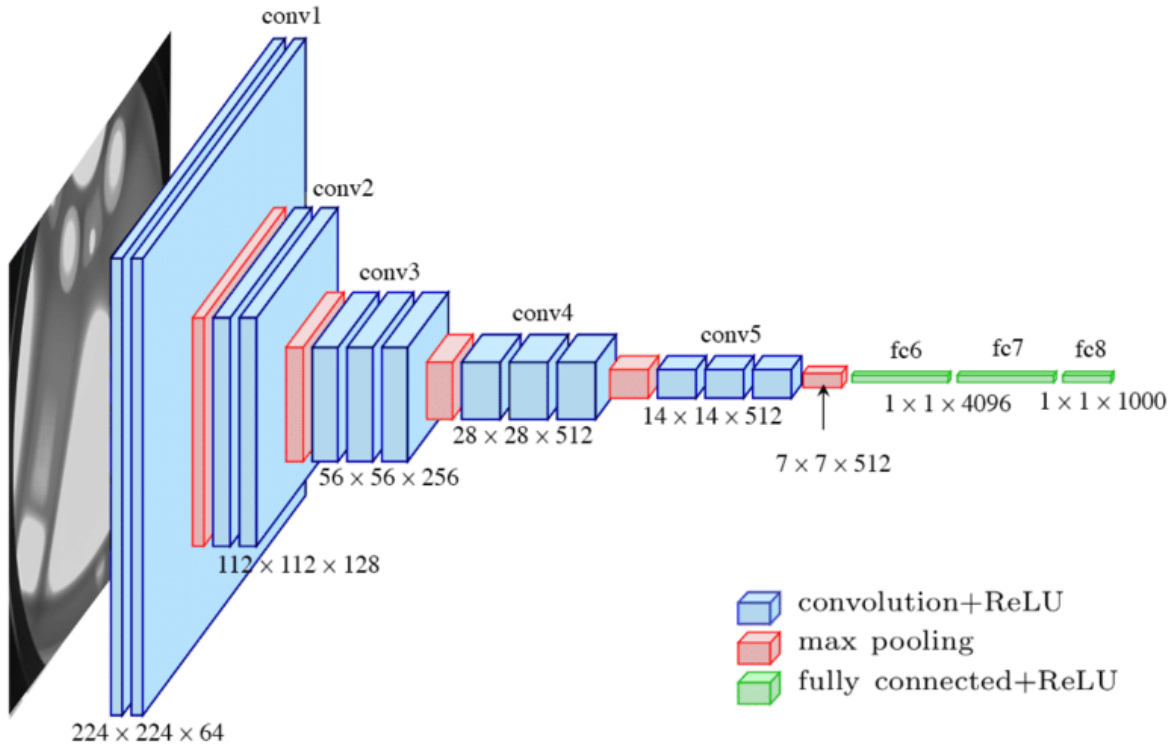
Mạng nơ-ron tích chập (CNN) thường được sử dụng rộng rãi trong các ứng dụng như phân loại ảnh (*image classification*), phát hiện đối tượng (*object detection*), phân đoạn ảnh (*image segmentation*), nhận dạng khuôn mặt, đọc chữ viết tay và nhiều lĩnh vực khác. Ngoài ra, CNN còn được mở rộng để xử lý âm thanh, tín hiệu thời gian và văn bản bằng cách tiền xử lý dữ liệu về dạng lưới.

Một số kiến trúc CNN tiêu biểu như LeNet-5 [6], AlexNet [7], VGGNet [8], GoogLeNet [9] và ResNet [10]. Các mô hình này đã thúc đẩy mạnh mẽ hiệu năng của CNN trong thực tiễn.

CNN hiện nay không chỉ là công cụ phổ biến trong nghiên cứu mà còn được triển khai rộng rãi trên các thiết bị nhúng và hệ thống trên chip (SoC), nhờ vào khả năng thực hiện các phép toán song song và tối ưu tài nguyên phần cứng.

2.1.2 Kiến trúc CNN cơ bản

Cấu trúc cơ bản của mạng CNN có thể chia thành hai giai đoạn chính: *trích xuất đặc trưng* và *phân loại*. Trong giai đoạn trích xuất, hình ảnh đầu vào được đưa qua một chuỗi các khối tích chập, mỗi khối gồm các thành phần: convolution → activation → (batch normalization) → pooling. Giai đoạn này được thể hiện ở hình 2.6 với các lớp như conv1 đến conv5 và lớp max pooling. Giai đoạn phân loại sẽ biến các bản đồ đặc trưng (feature maps) thành vector và đưa qua các lớp kết nối đầy đủ (fully connected) và qua hàm ReLU để đưa ra dự đoán cuối cùng (Hình 2.6 thể hiện một ví dụ cho ảnh đầu vào với kích thước 224x224x3).



Hình 2.6 Luồng dữ liệu của mạng CNN

Các thành phần chính trong mô hình mạng nơ-ron tích chập:

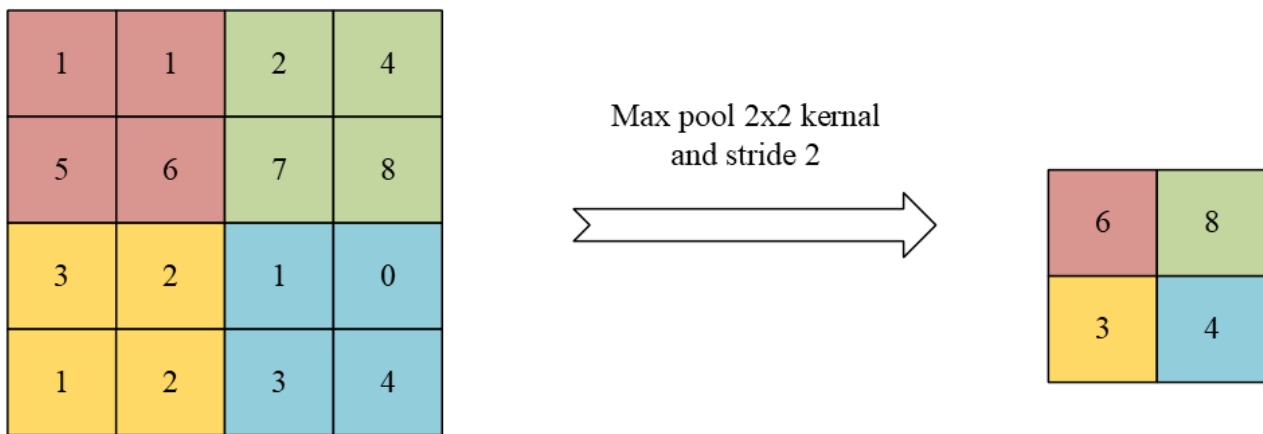
Lớp tích chập (Convolutional Layer) Mỗi lớp convolution sử dụng K bộ lọc (kernel) có kích thước $k_h \times k_w$, với sai bước (stride) s và đệm (padding) p , để tạo ra K bản đồ đặc trưng. Kích thước đầu ra được tính bởi:

$$H_{\text{out}} = \left\lfloor \frac{H - k_h + 2p}{s} \right\rfloor + 1 \quad W_{\text{out}} = \left\lfloor \frac{W - k_w + 2p}{s} \right\rfloor + 1 \quad (2.6)$$

Batch Normalization và Dropout Batch Normalization thường được đặt ngay sau lớp tích chập để chuẩn hóa đầu vào của mỗi lô nhỏ (minibatch), giúp giảm sự thay đổi phân phối nội bộ (internal covariate shift) và cho phép sử dụng tốc độ học cao hơn.

Dropout được dùng tại các lớp kết nối đầy đủ để giảm tình trạng quá khớp (overfitting) bằng cách “tắt” ngẫu nhiên một phần neuron trong quá trình huấn luyện.

Lớp Pooling Max-Pooling làm giảm độ phân giải không gian bằng cách chọn giá trị lớn nhất trong từng cửa sổ $p \times p$ (Hình 2.7).



Hình 2.7 Lớp Pooling

Flatten và Fully Connected Sau chuỗi convolution–pooling, các bản đồ đặc trưng 3D được *làm phẳng* (flatten) thành vector 1D, sau đó đi qua các lớp kết nối đầy đủ.

Lớp đầu ra và hàm Softmax Lớp cuối cùng áp dụng hàm **Softmax** để chuyển đổi logits thành xác suất.

Ví dụ kiến trúc mẫu

Đầu vào Ảnh kích thước $224 \times 224 \times 3$.

Khối 1 ($\times 2$) Conv(3×3 , 64) $\rightarrow$ ReLU $\rightarrow$ BN $\rightarrow$ MaxPool(2×2).

Khối 2 ($\times 2$) Conv(3×3 , 128) $\rightarrow$ ReLU $\rightarrow$ BN $\rightarrow$ MaxPool(2×2).

Phân loại FC(1024) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5) $\rightarrow$ FC(C) $\rightarrow$ Softmax.

Mở rộng và tối ưu trên phần cứng

- **Residual Connection (ResNet):** Chống suy giảm gradient [10].
- **Inception Module:** Kết hợp nhiều kích thước bộ lọc song song [9].
- **Depthwise Separable Convolution:** Giảm phép tính (FLOPs) cho thiết bị di động [11].

Khi triển khai trên hệ thống nhúng, việc tối ưu số lượng tham số, kích thước feature maps và FLOPs là rất quan trọng.

2.1.3 Ưu điểm và nhược điểm của CNN

Mặc dù Mạng nơ-ron tích chập (CNN) đã cách mạng hóa lĩnh vực thị giác máy tính, kiến trúc này vẫn tồn tại những ưu điểm và hạn chế cố hữu. Việc hiểu rõ các khía cạnh này là rất quan trọng để có thể lựa chọn và áp dụng mô hình một cách hiệu quả trong các bài toán thực tế.

Ưu điểm nổi bật

- **Tự động trích xuất đặc trưng (Automatic Feature Extraction):** Đây là ưu điểm mạnh mẽ nhất của CNN. Thay vì phải thiết kế các bộ trích xuất đặc trưng thủ công (hand-crafted features) vốn tốn nhiều thời gian và kiến thức chuyên môn, CNN có khả năng tự động học các đặc trưng phân cấp trực tiếp từ dữ liệu ảnh. Các lớp đầu học các đặc trưng đơn giản như cạnh, góc, màu sắc, trong khi các lớp sau tổng hợp chúng thành các cấu trúc phức tạp hơn như bộ phận của vật thể và cuối cùng là toàn bộ vật thể. Quá trình này đã được chứng minh hiệu quả vượt trội kể từ thành công của AlexNet [12].
- **Chia sẻ tham số (Parameter Sharing):** Trong một lớp tích chập, một bộ lọc (kernel) được áp dụng cho tất cả các vùng của ảnh đầu vào. Cơ chế này làm giảm đáng kể tổng số lượng tham số so với mạng kết nối đầy đủ (Fully Connected Network). Việc này không chỉ giúp giảm chi phí tính toán mà còn có tác dụng như một cơ chế điều chuẩn (regularization), hạn chế hiện tượng quá khớp (overfitting) một cách hiệu quả.
- **Bất biến với các phép biến đổi nhỏ (Translation Invariance):** Nhờ vào lớp gộp (pooling layer), CNN có khả năng bất biến ở mức độ nhất định với các phép dịch chuyển nhỏ, co giãn, hoặc xoay của đối tượng. Điều này cho phép mạng nhận ra một đối tượng ngay cả khi vị trí của nó không hoàn toàn giống với dữ liệu trong tập huấn luyện, một đặc tính cực kỳ quan trọng trong các bài toán nhận dạng vật thể.
- **Hiệu năng vượt trội:** CNN và các biến thể của nó hiện đang là kiến trúc tân tiến nhất (state-of-the-art) cho hầu hết các bài toán thị giác máy tính, từ phân loại ảnh, phát hiện vật thể, cho đến phân tích ảnh y tế.

Nhược điểm và Thách thức

- **Yêu cầu lượng lớn dữ liệu huấn luyện:** CNN là một mô hình "đói" dữ liệu (data-hungry). Để huấn luyện một mạng sâu từ đầu, cần một tập dữ liệu cực lớn và được gán nhãn cẩn thận, ví dụ như ImageNet [5]. Việc thu thập và gán nhãn dữ liệu như vậy rất tốn kém. Các kỹ thuật như học chuyển giao (transfer learning) và tăng cường dữ liệu (data augmentation) có thể giúp giảm nhẹ phần nào vấn đề này.
- **Chi phí tính toán cao:** Việc huấn luyện và suy luận trên các mô hình CNN sâu đòi hỏi tài nguyên tính toán khổng lồ, đặc biệt là các bộ xử lý đồ họa (GPU) mạnh mẽ. Điều này tạo ra rào cản về chi phí và năng lượng, đặc biệt khi triển khai trên các thiết bị có tài nguyên hạn chế.
- **Thiếu khả năng diễn giải (Tính "hộp đen"):** CNN thường bị xem là một mô hình "hộp đen" (black box) vì rất khó để diễn giải một cách chính xác tại sao nó lại đưa ra một quyết

định cụ thể. Mặc dù các kỹ thuật hiển thị hóa như Class Activation Maps (CAM) có thể cho thấy vùng ảnh nào được mô hình "chú ý" tới, chúng vẫn không giải thích được toàn bộ logic suy luận.

- **Hạn chế trong việc nắm bắt mối quan hệ không gian phức tạp:** Do hoạt động dựa trên các vùng lân cận cục bộ, CNN truyền thống gặp khó khăn trong việc hiểu các mối quan hệ không gian ở phạm vi toàn cục. Các kiến trúc mới hơn như Vision Transformer (ViT) và Capsule Networks đang được nghiên cứu để khắc phục hạn chế này.

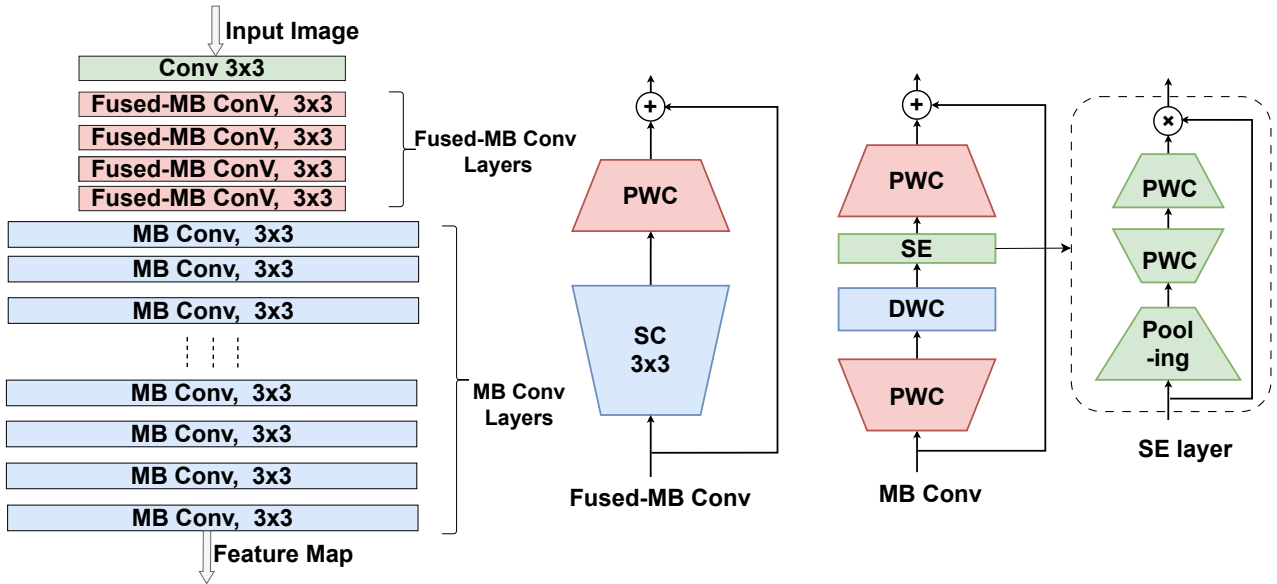
2.2 MobileNet và EfficientNet

2.2.1 Sự ra đời của mạng MobileNet và EfficientNet

MobileNet[13] được giới thiệu lần đầu tiên vào năm 2017 bởi Google, với mục tiêu tối ưu hóa các mô hình học sâu cho các thiết bị di động và các hệ thống nhúng có tài nguyên hạn chế. MobileNet áp dụng một kỹ thuật gọi là Depthwise Separable Convolutions, giúp giảm số lượng tham số và phép toán tính toán mà vẫn duy trì độ chính xác cao. Điều này giúp mô hình có thể chạy mượt mà trên các thiết bị di động hoặc các hệ thống nhúng có bộ xử lý yếu và bộ nhớ hạn chế.

EfficientNet[14], được giới thiệu vào năm 2019, là một sự cải tiến lớn so với các mô hình CNN truyền thống. EfficientNet sử dụng phương pháp neural architecture search (NAS) để tối ưu hóa cấu trúc mạng và compound scaling để đồng thời mở rộng ba yếu tố quan trọng trong mô hình: độ sâu (depth), chiều rộng (width) và độ phân giải (resolution). Với phương pháp này, EfficientNet đạt được độ chính xác cao hơn các mô hình trước đó, nhưng vẫn tiết kiệm tài nguyên tính toán, giúp triển khai hiệu quả hơn trên các hệ thống mạnh mẽ và hạn chế tài nguyên.

Hình 2.8 minh họa kiến trúc tổng quát của EfficientNetV2 [15], cho thấy sự kết hợp giữa hai loại block chính là Fused-MB Conv và MB Conv, được tổ chức theo từng tầng từ đầu vào đến khi trích xuất đặc trưng (feature map).



Hình 2.8 Kiến trúc mạng EfficientNet-V2.

- **Fused-Conv 3x3** là tập hợp các Fused-Conv blocks, được sử dụng ở các tầng đầu nhằm giảm độ phức tạp và tăng tốc xử lý. Mỗi block gồm một lớp tích chập 3x3 (SC 3x3), theo sau là hàm kích hoạt và một lớp tích chập điểm (PWC).

1. Lớp **Standard Convolution (SC 3x3)** (tích chập chuẩn) thực hiện phép tích chập với bộ lọc kích thước 3x3 và công thức tính tích chập cho lớp SC 3x3 được cho bởi:

$$Y_{i,j} = \sum_{m=0}^2 \sum_{n=0}^2 X_{i+m,j+n} \cdot W_{m+1,n+1} \quad (2.7)$$

Trong đó:

- X là ma trận đầu vào (input), W là ma trận trọng số (weight),
- $Y_{i,j}$ là giá trị của feature map đầu ra tại vị trí (i, j) ,
- m, n là các chỉ số của bộ lọc 3x3.

2. Lớp **tích chập điểm (PWC)** thực hiện phép tích chập với bộ lọc 1x1 trên các đặc trưng không gian của đầu vào. Công thức tính tích chập cho lớp PWC là:

$$Y_{i,j} = \sum_{c=0}^C X_{i,j,c} \cdot W_c \quad (2.8)$$

Trong đó:

- $X_{i,j,c}$ là giá trị tại vị trí (i, j) của feature map đầu vào ở kênh thứ c ,
- W_c là trọng số cho kênh thứ c ,
- $Y_{i,j}$ là giá trị của feature map đầu ra tại vị trí (i, j) .

- Tiếp đến là các khối **MB Conv 3x3** (Mobile Inverted Bottleneck Convolution) đặc trưng của EfficientNet. Mỗi MB Conv bao gồm:
- **Lớp tích chập điểm đầu vào (PWC)** có công thức tương tự 2.8:

$$Y_{i,j} = \sum_{c=0}^C X_{i,j,c} \cdot W_c \quad (2.9)$$

Trong đó:

- $X_{i,j,c}$ là giá trị tại vị trí (i, j) của feature map đầu vào ở kênh thứ c ,
- W_c là trọng số cho kênh thứ c ,
- $Y_{i,j}$ là giá trị của feature map đầu ra tại vị trí (i, j) .

- **Lớp tích chập chiều sâu (DepthWise Convolution - DWC):**

$$Y_{i,j,c} = \sum_{m=0}^2 \sum_{n=0}^2 X_{i+m,j+n,c} \cdot W_{m+1,n+1,c} \quad (2.10)$$

Trong đó:

- $X_{i+m,j+n,c}$ là giá trị tại vị trí $(i+m, j+n)$ của feature map đầu vào ở kênh thứ c ,
- $W_{m+1,n+1,c}$ là trọng số tại vị trí $(m+1, n+1)$ trong bộ lọc chiều sâu của kênh c ,
- $Y_{i,j,c}$ là giá trị của feature map đầu ra tại vị trí (i, j) ở kênh c .

- **Khối nén và kích thích (Squeeze-and-Excitation) (SE-layer):**

Lớp SE bao gồm ba phần chính: **Pooling** và hai **lớp tích chập điểm** (PWC)

1. **Squeeze (Global Average Pooling):** Phép toán này thu gọn chiều không gian (height và width) của feature map, chỉ giữ lại các đặc trưng theo chiều kênh. Công thức cho phép toán này là:

$$z_c = \frac{1}{H \times W} \sum_{i=0}^H \sum_{j=0}^W X_{i,j,c} \quad (2.11)$$

Trong đó:

- $X_{i,j,c}$ là giá trị tại vị trí (i, j) của feature map đầu vào ở kênh thứ c ,
- H và W là chiều cao và chiều rộng của feature map,
- z_c là đặc trưng đầu ra cho kênh thứ c sau khi áp dụng global average pooling.

2. **Lớp tích chập điểm thứ nhất (PWC)** lớp này được gọi là lớp suy giảm (reduce layer). Công thức cho phép toán này tương tự như lớp tích chập điểm PWC ở lớp thứ nhất 2.9

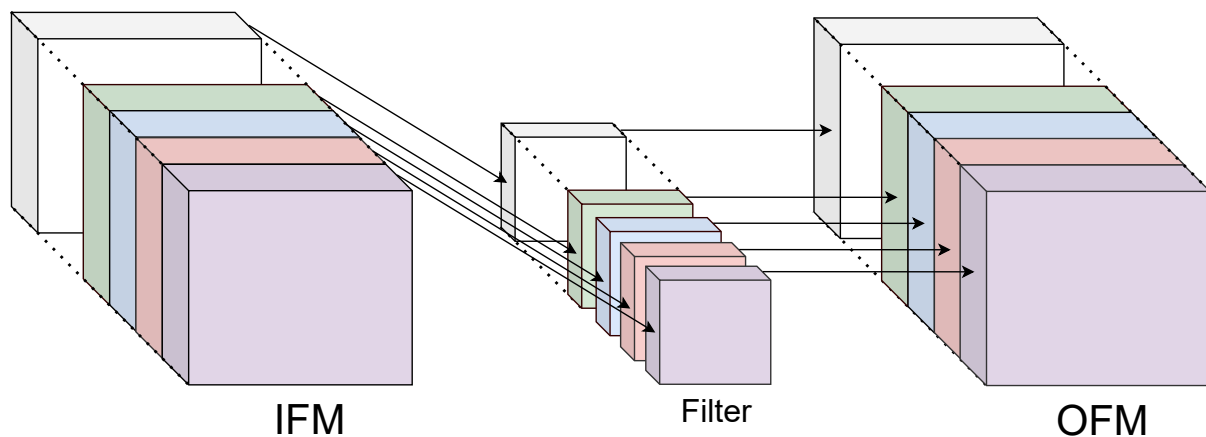
3. **Lớp tích chập điểm thứ hai (PWC)** lớp này được gọi là lớp mở rộng(expand layer). Công thức cho phép toán này tương tự như lớp tích chập điểm PWC ở lớp thứ nhất 2.9

- **Lớp tích chập điểm (PWC)** đầu ra cũng có công thức tương tự như lớp tích chập điểm PWC ở lớp thứ nhất 2.9
- Ở bên phải hình là sơ đồ khối SE, minh họa cách sử dụng pooling và các lớp tích chập điểm liên tiếp nhau để học trọng số quan trọng cho từng channel đầu ra.

Liên hệ với nội dung mục 2.2, hình ảnh này giúp cụ thể hóa cấu trúc EfficientNet, cho thấy rõ:

- Sự phân chia giữa các khối Fused-MB Conv (dành cho tầng đầu) và MB Conv (cho các tầng sâu hơn), là đặc điểm đặc trưng tạo nên hiệu quả cao về hiệu suất và độ chính xác của EfficientNet.

2.2.2 Lớp tích chập Depth wise đặc trưng của mạng MobileNet và Efficient Net



Hình 2.9 Luồng dữ liệu tính toán cho lớp depthwise

Depthwise Convolution là một loại lớp convolution trong các mạng nơ-ron sâu (CNN), đặc biệt là trong các mạng di động như MobileNet. Thay vì áp dụng một bộ lọc (filter) cho toàn bộ các kênh đầu vào như trong convolution thông thường (standard convolution), trong Depthwise Convolution, mỗi bộ lọc (filter) sẽ chỉ áp dụng cho một kênh riêng biệt của đầu vào (IFM) để tạo ra đầu ra tương ứng (OFM). Cách tiếp cận này giúp giảm số lượng tham số và tính toán đáng kể, đặc biệt hữu ích khi triển khai mạng trên các thiết bị có tài nguyên hạn chế như điện thoại di động.

Công thức tính số tham số

Giả sử ta có một đầu vào có kích thước $(H \times W \times C_{in})$ (Hình 2.9) với H là chiều cao, W là chiều rộng và C_{in} là số lượng kênh (channels) đầu vào. Nếu bạn sử dụng một bộ lọc có kích

thước ($K \times K \times C_{in}$) trong một lớp convolution thông thường, số lượng tham số trong bộ lọc sẽ là:

$$\text{Số tham số trong convolution thông thường} = K \times K \times C_{in} \times C_{out}$$

Trong đó C_{out} là số lượng kênh đầu ra.

Tuy nhiên, trong Depthwise Convolution, mỗi bộ lọc chỉ áp dụng cho một kênh đầu vào, vì vậy số lượng tham số trong bộ lọc sẽ là:

$$\text{Số tham số trong Depthwise Convolution} = K \times K \times 1 \times C_{out} = K \times K \times C_{out}$$

Sự khác biệt về tham số giữa Depthwise Convolution và Standard Convolution

1. Trong Convolution thông thường (Standard Convolution):

$$\text{Số tham số} = K \times K \times C_{in} \times C_{out}$$

2. Trong Depthwise Convolution:

$$\text{Số tham số} = K \times K \times C_{in} + C_{out}$$

Như vậy, số tham số trong Depthwise Convolution sẽ ít hơn rất nhiều so với Convolution thông thường, vì mỗi bộ lọc chỉ cần xử lý một kênh riêng biệt thay vì tất cả các kênh như trong trường hợp của Convolution thông thường.

Lợi ích

Việc giảm số lượng tham số giúp giảm bớt độ phức tạp tính toán và giảm yêu cầu về bộ nhớ, giúp cải thiện tốc độ xử lý và hiệu quả của mô hình khi triển khai trên các thiết bị có tài nguyên hạn chế, như điện thoại di động và các hệ thống nhúng.

2.2.3 Hàm kích hoạt (Activation)

Trong mạng EfficientNet, các hàm kích hoạt (activation functions) đóng vai trò quan trọng trong việc thêm tính phi tuyến vào mô hình, giúp mạng có thể học và mô hình hóa các quan hệ phức tạp. Hai hàm kích hoạt phổ biến được sử dụng trong EfficientNet là **ReLU** (**Rectified Linear Unit**) và **Swish**. Dưới đây là mô tả và công thức của hai hàm này:

- **Hàm kích hoạt ReLU** (Rectified Linear Unit):

ReLU là một trong những hàm kích hoạt phổ biến nhất trong các mạng nơ-ron, bao gồm trong EfficientNet. Hàm ReLU có tác dụng loại bỏ các giá trị âm và giữ lại các giá trị dương, cụ thể:

$$f(x) = \max(0, x) \quad (2.12)$$

Trong đó:

- x là giá trị đầu vào của hàm kích hoạt (có thể là output của một lớp convolution hoặc lớp dense),
- $f(x)$ là giá trị đầu ra sau khi áp dụng hàm ReLU.

ReLU rất phổ biến vì sự đơn giản và khả năng giảm thiểu vấn đề biến mất gradient, giúp tăng tốc quá trình huấn luyện.

• Hàm kích hoạt Swish:

Hàm Swish là một hàm kích hoạt phi tuyến tính mới được Google nghiên cứu và áp dụng trong EfficientNet và đã chứng minh khả năng vượt trội hơn so với ReLU trong nhiều bài toán. Công thức của Swish là:

$$f(x) = x \cdot \sigma(x) \quad (2.13)$$

Trong đó:

- x là giá trị đầu vào của hàm kích hoạt,
- $\sigma(x)$ là hàm sigmoid, được tính theo công thức: $\sigma(x) = \frac{1}{1+e^{-x}}$,
- $f(x)$ là giá trị đầu ra của hàm Swish.

Swish có khả năng cho phép các giá trị âm nhỏ đi qua trong quá trình huấn luyện, giúp mạng học tốt hơn và cải thiện độ chính xác so với ReLU, đặc biệt trong các mô hình phức tạp như EfficientNet.

Tổng quan:

Trong mô hình EfficientNet, ReLU và Swish được sử dụng linh hoạt tùy theo từng tầng trong mạng. ReLU thường được dùng trong các lớp tích chập (convolution) để tăng tốc tính toán, trong khi Swish được áp dụng trong các lớp kết nối đầy đủ (fully connected) để cải thiện độ chính xác của mạng. Cả hai hàm kích hoạt đều giúp mạng EfficientNet học được các đặc trưng quan trọng và tạo ra các dự đoán chính xác.

2.2.4 *Lớp Ép và Kích thích (Squeeze-and-Excitation) (SE) trong MobileNet và EfficientNet*

Lớp Squeeze-and-Excitation (SE) là một kỹ thuật mạnh mẽ được giới thiệu trong các mạng nơ-ron sâu, giúp cải thiện khả năng biểu diễn của mô hình mà không làm tăng đáng kể độ phức tạp tính toán. SE module được áp dụng sau mỗi khối convolution nhằm tối ưu hóa các trọng số đầu ra của mạng, giúp mạng học cách "tinh chỉnh" mức độ quan trọng của từng kênh trong đặc trưng đầu ra. Đây là một cách tiếp cận hiệu quả trong việc cải thiện hiệu suất của mạng mà không cần phải tăng thêm nhiều tham số, đặc biệt hữu ích trong các mạng như MobileNet và EfficientNet, nơi việc tối ưu hóa bộ lọc convolution là rất quan trọng.

Quá trình hoạt động của SE module bao gồm hai bước chính:

- **Squeeze:** Đầu vào của SE module là một tensor đầu ra từ các lớp trước đó của mạng, chứa nhiều kênh. Bước "squeeze" được thực hiện bằng cách áp dụng phép toán global average pooling (GAP) trên mỗi kênh của tensor đầu vào. Sau khi thực hiện GAP, ta thu được một vector có chiều dài bằng số kênh, thể hiện mức độ quan trọng của từng kênh.
- **Excitation:** Vector này sau đó được đưa qua một mạng fully connected (FC) với một hàm kích hoạt sigmoid, qua đó mỗi kênh sẽ nhận được một trọng số điều chỉnh. Kết quả là một vector Excitation, biểu thị mức độ quan trọng của từng kênh trong đặc trưng đầu vào. Mỗi trọng số này sẽ điều chỉnh lại các kênh đầu vào, giúp mạng tập trung vào những đặc trưng quan trọng hơn, từ đó cải thiện khả năng nhận diện của mạng.

Cuối cùng, vector *Excitation* này sẽ được nhân vào đầu vào của mạng, giúp điều chỉnh lại giá trị của các kênh theo mức độ quan trọng của chúng, từ đó tối ưu hóa đầu ra của mạng.

Lớp SE giúp mạng học cách trọng số các kênh theo mức độ quan trọng của chúng, từ đó giúp mạng tập trung vào các đặc trưng quan trọng hơn, đồng thời cải thiện hiệu suất mà không làm tăng quá nhiều số lượng tham số. Điều này đặc biệt hiệu quả trong các mạng như EfficientNet, nơi tối ưu hóa bộ lọc convolution có thể giúp giảm thiểu độ phức tạp tính toán mà vẫn duy trì hiệu quả cao.

2.2.5 *Ưu điểm và nhược điểm của mạng nơ-ron tích chập (CNN) nói chung và EfficientNet nói riêng*

Ưu điểm của mạng nơ-ron tích chập (CNN) so với các mạng thần kinh thông thường khác. Cụ thể

- Khả năng trích xuất đặc trưng mạnh mẽ: CNN có khả năng học và phát hiện các đặc trưng không gian trong ảnh (như cạnh, hình dạng, hoa văn) mà không cần tiền xử lý thủ công, rất hiệu quả trong các bài toán thị giác máy tính.
- Nhờ kiến trúc nhiều lớp, CNN có thể học được biểu diễn phức tạp từ dữ liệu và tổng quát

tốt sang dữ liệu mới, được ứng dụng rộng rãi trong nhận diện khuôn mặt, y tế, giao thông, nông nghiệp,...

- Hiệu quả hơn với EfficientNet: EfficientNet sử dụng phương pháp compound scaling để đồng thời tối ưu chiều sâu, chiều rộng và độ phân giải mô hình, từ đó đạt độ chính xác cao hơn so với các mạng CNN truyền thống với cùng mức tài nguyên.

Song song với ưu điểm mang lại, mạng CNN cũng có những hạn chế:

- Chi phí tính toán lớn: CNN thường đòi hỏi nhiều phép toán tích chập, khiến việc huấn luyện và suy luận chậm trên các thiết bị có tài nguyên hạn chế. Điều này càng nghiêm trọng với các mô hình sâu, rộng hoặc độ phân giải đầu vào cao.
- Khó triển khai trên thiết bị nhúng: Các mô hình CNN truyền thống như VGG hay ResNet thường quá nặng để chạy trên thiết bị di động, dẫn đến nhu cầu phát triển các mô hình nhẹ hơn như MobileNet hay EfficientNet.
- Cấu trúc phức tạp (với EfficientNet): Mặc dù EfficientNet tối ưu tài nguyên tốt, nhưng kiến trúc gồm nhiều khối như MB Conv, SE block và các kết nối shortcut khiến quá trình triển khai trên phần cứng như FPGA trở nên phức tạp và đòi hỏi tối ưu hóa kỹ lưỡng.

2.2.6 Ứng dụng của Mạng EfficientNet

Mạng EfficientNet đã nhanh chóng trở thành một trong những mạng nơ-ron phổ biến nhất trong lĩnh vực học sâu nhờ vào hiệu suất vượt trội và khả năng tiết kiệm tài nguyên. Mạng này được thiết kế với mục tiêu tối ưu hóa độ chính xác trong khi giảm thiểu độ phức tạp tính toán, giúp giảm bớt nhu cầu về bộ nhớ và năng lượng so với các mạng truyền thống. Đặc biệt, EfficientNet sử dụng phương pháp tối ưu hóa quy mô mạng (network scaling) để cải thiện độ chính xác mà không cần phải tăng thêm các tham số.

Các ứng dụng chính của EfficientNet bao gồm:

- **Nhận dạng hình ảnh (Image Classification):** EfficientNet đã đạt được hiệu quả vượt trội trong các bài toán phân loại hình ảnh, chẳng hạn như các bộ dữ liệu ImageNet, nơi mà mạng này đạt được độ chính xác cao với số lượng tham số ít hơn nhiều so với các mạng nơ-ron sâu khác. Việc tối ưu hóa quy mô mạng của EfficientNet cho phép nó hoạt động tốt trên nhiều loại dữ liệu hình ảnh khác nhau, từ các hình ảnh phức tạp đến các hình ảnh đơn giản.
- **Nhận diện vật thể (Object Detection):** Mặc dù EfficientNet được thiết kế ban đầu cho các bài toán phân loại hình ảnh, nhưng mạng này cũng được áp dụng hiệu quả trong các bài toán nhận diện vật thể, ví dụ như với các kiến trúc như EfficientDet. Các cải tiến

trong cấu trúc mạng EfficientNet giúp tăng cường khả năng phát hiện các vật thể trong hình ảnh với tốc độ và độ chính xác cao, đặc biệt trong các ứng dụng yêu cầu khả năng xử lý thời gian thực.

- **Nhận dạng văn bản và phân tích ngữ nghĩa (Text Recognition and Semantic Analysis):** Mạng EfficientNet cũng đã được điều chỉnh và áp dụng vào các bài toán nhận dạng văn bản, chẳng hạn như nhận dạng văn bản viết tay hay nhận dạng chữ viết trong ảnh. Mạng này giúp cải thiện độ chính xác của các mô hình nhận diện văn bản với chi phí tính toán thấp hơn.
- **Ứng dụng di động và thiết bị có tài nguyên hạn chế (Mobile and Embedded Devices):** Nhờ vào khả năng tối ưu hóa hiệu suất và tiết kiệm tài nguyên, EfficientNet rất thích hợp cho các ứng dụng di động và trên các thiết bị có tài nguyên hạn chế như điện thoại thông minh, máy tính nhúng, hoặc các hệ thống Internet of Things (IoT). Các thiết bị này đòi hỏi phải thực hiện các tác vụ nhận dạng hình ảnh hoặc phân tích ngữ nghĩa trong khi vẫn tiết kiệm năng lượng và bộ nhớ.
- **Các ứng dụng trong y tế (Healthcare Applications):** Mạng EfficientNet đã được áp dụng trong các bài toán y tế như chẩn đoán hình ảnh y học (chẳng hạn như chẩn đoán bệnh từ ảnh X-quang hoặc MRI). Khả năng phân loại hình ảnh chính xác và tiết kiệm tài nguyên làm cho EfficientNet trở thành lựa chọn lý tưởng cho các ứng dụng y tế yêu cầu xử lý hình ảnh trong môi trường thực tế với tài nguyên hạn chế.

Với sự kết hợp giữa tính hiệu quả và độ chính xác cao, EfficientNet đang mở rộng khả năng ứng dụng trong các lĩnh vực từ nhận diện hình ảnh đến các thiết bị di động, y tế và nhận dạng văn bản, chứng minh khả năng sử dụng rộng rãi của mạng này trong các bài toán học sâu.

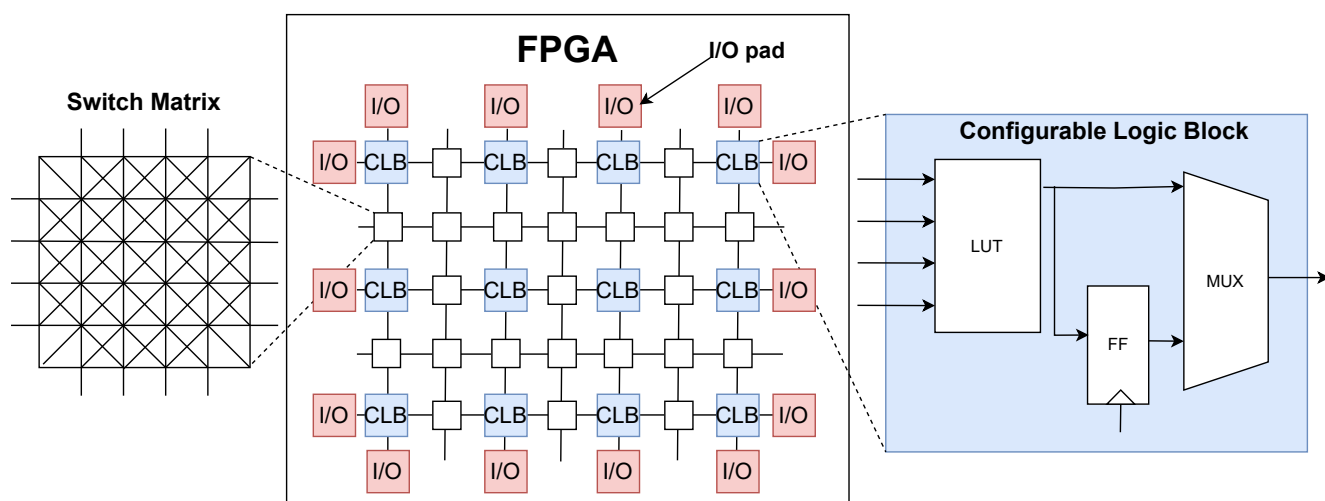
2.3 CNN Accelerator trên FPGA

2.3.1 Tổng quan về FPGA

FPGA (Field Programmable Gate Array) là một loại mạch tích hợp cỡ lớn, bao gồm các khối logic có thể cấu hình, cho phép người dùng lập trình để thực hiện các chức năng số theo yêu cầu. Nhờ khả năng lập trình linh hoạt và đa dạng, FPGA đã trở thành một công cụ quan trọng trong lĩnh vực thiết kế phần cứng số. FPGA được ứng dụng rộng rãi trong nhiều lĩnh vực như xử lý tín hiệu số, hệ thống nhúng, hàng không vũ trụ, viễn thông và tự động hóa công nghiệp.

Kiến trúc cơ bản của một FPGA bao gồm ba thành phần chính, như được minh họa trong Hình 2.10, đó là:

- **Khối logic có thể cấu hình (Configurable Logic Block – CLB)**



Hình 2.10 Kiến trúc của FPGA

- **Ma trận chuyển mạch (Switch Matrix – SM)**
- **Khối vào/ra (I/O Pad)**

CLB (Configurable Logic Block): Đây là thành phần cơ bản và quan trọng nhất trong FPGA. Mỗi CLB thường bao gồm một bộ LUT 4 input, thanh ghi (có thể là flip-flop hoặc latch) và một bộ chọn đa (multiplexer – MUX). CLB được thiết kế để thực hiện nhiều chức năng logic và macro khác nhau. Các đầu vào được bố trí xung quanh các CLB, cho phép khả năng ánh xạ linh hoạt và phân vùng logic hiệu quả trong toàn bộ kiến trúc FPGA.

I/O Pad: Các khối I/O pad chịu trách nhiệm giao tiếp giữa FPGA và các thành phần ngoại vi bên ngoài. Chúng đảm bảo tín hiệu được truyền nhận đúng chuẩn điện áp và tốc độ yêu cầu.

Switch Matrix: Đây là ma trận chuyển mạch cho phép kết nối linh hoạt giữa các CLB thông qua các đường liên kết ngắn (short interconnect) hoặc dài (long interconnect). Bên trong switch matrix chứa các bóng bán dẫn (transistor) đóng vai trò bật/tắt các kết nối tùy theo cấu hình được lập trình, từ đó tạo ra các đường dẫn dữ liệu phù hợp với thiết kế.

2.3.2 *Khó khăn khi triển khai CNN trên CPU và GPU tại thiết bị biên*

Việc triển khai các mạng nơ-ron sâu (CNN) trên các thiết bị biên (edge devices) như điện thoại thông minh, máy tính nhúng, hoặc các hệ thống IoT đối mặt với nhiều thử thách do hạn chế về tài nguyên tính toán và bộ nhớ. Các thiết bị này không chỉ yêu cầu khả năng xử lý mạnh mẽ mà còn cần tiết kiệm năng lượng, vì vậy việc triển khai CNN trên các CPU và GPU của các thiết bị biên gặp phải một số khó khăn như sau:

- **Tài nguyên tính toán hạn chế:** Các CPU và GPU của thiết bị biên thường có sức mạnh

tính toán thấp hơn nhiều so với các máy chủ hoặc trung tâm dữ liệu. Các phép toán phức tạp trong CNN, đặc biệt là trong các lớp convolution, yêu cầu một lượng lớn tài nguyên tính toán, điều này dẫn đến việc các mạng CNN truyền thống gặp khó khăn khi triển khai trên các thiết bị này.

- **Hạn chế về bộ nhớ:** CNN yêu cầu bộ nhớ lớn để lưu trữ các trọng số của mạng, đặc biệt là trong các mô hình có số lượng lớp lớn hoặc mạng có nhiều tham số. Hầu hết các thiết bị biên có bộ nhớ hạn chế, điều này khiến cho việc tải và xử lý dữ liệu trở thành một thách thức lớn, dẫn đến việc hiệu suất giảm mạnh khi triển khai CNN.
- **Tiêu thụ năng lượng cao:** Các phép toán trong CNN tiêu tốn nhiều tài nguyên tính toán, dẫn đến mức tiêu thụ năng lượng cao. Điều này rất khó khăn đối với các thiết bị di động hoặc IoT, nơi việc tiết kiệm năng lượng là rất quan trọng. Các GPU, mặc dù có thể xử lý tốt các phép toán nặng, nhưng lại tiêu thụ rất nhiều năng lượng, không phù hợp cho các ứng dụng trên thiết bị biên.
- **Tốc độ xử lý và độ trễ cao:** Do các hạn chế về tài nguyên tính toán và bộ nhớ, việc xử lý dữ liệu và chạy các mô hình CNN trên các thiết bị biên có thể dẫn đến độ trễ cao. Điều này đặc biệt quan trọng trong các ứng dụng yêu cầu thời gian phản hồi nhanh như nhận dạng hình ảnh trong thời gian thực hoặc các ứng dụng giám sát video.
- **Vấn đề về tối ưu hóa:** Việc tối ưu hóa các mô hình CNN để chạy hiệu quả trên thiết bị biên yêu cầu các kỹ thuật như giảm độ chính xác (quantization), cắt bớt mô hình (pruning) và giảm độ phức tạp tính toán (compression). Tuy nhiên, các kỹ thuật này đôi khi gây ra sự suy giảm độ chính xác hoặc làm giảm chất lượng của mô hình, tạo ra sự đánh đổi giữa hiệu suất và độ chính xác.

Mặc dù vậy, các giải pháp như sử dụng các mạng CNN có số lượng tham số nhẹ hơn như MobileNet hoặc EfficientNet, áp dụng quantization và các kỹ thuật tối ưu hóa phần cứng (như sử dụng FPGA hoặc các chip chuyên dụng) đang được phát triển để giải quyết các vấn đề này và mang lại hiệu suất cao hơn trên các thiết bị biên như tối ưu hiệu suất và độ trễ. Sử dụng các phần tử tính toán chuyên biệt (MAC) giúp tăng tốc các phép toán tích chập (CONV), từ đó cải thiện hiệu suất tổng thể. Đồng thời, xây dựng các kiến trúc tính toán đặc trưng hỗ trợ tính toán song song giúp tăng tốc đáng kể quá trình xử lý, giảm thiểu độ trễ và nâng cao khả năng mở rộng, đặc biệt là khi triển khai trên các thiết bị có tài nguyên hạn chế như các hệ thống nhúng và thiết bị biên.

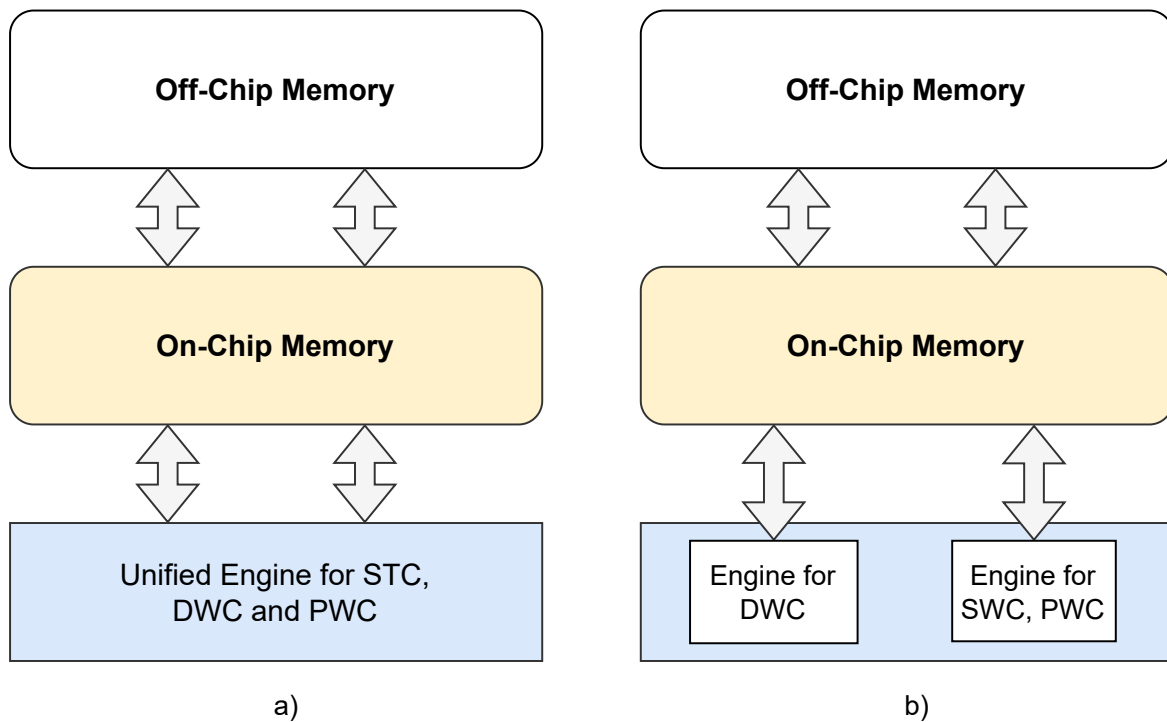
2.3.3 Sự cần thiết của bộ tăng tốc CNN

Dựa trên những hạn chế đã trình bày trong chương 2.3.2, việc triển khai mạng nơ-ron tích chập (CNN) trên CPU và GPU truyền thống, đặc biệt trong các thiết bị biên, bộc lộ nhiều

bất cập về hiệu năng, tiêu thụ năng lượng và giới hạn bộ nhớ. Điều này đặt ra yêu cầu cấp thiết đối với việc phát triển các kiến trúc phần cứng chuyên biệt – còn gọi là bộ tăng tốc CNN (CNN accelerator) – nhằm tối ưu hóa quá trình suy luận của các mô hình học sâu. Những bộ tăng tốc này thường được thiết kế với các đặc điểm nổi bật sau:

- **Giảm độ trễ tính toán:** Thông qua việc xây dựng các khối xử lý chuyên dụng, chẳng hạn như các mạch nhân nhân-cộng (Multiply-Accumulate – MAC), bộ tăng tốc CNN có khả năng thực hiện các phép tính tích chập và tích chập chiều sâu một cách song song và hiệu quả, từ đó giảm thiểu đáng kể độ trễ suy luận.
- **Tiết kiệm năng lượng:** Khác với CPU và GPU vốn mang tính chất xử lý tổng quát, bộ tăng tốc CNN được tối ưu hóa chỉ cho các phép toán đặc trưng của mạng nơ-ron, từ đó loại bỏ các khối chức năng dư thừa và giảm thiểu các thao tác không cần thiết, giúp tiết kiệm năng lượng tiêu thụ.
- **Tối ưu hóa bộ nhớ và băng thông:** Các bộ tăng tốc thường tích hợp chiến lược tái sử dụng dữ liệu (data reuse) và bộ đệm cục bộ (local buffer) nhằm giảm thiểu truy cập bộ nhớ ngoài, từ đó tối ưu băng thông và giảm độ trễ truyền dữ liệu.

2.3.4 Kiến trúc tổng thể của một CNN Accelerator trên FPGA



Hình 2.11 Sơ đồ khối bộ gia tốc CNN thông thường hiện nay [3]

Hình 2.11 minh họa hai cấu trúc khối thường gặp trong các hệ thống CNN Accelerator triển khai trên FPGA. Bên trái (hình a) là kiến trúc truyền thống với một Unified Engine hỗ trợ tính toán cho các loại lớp tích chập khác nhau, bao gồm STC (Standard Convolution), DWC (Depthwise Convolution) và PWC (Pointwise Convolution). Trong cấu trúc này, bộ đệm trong chip (On-Chip Buffer) được sử dụng để lưu trữ các dữ liệu tạm thời trong suốt quá trình xử lý, giúp giảm thiểu độ trễ khi truy xuất từ bộ nhớ ngoài (Off-Chip Memory). Cấu trúc này thường yêu cầu một bộ tính toán đơn lẻ (engine) để thực hiện tất cả các phép toán convolution, dẫn đến việc sử dụng tài nguyên không tối ưu cho từng loại lớp convolution riêng biệt.

Bên phải (hình b) là một cấu trúc CNN Accelerator hiện đại hơn, sử dụng các engine chuyên biệt cho từng loại phép toán convolution. Cụ thể, engine cho DWC được thiết kế riêng biệt để tối ưu hóa hiệu suất cho lớp Depthwise Convolution, trong khi engine cho SWC (Standard Convolution) và PWC (Pointwise Convolution) cũng có cấu trúc riêng biệt. Điều này giúp tối ưu hóa việc sử dụng tài nguyên FPGA, giảm thiểu tình trạng dư thừa tài nguyên và cải thiện hiệu suất toàn hệ thống.

Trong cả hai cấu trúc, bộ đệm trong chip đóng vai trò quan trọng trong việc giảm thiểu độ trễ và nâng cao hiệu suất của hệ thống bằng cách lưu trữ các dữ liệu cần thiết trong suốt quá trình tính toán. Ngoài ra, bộ nhớ ngoài (Off-Chip Memory) cung cấp dung lượng lớn để lưu trữ các dữ liệu cần thiết cho quá trình tính toán, tuy nhiên, việc truy xuất từ bộ nhớ ngoài luôn là yếu tố có thể làm giảm hiệu suất của hệ thống.

2.3.5 Hệ thống bộ nhớ

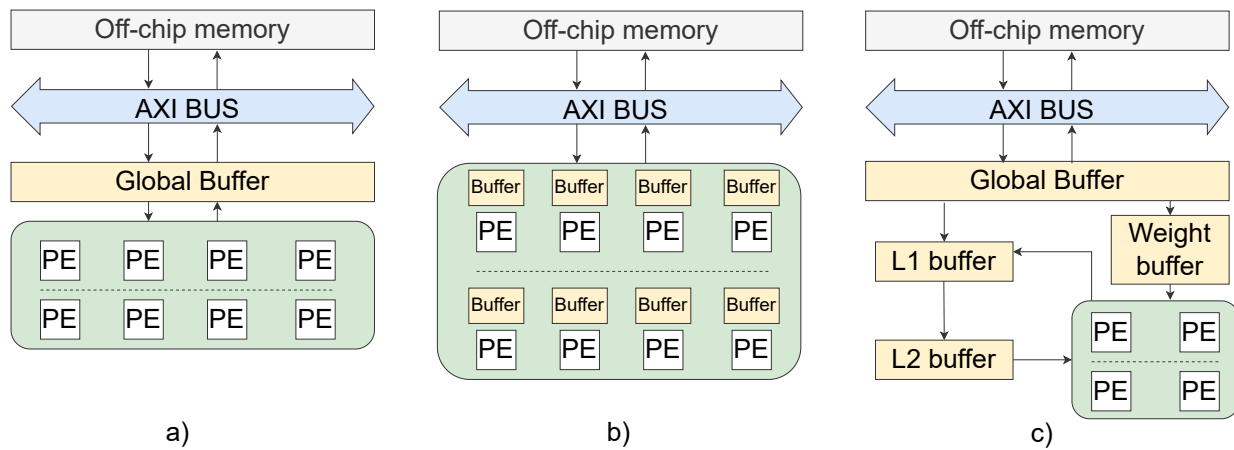
Như đã đề cập trong Mục 2.3.4, trong các bộ tăng tốc CNN hiện đại, hệ thống bộ nhớ thường bao gồm hai loại chính: bộ nhớ ngoài (Off-chip memory) và bộ nhớ trong (On-chip memory).

Off-chip memory là bộ nhớ được tích hợp trong hệ thống SoC nhưng nằm ngoài khối xử lý chính, thường là DRAM. Với dung lượng lớn, có thể lên tới vài gigabyte, loại bộ nhớ này thường được sử dụng để lưu trữ toàn bộ trọng số (weights) và dữ liệu đầu vào có kích thước lớn. Tuy nhiên, nhược điểm của off-chip memory là băng thông hạn chế và độ trễ truy cập cao, ảnh hưởng đáng kể đến hiệu năng tổng thể nếu truy cập thường xuyên.

Để khắc phục vấn đề này, các bộ tăng tốc CNN còn tích hợp thêm on-chip memory hay còn được gọi là buffer, thường là SRAM hoặc BRAM, với ưu điểm là tốc độ truy cập cao và độ trễ thấp. Mặc dù dung lượng của on-chip memory tương đối nhỏ, nhưng nó được sử dụng để lưu trữ tạm thời các dữ liệu trung gian hoặc một phần nhỏ của trọng số và đầu vào, giúp giảm thiểu truy cập đến bộ nhớ ngoài và nâng cao hiệu quả xử lý.

Ba kiến trúc bộ nhớ được minh họa trong Hình 2.12 thể hiện ba cách tổ chức hệ thống bộ nhớ thường gặp trong các bộ tăng tốc CNN [16]:

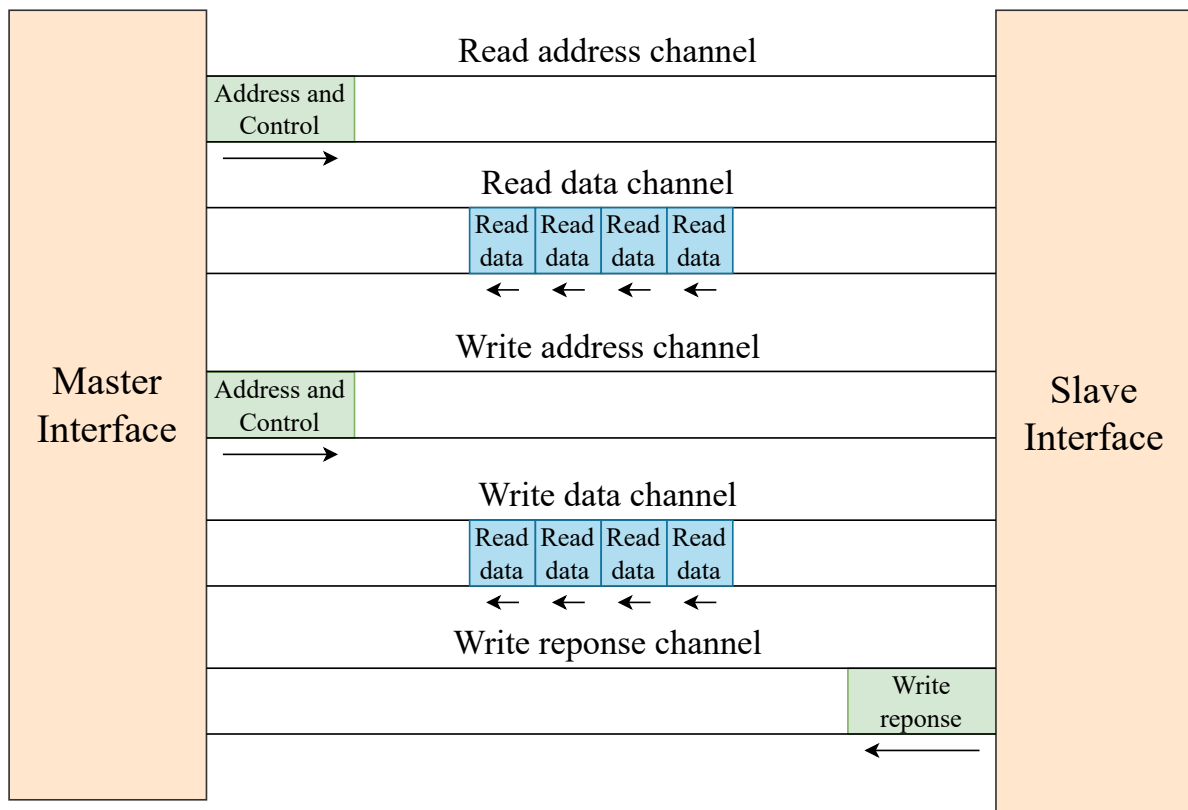
- **Hình (a):** Hệ thống sử dụng một bộ đệm chung (Global Buffer) làm trung tâm truy cập dữ liệu cho tất cả các phần tử xử lý (Processing Elements – PEs).
- **Hình (b):** Mỗi phần tử xử lý (PE) được trang bị một bộ đệm(Buffer) riêng biệt nhằm phục vụ truy cập dữ liệu độc lập.
- **Hình (c):** Hệ thống bộ nhớ được tổ chức theo cấu trúc nhiều cấp, bao gồm các cấp bộ nhớ đệm như Global Buffer, L1 Buffer và L2 Buffer và Weight. Các bộ đệm này sẽ có vai trò riêng và chia sẻ dữ liệu cho nhau.



Hình 2.12 Hệ thống bộ nhớ

2.3.6 Giao tiếp nội bộ và ngoại vi

Trong hệ thống bộ gia tốc tính toán, với số lượng phép tính cần thực hiện lên đến hàng trăm triệu, lượng dữ liệu di chuyển qua lại giữa các khối thành phần cũng rất lớn. Vì vậy, giao tiếp giữa các thành phần, chẳng hạn như bộ xử lý (PE), bộ nhớ ngoài chip và bộ nhớ trên chip, đóng vai trò quan trọng trong việc đảm bảo hiệu suất tổng thể của hệ thống. Một trong những giao thức phổ biến nhất được sử dụng trong các hệ thống FPGA là giao thức AXI4 (Advanced eXtensible Interface 4)[17], được thiết kế đặc biệt để tối ưu hóa việc truyền tải dữ liệu và quản lý các tài nguyên trong hệ thống.



Hình 2.13 AXI bus

Giao thức **AXI4** cung cấp một mô hình giao tiếp đa kênh được thể hiện tại hình 2.13. Giao thức này bao gồm năm kênh chính [17, 18]:

- **Read address channel:** Kênh này chịu trách nhiệm truyền địa chỉ đọc từ master đến slave, để xác định vị trí cần truy xuất trong bộ nhớ.
- **Read data channel:** Sau khi địa chỉ được xác định, kênh này truyền dữ liệu đọc từ slave đến master.
- **Write address channel:** Kênh này dùng để gửi địa chỉ ghi từ master đến slave để xác định địa chỉ ghi trong bộ nhớ.
- **Write data channel:** Sau khi địa chỉ ghi được xác định, kênh này sẽ truyền dữ liệu cần ghi từ master đến slave.
- **Write response channel:** Sau khi ghi dữ liệu thành công, kênh này sẽ gửi tín hiệu phản hồi từ slave đến master để xác nhận việc ghi đã hoàn tất.

Các yếu tố như tần số hoạt động và băng thông) của giao thức AXI4 ảnh hưởng trực tiếp đến hiệu suất của hệ thống. Tốc độ của giao thức quyết định độ trễ trong việc truyền tải dữ

liệu giữa các thành phần, trong khi băng thông xác định khả năng truyền tải lượng dữ liệu lớn trong thời gian ngắn. Với một hệ thống FPGA, việc tối ưu hóa các kênh giao tiếp này có thể giảm thiểu độ trễ và cải thiện hiệu suất tính toán, đặc biệt khi phải xử lý các mô hình mạng nơ-ron phức tạp yêu cầu băng thông lớn và tốc độ truyền tải dữ liệu nhanh.

Bên cạnh đó, việc tối ưu hóa các tham số như kích thước dữ liệu, chiến lược truyền tải và sử dụng bộ đệm (buffer) sẽ giúp đảm bảo việc sử dụng tài nguyên hệ thống hiệu quả hơn, đồng thời giảm thiểu tình trạng nghẽn cổ chai trong quá trình truyền tải dữ liệu giữa các thành phần. Tất cả những yếu tố này đều đóng vai trò quan trọng trong việc đảm bảo hiệu suất tối ưu của hệ thống trong các ứng dụng xử lý ảnh, video hoặc các tác vụ tính toán số lượng lớn như mạng nơ-ron tích chập.

2.4 Công cụ và phần mềm thiết kế

2.4.1 Vivado

Vivado [19] là công cụ phát triển (EDA - Electronic Design Automation) mạnh mẽ của Xilinx, được sử dụng chủ yếu để thiết kế và tối ưu hóa các mạch tích hợp (IC), đặc biệt là cho FPGA và SoC (System on Chip). Công cụ này hỗ trợ thiết kế phần cứng và phần mềm trong một môi trường tích hợp, giúp tối ưu hóa các đặc tính như tốc độ, diện tích và hiệu quả năng lượng của hệ thống.

Vivado cung cấp các công cụ mạnh mẽ cho việc triển khai các hệ thống Deep Learning Accelerator (DLA) trên FPGA. Công cụ này rất phù hợp với việc triển khai các mạng nơ-ron như EfficientNet, MobileNet, đặc biệt khi cần tối ưu hóa các phép toán phức tạp như tích chập (convolution) và pooling.

Các tính năng chính của Vivado bao gồm:

- **Thiết kế phần cứng với HDL:** Vivado hỗ trợ các ngôn ngữ mô tả phần cứng như VHDL và Verilog, giúp người dùng mô phỏng và triển khai và ước tính tài nguyên sử dụng cho hệ thống (Synthesis/Implementation) cho thiết kế.
- **High-Level Synthesis (HLS):** Vivado hỗ trợ High-Level Synthesis (HLS), cho phép chuyển mã từ C, C++ sang mô hình phần cứng RTL (Register Transfer Level), giúp giảm thiểu thời gian phát triển và nâng cao khả năng tối ưu hóa thiết kế.
- **Tối ưu hóa phần cứng cho AI:** Vivado cung cấp các công cụ tối ưu hóa cho việc triển khai các thuật toán AI, đặc biệt là các mạng CNN (Convolutional Neural Networks). Vivado có khả năng tối ưu hóa các mạng như EfficientNet để giảm thiểu độ phức tạp tính toán và nâng cao hiệu quả sử dụng tài nguyên.
- **Thiết kế và tối ưu hóa cho FPGA:** Vivado hỗ trợ nhiều dòng FPGA của Xilinx, bao gồm các dòng ZCU104, ZCU102 [20] và các FPGA từ dòng Virtex và Kintex. Điều này

rất quan trọng khi triển khai Deep Learning Accelerator (DLA) trên các hệ thống FPGA, giúp tận dụng tối đa hiệu suất của phần cứng.

- **Tối ưu hóa cho băng thông và độ trễ:** Với các công cụ phân tích và tối ưu hóa tài nguyên, Vivado giúp tối ưu hóa băng thông bộ nhớ, giảm độ trễ và tối ưu hóa tốc độ tính toán cho các ứng dụng Deep Learning.
- **Phân tích và mô phỏng hiệu suất:** Vivado cho phép phân tích và mô phỏng hiệu suất của thiết kế, giúp xác định các điểm yếu trong hệ thống và cải thiện khả năng xử lý tính toán trên FPGA.

Cách Vivado hỗ trợ triển khai Deep Learning Accelerator (DLA):

- Vivado cho phép tối ưu hóa việc sử dụng tài nguyên FPGA (LUTs, BRAMs, DSPs) cho các tác vụ tính toán AI, đảm bảo tốc độ và hiệu quả sử dụng tài nguyên cho các mạng nơ-ron phức tạp.
- Với Vivado HLS, các mô hình mạng sâu như EfficientNet có thể được tối ưu hóa cho FPGA mà không cần phải viết mã HDL thủ công, giảm thiểu công sức phát triển và nâng cao hiệu quả hệ thống.
- Vivado hỗ trợ thiết kế các khối phần cứng chuyên dụng như các đơn vị PE (Processing Elements) cho các phép toán Convolution, Pooling và các phép toán đặc trưng khác trong deep learning.
- Vivado có thể mô phỏng toàn bộ hệ thống phần cứng để kiểm tra tính đúng đắn và hiệu suất của thiết kế trước khi triển khai lên FPGA thực tế, giúp giảm thiểu lỗi và tối ưu hóa thiết kế.

Vivado là công cụ quan trọng trong việc triển khai các hệ thống tăng tốc Deep Learning, đặc biệt là trên FPGA. Công cụ này không chỉ hỗ trợ thiết kế phần cứng hiệu quả mà còn giúp tối ưu hóa hiệu suất và tài nguyên khi triển khai các mạng nơ-ron phức tạp như EfficientNet và MobileNet trên các thiết bị nhúng.

2.5 Thông số đánh giá

Khi đánh giá hiệu năng của một bộ gia tốc (accelerator) triển khai trên FPGA cho các mô hình mạng nơ-ron sâu, các thông số kỹ thuật sau thường được sử dụng để phản ánh khả năng tính toán, mức tiêu thụ tài nguyên và hiệu suất năng lượng của hệ thống:

- **Tần số (Frequency):** Là tốc độ xung nhịp của hệ thống, thường được đo bằng megahertz (MHz). Tần số càng cao thì tốc độ xử lý dữ liệu của hệ thống càng nhanh, nhưng đồng thời cũng có thể làm tăng tiêu thụ năng lượng.

- **Số lượng phép toán (GOPs):** Viết tắt của Giga Operations, biểu thị tổng số phép toán được thực hiện trong một lần suy luận (inference) của bộ tăng tốc CNN.
- **Thông lượng (Throughput – GOPS):** Thông lượng biểu thị số lượng phép toán điểm nổi (operations) mà hệ thống thực hiện được trong một giây, đơn vị đo là GOPS (Giga Operations Per Second). Chỉ số này phản ánh trực tiếp tốc độ tính toán của bộ tăng tốc. Thông lượng được xác định theo công thức:

$$\text{Throughput (GOPS)} = \frac{\text{GOPs}}{\frac{T_{\text{cal}}}{f_{\text{clk}}}} = \frac{f_{\text{clk}} \times \text{GOPs}}{T_{\text{cal}}} \quad (2.14)$$

trong đó

T_{cal} là tổng số chu kỳ đồng hồ để hoàn thành một suy luận (inference), f_{clk} là tần số xung nhịp của hệ thống (Hz) và (GOPs) là số lượng phép toán (tính theo Giga-Operations) thực hiện trong một suy luận. Công thức (2.14) cho thấy thông lượng tỉ lệ thuận với tần số xung nhịp và số phép toán của một suy luận, đồng thời tỉ lệ nghịch với tổng số chu kỳ cần thiết để hoàn thành suy luận đó.

- **LUTs (Look-Up Tables) :** Số bảng tra cứu được dùng để hiện thực hàm logic tổ hợp trong FPGA. Thông số được trích từ báo cáo Utilization ở giai đoạn Synthesis/Implementation của Vivado.
- **FFs (Flip-Flops) :** Tổng số thanh ghi lưu trữ trạng thái (pipeline registers, thanh ghi điều khiển, ...) theo báo cáo Utilization sau Synthesis/Implementation.
- **BRAMs (Block RAMs) :** Khối SRAM hai cổng bên trong FPGA, dung lượng điển hình 18 kbit hoặc 36 kbit. BRAM cung cấp băng thông cao và độ trễ thấp nên thường được dùng làm bộ đệm IFM/OFM, trọng số và FIFO. Số lượng BRAM lấy từ báo cáo Utilization.
- **RAMs (On-chip Memory, KB):** Chỉ số tổng hợp biểu thị dung lượng bộ nhớ on-chip, được tính bằng cách cộng dung lượng của tất cả BRAM và URAM rồi quy đổi sang kilobyte, giúp so sánh thống nhất giữa các kiến trúc.
- **URAMs (UltraRAMs):** Bộ nhớ mật độ cao tích hợp sẵn (288 kbit mỗi khối) trên các thiết bị UltraScale(+), thích hợp lưu trữ ma trận lớn hoặc bộ đệm dung lượng lớn nhờ dung lượng/bit cao hơn BRAM và hỗ trợ ECC nội bộ. Thông số URAM cũng được trích từ báo cáo Utilization.
- **DSPs (Digital Signal Processors):** Số khối phần cứng chuyên dụng (ví dụ DSP48E2) thực hiện đa hàm MAC (multiply–accumulate) độ trễ thấp, lấy từ báo cáo Utilization.

- **Power (W):** Công suất tiêu thụ của hệ thống khi hoạt động, đo bằng watt. Giá trị ước tính qua báo cáo Power sau Implementation hoặc đo thực nghiệm trên phần cứng.
- **FPS (Frames Per Second):** Số lượng ảnh được xử lý mỗi giây trong quá trình suy luận – chỉ số phổ biến trong các bài toán thị giác máy tính. FPS được tính bằng công thức:

$$FPS = \frac{f_{clk}}{T_{cal}} \quad (2.15)$$

- **GOPS/W (Hiệu suất năng lượng):** Tỷ lệ giữa thông lượng và công suất tiêu thụ, phản ánh hiệu quả sử dụng năng lượng của hệ thống được tính bằng công thức:

$$GOPS/W = \frac{\text{Throughput (GOPS)}}{W} \quad (2.16)$$

- **GOPS/RAMs:** Tỷ lệ giữa thông lượng và số lượng RAM sử dụng, phản ánh hiệu quả khai thác bộ nhớ trên chip được tính bằng công thức:

$$\frac{GOPS}{RAMs} = \frac{\text{Throughput (GOPS)}}{RAMs} \quad (2.17)$$

Các thông số trên là cơ sở để so sánh giữa các kiến trúc accelerator khác nhau, từ đó lựa chọn giải pháp tối ưu cho các hệ thống nhúng hoặc các ứng dụng biên có tài nguyên hạn chế.

2.6 Kết luận chương

Trong chương này, chúng ta đã trình bày tổng quan về mạng Neural tích chập (CNN), đặc biệt là các mạng như MobileNet, EfficientNet và các kỹ thuật liên quan như Depthwise Convolution và Squeeze-and-Excitation (SE). Chúng ta đã phân tích và làm rõ các đặc điểm của các mô hình này, nhấn mạnh vào sự cải tiến trong kiến trúc và hiệu suất so với các mô hình truyền thống.

Cùng với đó, chúng ta cũng đề cập đến những thách thức khi triển khai các mạng nơ-ron trên các hệ thống có tài nguyên hạn chế như FPGA và các thiết bị biên. Các vấn đề về hiệu suất, tiêu thụ năng lượng và sự tối ưu hóa các phép toán convolution đã được thảo luận và trình bày các giải pháp như việc sử dụng các bộ xử lý chuyên dụng, tối ưu hóa các phép toán với mixed-precision quantization và các kiến trúc tối ưu như Fused-MB Conv.

Bài toán triển khai các mô hình EfficientNet và MobileNet trên FPGA không chỉ dừng lại ở những vấn đề về tài nguyên phần cứng mà còn cần tối ưu hóa cách thức tổ chức và xử lý dữ liệu. Do đó, việc nghiên cứu và áp dụng các kỹ thuật như pipelining, sử dụng buffer và các kiến trúc tính toán đặc biệt hỗ trợ riêng cho DWC hay PWC có thể giúp cải thiện đáng kể hiệu suất và sử dụng tài nguyên hiệu quả.

Chúng ta cũng đã xem xét các thông số đánh giá quan trọng như tần số, throughput, hiệu suất năng lượng và tốc độ xử lý (FPS) để so sánh giữa các phương pháp thiết kế khác nhau,

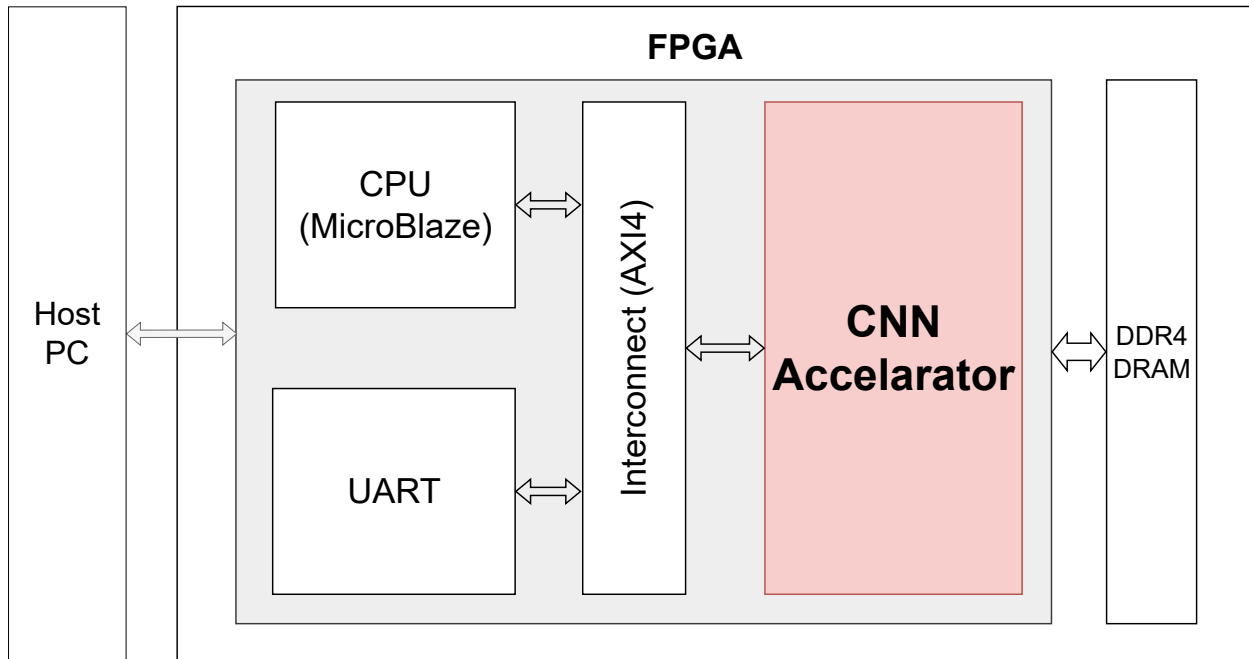
từ đó đưa ra các phương án tối ưu cho việc triển khai các mô hình mạng nơ-ron phức tạp trên FPGA.

Nhìn chung, chương này đã cung cấp một cái nhìn tổng quan và sâu sắc về các yếu tố kỹ thuật quan trọng trong việc triển khai các mạng CNN trên FPGA, từ đó tạo nền tảng cho việc xây dựng các hệ thống gia tốc phần cứng hiệu quả và tiết kiệm năng lượng cho các ứng dụng AI. Chương 3 đề cập đến thiết kế hệ thống và kiến trúc chi tiết của bộ tăng tốc tính toán mạng EfficientNet của chúng tôi.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

Tại chương 3 này chúng tôi sẽ trình bày chi tiết về bộ tăng tốc tính toán mạng EfficientNet V2 của chúng tôi bao gồm kiến trúc chi tiết của từng thành phần trong hệ thống và luồng dữ liệu xuyên suốt hệ thống.

3.1 Kiến trúc tổng thể hệ thống



Hình 3.1 Sơ đồ khối hệ thống

Kiến trúc của hệ thống máy tăng tốc tính toán mạng EfficientNet bao gồm:

- **Host PC:** Đây là máy tính chủ dùng để giao tiếp với hệ thống FPGA. Các dữ liệu hình ảnh đầu vào được gửi từ Host PC đến FPGA thông qua giao diện UART.
- **CPU (MicroBlaze):** MicroBlaze là bộ xử lý được sử dụng trên FPGA để thực hiện các tác vụ khởi tạo dữ liệu về cấu hình cho các khối trong hệ thống. MicroBlaze kết nối với phần cứng và thực hiện các phép toán điều khiển như phân phối dữ liệu hình ảnh đầu vào và lưu trữ dữ liệu đầu ra trả từ bộ tăng tốc.
- **UART[21]:** UART chịu trách nhiệm giao tiếp giữa hệ thống FPGA và Host PC. Nó sử dụng các tín hiệu nối tiếp để truyền tải dữ liệu giữa hai hệ thống, giúp điều khiển và trao đổi thông tin về hình ảnh truyền vào từ môi trường bên ngoài.
- **Interconnect (AXI4):** Giao tiếp AXI4 đảm nhận vai trò kết nối và truyền dữ liệu giữa các

thành phần của hệ thống, đảm bảo dữ liệu giữa CPU, UART, DRAM và CNN Accelerator được chuyển tải.

- **CNN Accelerator:** Bộ gia tốc nhân dữ liệu từ DRAM qua giao diện AXI4 và thực hiện các phép toán tính toán tích chập trong mô hình CNN và trả lại kết quả đầu ra của mô hình thông qua giao diện chung AXI.
- **DDR4 DRAM[22]:** Bộ nhớ ngoài (DDR4 DRAM) lưu trữ dữ liệu đầu vào, dữ liệu trung gian và trọng số mạng. Dữ liệu của DRAM sẽ được khởi tạo thông qua quá trình CPU lấy dữ liệu từ Host PC thông qua UART, ngoài ra DRAM lưu trữ dữ liệu tạm thời cho quá trình tính toán của bộ gia tốc.

Kiến trúc này mục tiêu sẽ giúp tối ưu hóa việc sử dụng tài nguyên phần cứng và tăng hiệu suất của hệ thống khi triển khai các mô hình CNN phức tạp trên FPGA, đồng thời giảm độ trễ và tăng tốc độ xử lý so với các hệ thống không sử dụng phần cứng chuyên dụng.

3.2 Thiết kế bộ tăng tốc tính toán mạng EfficientNet-V2

Nhìn vào Hình 2.8, có thể phân tích cấu trúc của bộ gia tốc CNN được chia thành hai khối chính: Fused-MB Conv và MB Conv. Mỗi khối này có các lớp chuyên biệt được thiết kế để xử lý các tính toán của mạng hiệu quả hơn

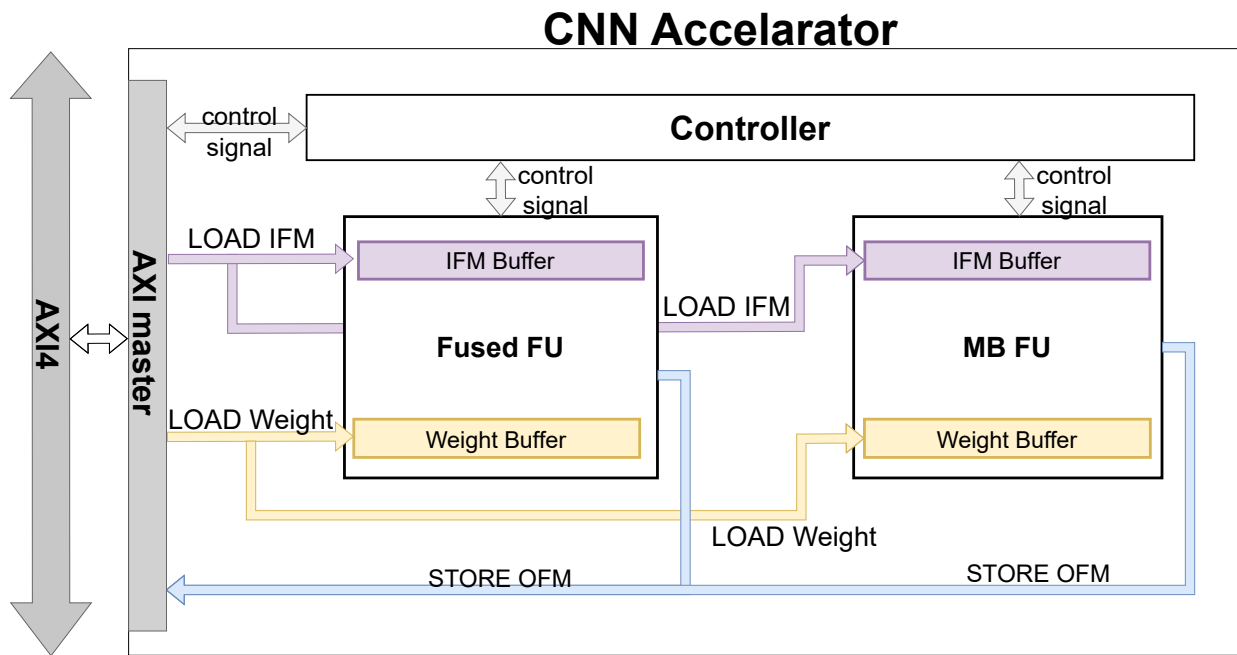
Vì kiến trúc của hai khối này bao gồm các lớp tính toán khác nhau, chúng tôi đề xuất xây dựng hai khối tính toán chuyên biệt cho hai khối này, bao gồm Fused FU cho khối Fused-MB Conv và MB FU cho khối MB Conv. Kiến trúc chi tiết của từng khối sẽ được trình bày chi tiết trong phần 3.2.2 và phần 3.2.3

3.2.1 Luồng dữ liệu trong hệ thống

Hệ thống CNN Accelerator của chúng tôi bao gồm ba thành phần chính: Fused FU, MB FU và AXI Master, với một bộ Controller quản lý các hoạt động của toàn bộ hệ thống như trong Hình 3.2

Fused FU và MB FU là hai khối tính toán chuyên biệt, mỗi khối sẽ thực hiện các phép toán tính toán cho các lớp Fused-Conv và MB-CONV tương ứng trong mạng CNN. Mỗi khối tính toán có các bộ nhớ tích hợp IFM Buffer và Weight Buffer để lưu trữ tạm thời các IFM (Input Feature Map), trọng số (Weight), hoặc là kết quả tính toán OFM trung gian. Khối tính toán Fused FU hay là MB FU sẽ dùng trực tiếp dữ liệu từ các buffer này để tính toán.

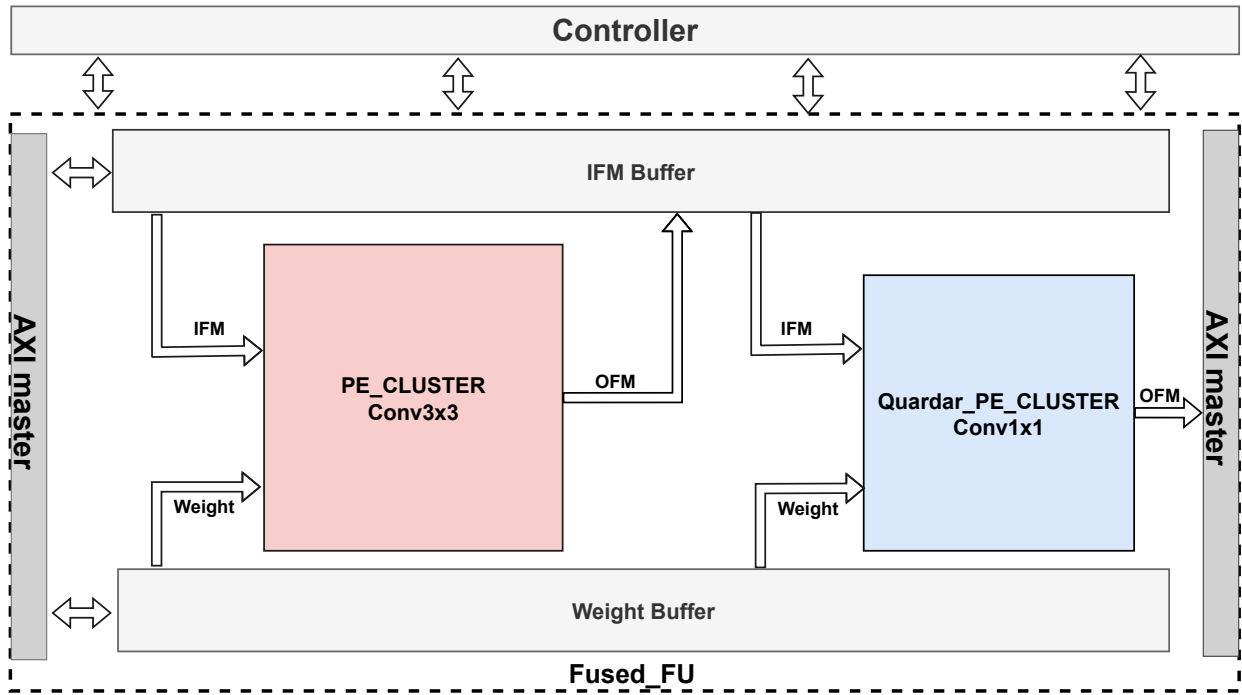
AXI Master có nhiệm vụ giao tiếp với bộ nhớ ngoài và thực hiện quá trình đọc/ghi dữ liệu. Cụ thể, AXI Master sẽ: đọc dữ liệu IFM (LOAD IFM) và Weight (LOAD Weight) từ bộ nhớ ngoài và tải vào các buffer của Fused FU hoặc MB FU. Sau khi các khối tính toán hoàn tất, AXI Master sẽ lưu trữ kết quả tính toán OFM (STORE OFM) ra bộ nhớ ngoài.



Hình 3.2 Luồng dữ liệu giữa bộ nhớ trong chip và bộ nhớ ngoài chip

Bộ Controller trong hệ thống có nhiệm vụ điều khiển các hoạt động của toàn bộ hệ thống như đọc ghi dữ liệu của các khối buffer hay là tính toán của các khối tính toán Fused FU và MB FU thông qua các tín hiệu điều khiển (control signal). Ta sẽ lấy ví dụ về một quá trình tính toán trong hệ thống với khối Fused: Bộ Controller sẽ tiến hành nạp dữ liệu IFM và Weight (LOAD Weight và LOAD IFM) vào các buffer tương ứng tại Fused FU. Sau khi khối Fused FU hoàn tất tính toán, kết quả đầu ra (OFM) sẽ được lưu ra ngoài (STORE OFM) thông qua AXI master. Tương tự thì quá trình này cũng sẽ diễn ra với MB FU và hai khối sẽ kết hợp tính toán song song theo cơ chế pipeline để tính toán toàn bộ các khối trong kiến trúc mạng tại Hình 2.8. Các luồng dữ liệu cũng được thể hiện qua các mũi tên tại hình 3.2.

3.2.2 Triển khai khối tính xử lý chính Fused FU



Hình 3.3 Khối Fused

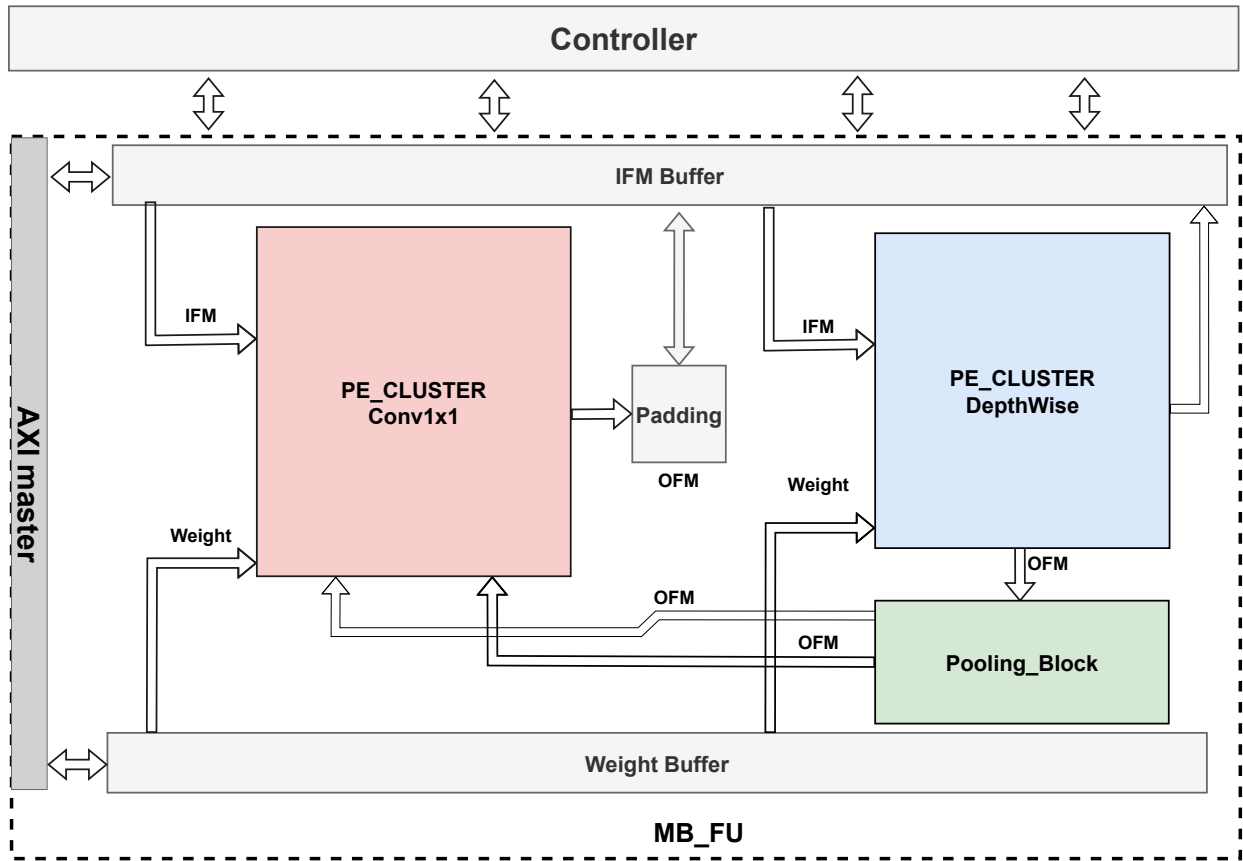
Khối tính toán Fused_FU được cấu thành từ hai khối xử lý chính: PE_Cluster Conv3x3 và Quardar_PE_Cluster Conv1x1. Hai khối này đảm nhiệm việc tính toán cho hai lớp convolution đặc trưng trong Fused-MB Conv, PE_Cluster Conv3x3 đảm nhiệm tính toán cho công thức 2.7 và Quarda_PE_Cluster Conv1x1 đảm nhiệm tính toán cho công thức 2.8

Bên cạnh các khối tính toán, Fused_FU còn tích hợp hai bộ buffer chính là IFM Buffer và Weight buffer. Trong đó, IFM Buffer đảm nhận việc lưu trữ cả dữ liệu IFM đầu vào và OFM trung gian giữa hai lớp tích chập. Các khối buffer này sẽ nhận dữ liệu và trả dữ liệu ra bộ nhớ ngoài (DDR4 RAM – Hình 3.1) thông qua module AXI master(sẽ được nói chi tiết tại phần 3.2.7)

Khối Fused FU gồm hai giai đoạn xử lý theo pipeline. Đầu tiên, cụm PE_Cluster CONV3x3 lấy dữ liệu IFM và trọng số từ IFM Buffer và Weight Buffer, sau đó tính toán và sinh ra OFM trung gian. Kết quả này được ghi trở lại vào IFM Buffer, nơi cụm Quardar_PE_Cluster sẽ sử dụng làm dữ liệu IFM đầu vào cùng với trọng số tương ứng để tiếp tục phép tính. Hai cụm xử lý chạy tuần tự, mỗi kết quả đầu ra của PE_Cluster CONV3x3 tức thì trở thành đầu vào của Quardar_PE_Cluster. Cuối cùng dữ liệu OFM của Quardar_PE_Cluster được lưu ra ngoài trực tiếp ngay sau khi tính xong thông qua module AXI master.

Toàn bộ hoạt động của các khối tính toán và các khối lưu trữ trong **Fused_FU** đều được điều khiển thông qua một bộ điều khiển trung tâm (controller) duy nhất, như minh họa tại Hình 3.3.

3.2.3 Triển khai khối tính xử lý chính MB FU



Hình 3.4 Khối MB FU

Khối MB Conv trong kiến trúc mạng EfficientNetV2 (Hình 2.8) đảm nhiệm ba phép toán chính: hai lớp tích chập – gồm tích chập điểm (Pointwise Convolution - PWC) và tích chập chiều sâu (Depthwise Convolution - DWC) – cùng với một lớp Pooling. Để thực hiện hiệu quả các phép toán này trên phần cứng, thiết kế của chúng tôi xây dựng ba khối tính toán chuyên biệt tương ứng.

Cụ thể, khối PE Cluster Conv1x1 được thiết kế để đảm nhiệm các lớp tích chập điểm, tương ứng với các công thức 2.9 đã trình bày trong mục 2.2.1. Trong khi đó, khối PE Cluster Depthwise thực hiện tính toán cho lớp tích chập chiều sâu, ứng với công thức 2.10. Cuối cùng, phép tính trung bình được đảm nhận bởi khối Pooling Block, thực hiện theo công thức 2.11. Các dữ liệu đầu vào (Input Feature Maps) và trọng số (Weights) được lưu trữ trong bộ nhớ BRAM trên chip để phục vụ cho các phép tính này.

Một thành phần không thể thiếu trong MB Conv là khối Padding – đảm nhiệm việc mở rộng biên dữ liệu IFM đầu vào trước khi đưa vào khối Depthwise Convolution.

Toàn bộ hoạt động của khối MB FU được mô tả chi tiết trong Hình 3.4. Các dữ liệu IFM, trọng số (Weights) và bias được truyền từ bộ nhớ ngoài (DDR4 RAM – Hình 3.1) thông qua khối AXI Master Interface. Sau khi tính toán hoàn tất, kết quả đầu ra (OFM) được ghi trả về bộ nhớ ngoài cũng qua giao tiếp AXI.

Dữ liệu đọc từ DDR4 được lưu vào các bộ đệm trên chip như IFM Buffer và Weight Buffer. Từ đây, khối điều khiển (Controller) sẽ điều phối quá trình đọc dữ liệu từ buffer, đưa vào các khối tính toán và xử lý đầu ra.

Quy trình xử lý dữ liệu trong MB FU diễn ra như sau:

IFM và trọng số được nạp vào khối PE Cluster Conv1x1 để thực hiện tích chập điểm. Kết quả OFM đầu ra sau đó đi qua khối Padding và dữ liệu đã được đệm sẽ được nạp trở lại vào IFM Buffer.

Tiếp theo, khối PE Cluster Depthwise sử dụng dữ liệu từ IFM Buffer để thực hiện tích chập chiều sâu và kết quả OFM đầu ra sẽ được truyền tới khối Pooling, đồng thời cũng lưu vào IFM Buffer, chuẩn bị cho lớp PWC cuối cùng của MB Conv (như thể hiện trong Hình 2.8).

Sau khi phép Pooling hoàn tất, dữ liệu kết quả lại tiếp tục đưa tới khối PE Cluster Conv1x1 để thực hiện tính toán cho 2 lớp tích chập điểm tiếp theo trong lớp nén và kích thích (SE layer) (như thể hiện trong Hình 2.8).

Sau khi toàn bộ các lớp trong MB Conv được xử lý, dữ liệu cuối cùng sẽ được đọc từ IFM Buffer và ghi trả về bộ nhớ ngoài qua AXI Master.

Tất cả các thao tác đọc/ghi, định tuyến dữ liệu và điều phối thứ tự xử lý đều được quản lý bởi Controller. Bộ điều khiển chịu trách nhiệm đảm bảo quá trình tính toán diễn ra tuần tự, đúng logic, đồng thời tối ưu hiệu suất truy xuất bộ nhớ và tính toán song song giữa các khối chức năng trong MB Conv.

3.2.4 Triển khai khối tính toán tích chập - Processing Elements

Thành phần tính toán tích chập (PE), là thành phần chính bên trong các khối tính toán PE_Cluster X thể hiện trong hình 3.5, PE_Cluster X tương ứng với các khối PE_Cluster Conv3x3 và Quardar_PE_Cluster Conv1x1 ở mục 3.2.2 và khối PE Cluster Conv1x1, PE Cluster Depthwise trong mục 3.2.3. Ngoài ra bên trong các PE_cluster còn có khối Active nhận dữ liệu đầu ra của các PE (tín hiệu Y), khối Active này sẽ được trình bày ở mục phần 3.2.5 tiếp theo. Cấu trúc của khối tính toán tích chập này được thể hiện ở phía bên trái hình 3.5, khối bao gồm một bộ nhân để thực hiện phép nhân giữa Input Feature Map (IFM) và Weight. Sau khi tính toán nhân xong, kết quả sẽ được chuyển vào một bộ cộng tích lũy để tính toán tổng cộng các giá trị. Quá trình này được đồng bộ hóa với tín hiệu đồng hồ clk. Khối tính toán tích chập này phục vụ tính toán cho công thức:

$$Y = \sum_{c=1}^C X_c \cdot W_c \quad (3.18)$$

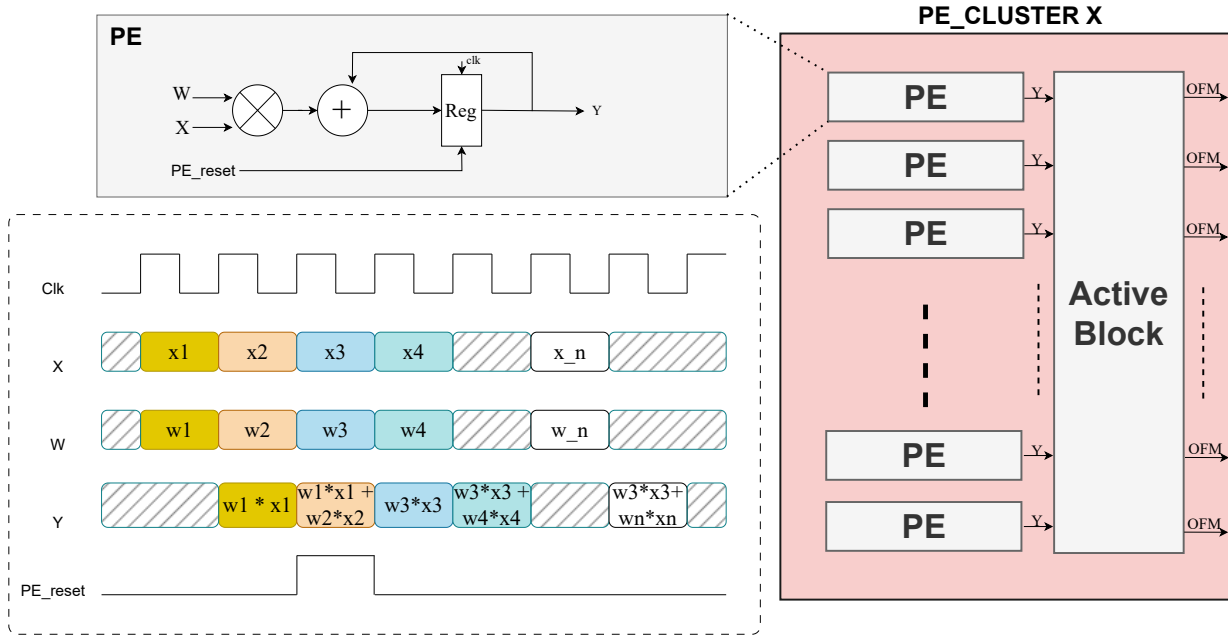
Trong đó:

- X_c là giá trị tại vị trí điểm ảnh nhất định của feature map đầu vào ở kênh thứ c ,

- W_c là trọng số cho kênh thứ c ,
- Y là giá trị của điểm ảnh tại vị trí tương ứng trong feature map đầu ra.

Với cơ chế hoạt động và công thức tính toán của PE như trên 3.18, ta liên hệ với công thức tính tích chập đã được giới thiệu trong mục 2.2.1, PE có thể hoàn thành việc tính toán cho các lớp tích chập điểm (PWC) và tích chập chiều sâu (DWC). Cụ thể, PE thực hiện tính toán từng điểm ảnh tương ứng trong feature map đầu ra và quá trình này sẽ lặp lại cho đến khi tất cả các điểm ảnh của feature map đầu ra được tính toán xong.

Để đảm bảo rằng PE có thể tính toán cho dữ liệu mới sau khi hoàn thành phép toán với dữ liệu cũ, PE còn có tín hiệu PE-reset để làm mới các giá trị trong bộ cộng tích lũy mỗi khi quá trình tính toán kết thúc. Điều này giúp đảm bảo rằng mỗi lần xử lý dữ liệu mới, PE sẽ bắt đầu lại từ trạng thái ban đầu. Hình vẽ dưới đây thể hiện sơ đồ khối của PE và sơ đồ thời gian (timing diagram) mô tả cách thức hoạt động của thành phần tính toán này.



Hình 3.5 Cấu trúc của khối tính toán và hoạt động của khối

Hình vẽ trên cho thấy quá trình xử lý tính toán của PE, nơi IFM (X) và Weight (W) được nhân với nhau để tạo ra giá trị mới, sau đó được cộng dồn lại và cuối cùng là kết quả đầu ra OFM. Tín hiệu PE-reset giúp reset giá trị tính toán trước khi phép tính mới được thực hiện.

3.2.5 Triển khai khối tính toán hàm kích hoạt - Active Block

Để thực hiện hai hàm kích hoạt của mạng EfficientNetV2 được cập tại hai công thức hàm ReLU 2.12 và hàm Swish 2.13, bao gồm hàm ReLU (Rectified Linear Unit) và Swish, chúng tôi đã thiết kế một khối tính toán đặc biệt gọi là Active Block. Khối này có trong các

khối PE Cluster và được thể hiện trong hình 3.5.

Khối Active Block có đầu vào là dữ liệu từ các khối tính toán PE (Processing Elements) sau khi đã thực hiện các phép tính tích chập và tính toán cho các lớp của mạng. Đầu ra của khối tính toán này là giá trị của feature map (tín hiệu OFM - hình 3.5) sau khi áp dụng hàm kích hoạt, giúp đưa tính phi tuyến vào mô hình.

3.2.6 Triển khai khối tính toán - Pooling

Thành phần tính toán trung bình (Average Pooling Block) được sử dụng để thực hiện phép tính trung bình trên một vùng nhỏ (window) của Feature Map đầu vào. Cấu trúc của khối tính toán này được minh họa trong Hình 3.6, bao gồm một bộ cộng có tích lũy, một thanh ghi và một bộ nhân để thực hiện chia trung bình.

Cụ thể, tại mỗi chu kỳ xung nhịp, một phần tử đầu vào x_i sẽ được cộng dồn vào tổng tích lũy (W). Kết quả của phép cộng sẽ được lưu trữ tạm thời trong một thanh ghi (Reg) nhằm duy trì giá trị tổng hiện tại cho đến khi phần tử tiếp theo đến. Sau khi toàn bộ các phần tử trong cửa sổ đã được cộng dồn, giá trị tổng này sẽ được nhân với hệ số $\text{Coef}=1/N$, với N là số phần tử trong cửa sổ pooling, để tạo ra đầu ra Y . Phép nhân này đóng vai trò chia trung bình.

Quá trình hoạt động được đồng bộ theo tín hiệu đồng hồ clk và việc khởi tạo lại tổng tích lũy được thực hiện thông qua tín hiệu reset. Điều này đảm bảo rằng mỗi vùng pooling mới sẽ được tính toán lại từ đầu, không bị ảnh hưởng bởi giá trị trước đó.

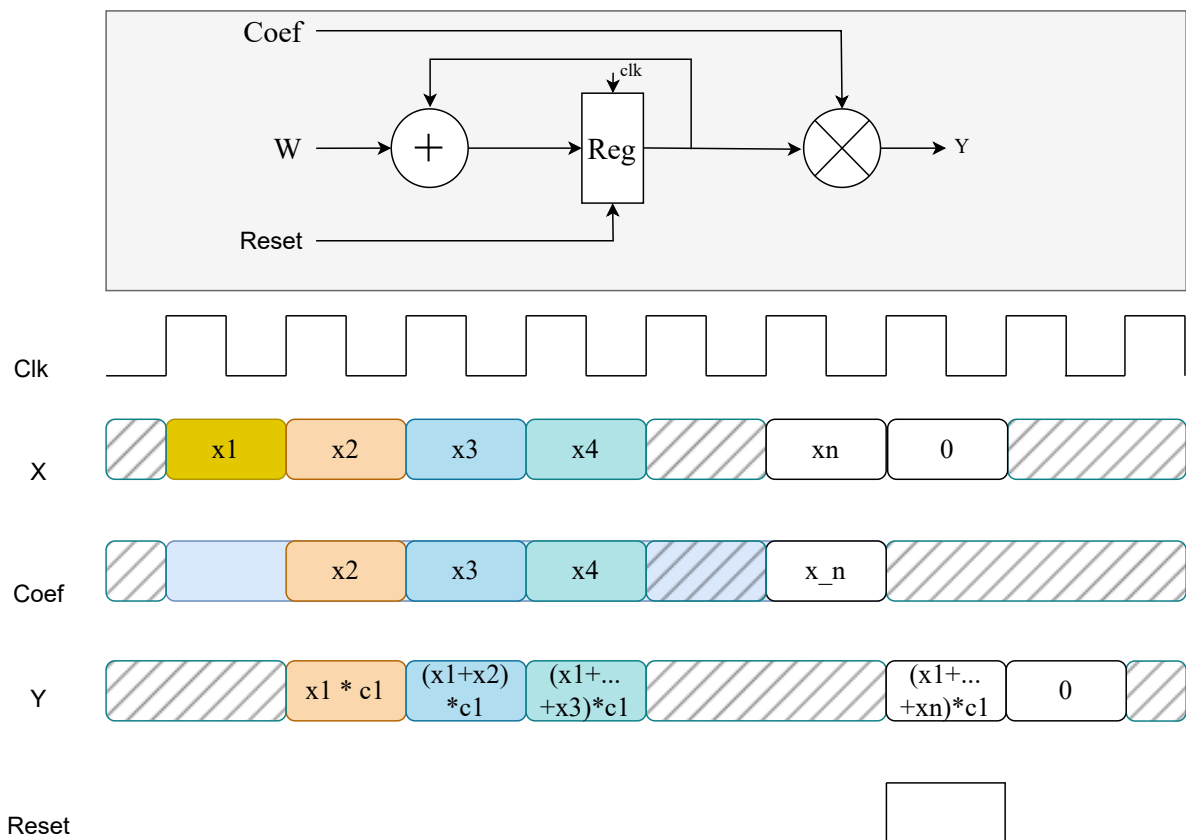
Công thức tổng quát cho phép tính Average Pooling là:

$$Y = \left(\sum_{i=1}^N x_i \right) \cdot \frac{1}{N} \quad (3.19)$$

Trong đó:

- x_i : giá trị các phần tử trong vùng pooling,
- N : số lượng phần tử trong vùng,
- Y : giá trị đầu ra sau khi tính trung bình.

Với cơ chế hoạt động và công thức tính toán Pooling như trên 3.19, ta liên hệ với công thức tính pooling 2.11 đã được giới thiệu trong chương 2.2.1 với $(N = H * W)$, Cơ chế này được mô tả trực quan trong Hình 3.6, bao gồm sơ đồ khối và biểu đồ thời gian hoạt động:

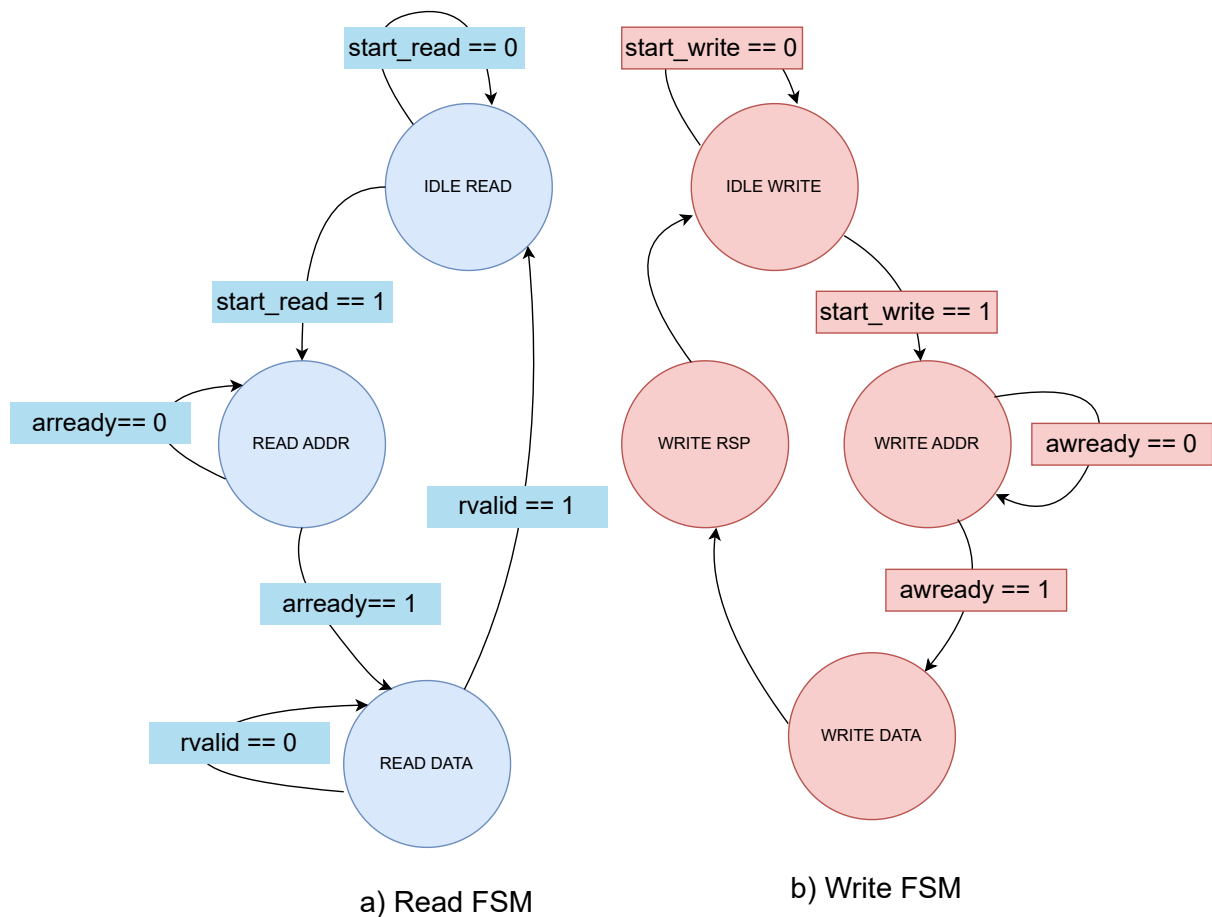


Hình 3.6 Cấu trúc và hoạt động của khối tính toán Average Pooling

Như hình minh họa, mỗi chu kỳ đồng hồ sẽ nạp thêm một phần tử đầu vào x_i , thực hiện cộng dồn với các giá trị trước đó. Sau chu kỳ cuối cùng, toàn bộ tổng tích lũy sẽ được nhân với hằng số $Coef=1/N$ để trả về giá trị trung bình Y .

3.2.7 AXI Master

Module AXI Master trong thiết kế của chúng tôi chịu trách nhiệm giao tiếp với bộ nhớ ngoài, đóng vai trò là Master trong bus AXI (DMA). Cụ thể, AXI Master sẽ đảm nhiệm việc đọc và ghi dữ liệu vào các bộ đệm IFM (Input Feature Map) và Weight Buffer như đã được mô tả trong Hình 3. Như đã giải thích trong phần 2.3.6, AXI cung cấp hai kênh cho quá trình đọc và ba kênh cho quá trình ghi, cho phép cả hai quá trình này diễn ra đồng thời. Chính vì vậy, chúng tôi đã thiết kế module AXI Master với hai máy trạng thái độc lập để hỗ trợ việc đọc và ghi đồng thời, như được thể hiện trong Hình 3.7 a) và 3.7 b).



Hình 3.7 Mô hình FSM của Module AXI Master

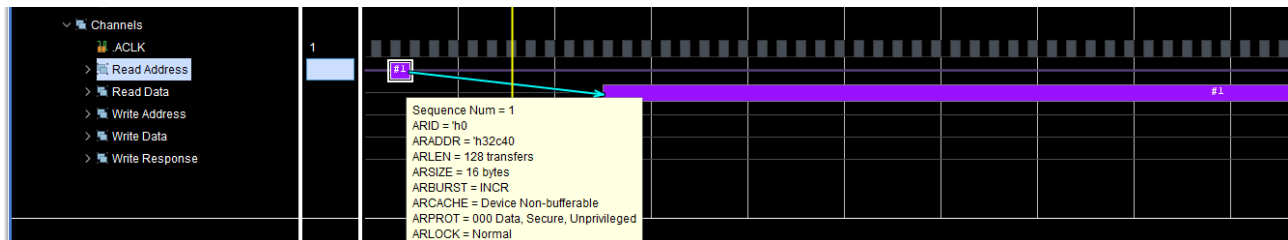
Máy trạng thái đọc bao gồm ba trạng thái chính được thể hiện trong sơ đồ FSM của hình 3.7 a):

- **IDLE:** Trạng thái chờ đợi lệnh đọc từ controller, sẽ đợi tín hiệu `start_read` (`start_read == 1`) để bắt đầu trạng thái đọc.
- **READ ADDR:** Trạng thái truyền địa chỉ cần đọc và các tín hiệu điều khiển giao dịch như `ARLEN`, `ARSIZE`, v.v, trạng thái này sẽ chờ đợi `arready` từ slave (`arready == 1`) để đi đến trạng thái đọc dữ liệu.
- **READ DATA:** Trạng thái nhận dữ liệu trả lại từ bộ nhớ và sẽ nhận dữ liệu cuối cùng khi tín hiệu `rlast` được đẩy lên 1 (`rlast == 1`) và quay về trạng thái IDLE.

Trong quá trình đọc, các tham số như `ARLEN` và `ARSIZE` sẽ được cấu hình phù hợp với lượng dữ liệu cần truy xuất từ controller để tận dụng hết dữ liệu của 1 giao dịch và hạn chế các truy cập lặp lại không cần thiết.

Quy trình đọc của AXI Master thông qua AXI Bus trong môi trường mô phỏng bằng

công cụ Vivado được thể hiện trên waveform tại Hình 3.8. Tại hình này, ô nhỏ nằm ở hàng Read Address đại diện cho trạng thái READ_ADDR trong máy trạng thái điều khiển (FSM). Mũi tên chỉ hướng sang phải biểu thị cho sự chuyển trạng thái kế tiếp là READ_DATA của giao dịch đó, trạng thái này sẽ được kích hoạt sau một khoảng thời gian nhất định kể từ khi gửi địa chỉ đọc.



Hình 3.8 Quy trình đọc dữ liệu của IP AXI bus trên vivado

Máy trạng thái ghi dữ liệu bao gồm ba trạng thái chính được thể hiện trong sơ đồ FSM của hình 3.7 b): (Những trạng thái không có điều kiện ra khỏi trạng thái là những trạng thái chỉ tồn tại 1 chu kỳ và sẽ thoát ra ngay tại chu kỳ sau mà không cần điều kiện xuất hiện)

- **IDLE:** Trạng thái chờ đợi lệnh ghi từ controller, sẽ đợi tín hiệu start_write (start_write == 1) để bắt đầu trạng thái ghi.
- **WRITE ADDR:** Trạng thái truyền địa chỉ cần ghi và các tín hiệu điều khiển giao dịch như WLEN, v.v.. Trạng thái này sẽ chờ đợi awready từ slave (awready == 1) để đi đến trạng thái ghi dữ liệu (WRITE DATA).
- **WRITE DATA:** Trạng thái ghi dữ liệu vào bộ nhớ và chúng tôi sẽ cố định trạng thái này luôn là 1 chu kỳ do chúng tôi chỉ ghi ra 128 bit trong 1 giao dịch.
- **WRITE RSP:** Trạng thái chờ phản hồi kết quả từ quá trình ghi dữ liệu, đồng thời trạng thái này cũng chỉ tồn tại 1 chu kỳ và ngay lập tức nhảy sang trạng thái mới để có thể tiếp tục ghi dao dịch mới ngay lập tức.

Trong quá trình ghi dữ liệu, do một số lớp trong pipeline trả dữ liệu ra từ khối xử lý (PE Cluster) rất nhanh, thậm chí nhanh hơn tốc độ phản hồi từ phía AXI slave, nên nếu máy trạng thái xử lý ghi chỉ thực hiện theo kiểu tuần tự, điều này sẽ dễ dẫn đến nghẽn cổ chai (bottleneck) tại khâu truyền dữ liệu.

Để khắc phục vấn đề này, chúng tôi đã thiết kế máy trạng thái điều khiển ghi dữ liệu theo hướng hỗ trợ nhiều giao dịch ghi song song, cho phép gửi các giao dịch ghi mới ngay cả khi chưa nhận được phản hồi từ các giao dịch trước đó. Cách thiết kế này giúp tăng cường hiệu suất truyền dữ liệu, giảm độ trễ toàn hệ thống và tận dụng hiệu quả băng thông của bus AXI.

Quy trình ghi này được thể hiện rõ trong waveform tại Hình 3.9, nơi các giao dịch được đánh số (#1, #2) để dễ theo dõi. Đây là dạng waveform tương tự như quá trình đọc đã trình bày trước đó, nhưng thể hiện các kênh ghi điển hình của AXI như Write Address, Write Data và Write Response. Các ô giao dịch ghi được đánh dấu bằng màu đỏ để phân biệt rõ từng giao dịch. Có thể thấy giao dịch #2 được khởi tạo và truyền dữ liệu trước khi giao dịch #1 nhận phản hồi (Write Response), thể hiện rõ tính chất non-blocking trong thiết kế máy trạng thái ghi.



Hình 3.9 Quy trình ghi dữ liệu của IP AXI bus trên vivado

3.3 Kết luận chương

Chương 3 đã trình bày kiến trúc của bộ tăng tốc tính toán mạng EfficientNet-V2 do chúng tôi đề xuất, bao gồm các thành phần tính toán, thành phần điều khiển và luồng dữ liệu trong hệ thống. Trên cơ sở đó, chương tiếp theo sẽ trình bày phương pháp xây dựng môi trường thực nghiệm 4.1 và tiến hành đánh giá kết quả của hệ thống 4.2.

CHƯƠNG 4. THỰC NGHIỆM VÀ KẾT QUẢ

Tại chương 4 chúng tôi tiến hành xây dựng môi trường thực nghiệm trên các công cụ phần mềm như vivado để đánh giá các thông số kỹ thuật của bộ tăng tốc tính toán EfficientNet của chúng tôi. Các thông số được đánh giá bao gồm tính đúng đắn của bộ tăng tốc, tài nguyên sử dụng và các thông số liên quan đến throughput và FPS.

4.1 Xây dựng môi trường thực nghiệm

4.1.1 *Xây dựng block design*

Để đánh giá toàn diện các chỉ tiêu kỹ thuật đã đề ra tại Chương 1.2, chúng tôi đã triển khai một hệ thống Block Design trên Vivado (Hình 4.1) với các thành phần chính sau:

MicroBlaze: Bộ xử lý trung tâm đảm nhiệm việc điều khiển toàn bộ hệ thống, bao gồm phân bổ vùng nhớ cho các khối tính toán, giao tiếp với các IP ngoại vi qua AXI UartLite, đồng thời tích hợp MicroBlaze Debug Module và Local Memory để hỗ trợ gỡ lỗi và lưu trữ dữ liệu tạm thời.

AXI SmartConnect (SMC): Cầu nối chịu trách nhiệm định tuyến các giao dịch AXI giữa MicroBlaze và các khối IP tính toán.

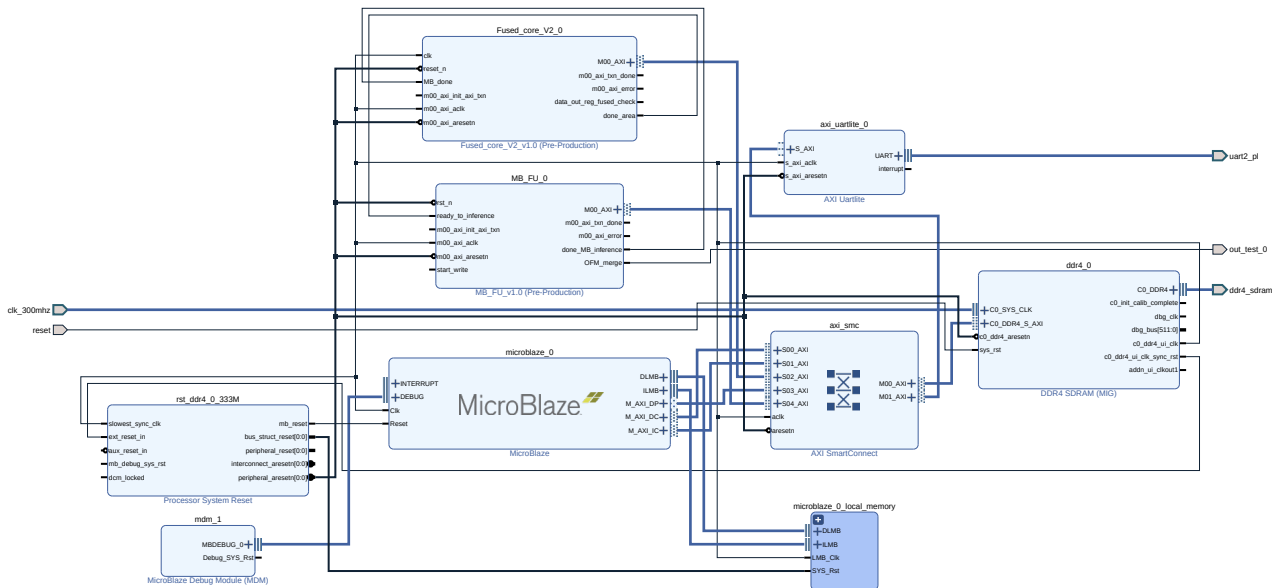
MB FU & Fused FU: Hai khối tính toán chính được đóng gói dưới dạng IP với giao diện AXI và các chân handshake để đồng bộ hóa quá trình tính toán.

DDR4 SDRAM (MIG): Memory Interface Generator đảm nhận giao tiếp với bộ nhớ ngoài DDR4.

AXI UartLite [21]: Kênh giao tiếp UART tốc độ thấp giữa hệ thống và Host PC, nhận giá trị như là Weight khởi tạo, ảnh đầu vào hay là trả kết quả ra bên ngoài.

Processor System Reset: Khởi khởi tạo và đồng bộ tín hiệu reset cho toàn bộ các IP trong thiết kế, đảm bảo hệ thống khởi động ổn định và có thể tái khởi động khi cần.

Hình 4.1 là mô hình Block Design thực tế của Hình 3.1



Hình 4.1 Hệ thống block design trên Vivado

4.1.2 Phân vùng địa chỉ trong block design

Để tránh xung đột truy cập trên bus AXI, mọi thiết bị MASTER và SLAVE phải được gán vào những miền địa chỉ tách biệt, được cấu hình bằng công cụ Address Editor (Hình 4.2). Cấu hình cụ thể như sau:

DDR4 DRAM (2 GB): 0x8000_0000 – 0xFFFF_FFFF Đây là vùng lưu trữ chính cho dữ liệu IFM, Weight và OFM. Các khối Fused FU, MB FU và bộ xử lý MicroBlaze đều có quyền truy cập đọc/ghi vào vùng này.

UART: 0x4000_0000 – 0x4080_0000: Vùng địa chỉ dành cho ngoại vi UART để MicroBlaze cấu hình và điều khiển.

Local Memory (đề cập ở 4.1.1): Bộ nhớ truy cập nhanh dành riêng cho MicroBlaze, có thể dùng cho cả mã lệnh và dữ liệu.

Vùng mã lệnh MicroBlaze (Instruction): MicroBlaze có thể lấy mã lệnh từ DDR4 DRAM hoặc Local Memory, tùy mục đích tối ưu hiệu năng hoặc độ linh hoạt.

Vùng dữ liệu MicroBlaze (Data): Tương tự, dữ liệu mà MicroBlaze xử lý có thể được đặt trong DDR4 DRAM (dung lượng lớn) hoặc Local Memory (truy cập nhanh) hoặc là UART để điều khiển module UART.

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/Fused_core_0					
/Fused_core_0/M00_AXI (32 address bits : 4G)					
/addr4_0/C0_DDR4_MEMORY_MAP	C0_DDR4...	C0_DDR4_ADDRE	0x8000_0000	2G	0xFFFF_FFFF
/MB_FU_0					
/MB_FU_0/M00_AXI (32 address bits : 4G)					
/addr4_0/C0_DDR4_MEMORY_MAP	C0_DDR4...	C0_DDR4_ADDRE	0x8000_0000	2G	0xFFFF_FFFF
/microblaze_0					
/microblaze_0/Data (32 address bits : 4G)					
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64k	0x4060_FFFF
/addr4_0/C0_DDR4_MEMORY_MAP	C0_DDR4...	C0_DDR4_ADDRE	0x8000_0000	2G	0xFFFF_FFFF
/microblaze_0_local_memory/dlmb_bram_if_cntlr/SL _{SLMB}		Mem	0x0	16k	0x3FFF
/microblaze_0/Instruction (32 address bits : 4G)					
/addr4_0/C0_DDR4_MEMORY_MAP	C0_DDR4...	C0_DDR4_ADDRE	0x8000_0000	2G	0xFFFF_FFFF
/microblaze_0_local_memory/ilmb_bram_if_cntlr/SL _{SLMB}		Mem	0x0	16k	0x3FFF

Hình 4.2 Hệ thống block design trên Vivado

4.1.3 Kiểm thử chức năng và đánh giá độ chính xác

Mục tiêu của quá trình kiểm thử là kiểm tra tính đúng đắn của bộ gia tốc CNN được triển khai trên FPGA, đồng thời đánh giá mức độ tương đồng giữa kết quả suy luận từ phần cứng và mô hình tham chiếu trên phần mềm.

• Môi trường và công cụ kiểm thử

- Tiến hành xây dựng một môi trường kiểm thử tự động để thực hiện: Kiểm tra chức năng của thiết kế phần cứng bằng cách mô phỏng tín hiệu và ghi nhận kết quả tại các *checkpoints*. Đánh giá độ chính xác bằng cách so sánh đầu ra từ phần cứng và phần mềm ở mỗi lớp tích chập trong mạng.

– Cụ thể:

Dữ liệu đầu vào: Sử dụng tập ảnh từ ImageNet, đã được tiền xử lý và chuyển sang định dạng số tương thích với hệ thống phần cứng.

Công cụ kiểm thử:

Môi trường kiểm thử được viết bằng ngôn ngữ SystemVerilog. Mô phỏng chức năng và tín hiệu được thực hiện trên phần mềm Vivado Simulator. Mô hình tham chiếu sử dụng thư viện Keras thuộc hệ sinh thái TensorFlow, viết trên môi trường Python.

Dữ liệu đầu ra:

Cả phần cứng và phần mềm sẽ xuất kết quả các feature map dưới định dạng file .hex. Mỗi giá trị pixel trong file .hex được biểu diễn bằng số nguyên 8-bit ở hệ thập lục phân. So sánh được thực hiện tự động bằng một script Python, để kiểm tra sự sai lệch từng điểm ảnh giữa hai kết quả.

• Cấu trúc môi trường kiểm thử

Hình 4.3 mô tả cấu trúc tổng thể của môi trường kiểm thử:

Nguồn ảnh đầu vào: Lấy ảnh ngẫu nhiên từ bộ dữ liệu ImageNet và chuyển đổi thành giá trị số.

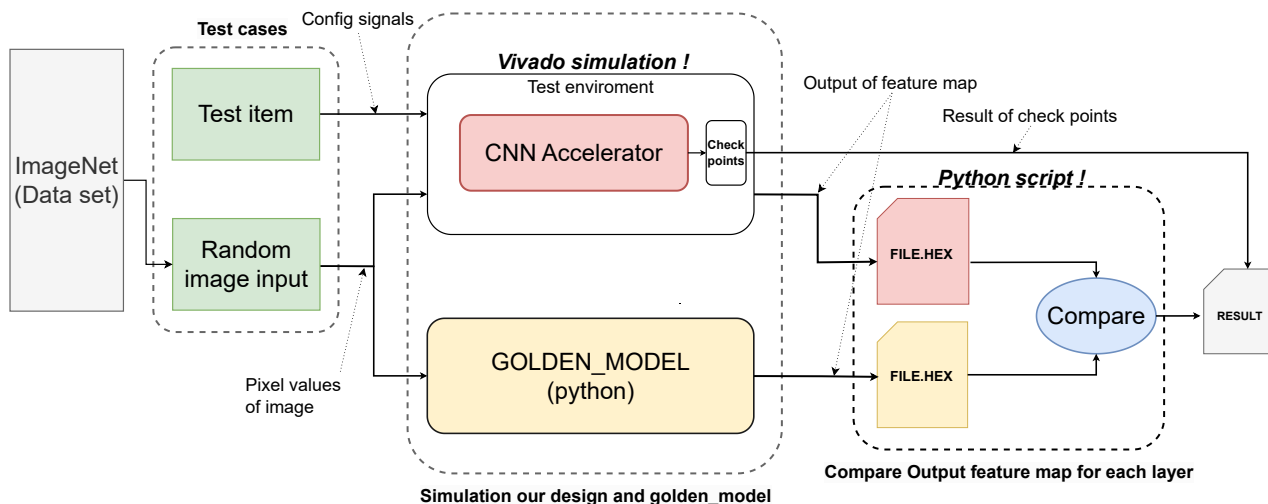
Test item và config signals: Cấu hình độ lớn các lớp mạng và tín hiệu điều khiển được gửi vào Vivado mô phỏng.

Vivado Simulation: Mô phỏng bộ gia tốc CNN, xuất kết quả từng lớp và ghi lại các checkpoint kiểm tra logic.

Golden Model (Python): Mô hình chuẩn mô phỏng toàn bộ mạng EfficientNetV2 và xuất file .hex làm kết quả tham chiếu.

Compare Script: So sánh các file .hex đầu ra giữa mô hình phần cứng và phần mềm theo từng lớp và từng điểm ảnh.

Output (Result): Kết quả bao gồm độ chính xác giữa hai mô hình và báo cáo trạng thái từng checkpoint.



Hình 4.3 Môi trường kiểm thử

- **Tập dữ liệu đánh giá – ImageNet:** Để kiểm tra hiệu năng và độ chính xác của hệ thống, chúng tôi sử dụng tập dữ liệu ImageNet – một trong những tập dữ liệu lớn và phổ biến nhất trong lĩnh vực thị giác máy tính. ImageNet chứa hơn 14 triệu hình ảnh thuộc 1000 lớp phân loại khác nhau, với độ đa dạng cao về chủ đề và góc nhìn. Các đối tượng trong tập dữ liệu bao gồm từ các loài động vật (chó, mèo, chim, cá, v.v.), vật dụng hàng ngày (bàn, ghế, ly, tủ lạnh, v.v.), đến phương tiện giao thông (xe đạp, ô tô, máy bay), cảnh vật thiên nhiên và các loài thực vật.

Các hình ảnh đầu vào được tiền xử lý bằng cách resize về kích thước chuẩn $224 \times 224 \times 3$ và chuẩn hóa giá trị pixel. Tập dữ liệu này không chỉ cung cấp độ khó cao cho bài toán phân loại, mà còn là chuẩn chung để so sánh hiệu năng giữa các mô hình học sâu khác nhau.

- **Đánh giá độ chính xác:** Để đảm bảo phép so sánh phản ánh đúng hiệu suất thực tế của phần cứng, mô hình tham chiếu được lượng tử hóa 8-bit, mô phỏng lại chi tiết cách phần tử tính toán (Processing Element – PE) hoạt động, bao gồm các phép nhân – cộng – tích lũy số nguyên. Nhờ đó, độ chính xác của thiết kế phần cứng hoàn toàn phụ thuộc vào mô hình tham chiếu và phép so sánh từng điểm ảnh trên feature map đầu ra là cơ sở đánh giá đáng tin cậy nhất.

- **Tiêu chí đánh giá kết quả**

Kiểm tra tín hiệu thiết kế thông qua trạng thái tại các điểm kiểm tra (pass/fail).

So sánh dữ liệu đầu ra: Đánh giá sai số giữa file .hex của phần cứng và phần mềm bằng cách so từng điểm ảnh.

Tính đa dạng kiểm thử: Dữ liệu ảnh được sinh ngẫu nhiên từ tập dữ liệu ImageNet nhằm kiểm thử với nhiều trường hợp đầu vào thực tế khác nhau.

4.1.4 *Đánh giá Throughput và FPS*

Trên môi trường mô phỏng Vivado, chúng tôi cho chạy toàn bộ thiết kế để thu thập số liệu hiệu năng. Với tần số xung nhịp $f_{clk} = 266$ MHz, mạng EfficientNetV2-B0 (có độ phức tạp 1.40 GOPs mỗi suy luận) được ánh xạ lên bộ tăng tốc. Bằng cách đếm chính xác tổng số chu kỳ T_{cal} cần cho một suy luận, thông lượng (Throughput) được xác định theo công thức (2.14). Tiếp đó, dùng cùng giá trị T_{cal} và công thức (2.15), chúng tôi suy ra tốc độ xử lý khung hình (FPS) của hệ thống.

4.1.5 *Tài nguyên và năng lượng tiêu thụ*

Sau bước Implementation trên Vivado, báo cáo Utilization cung cấp số lượng LUT, FF, BRAM và DSP đã sử dụng. Báo cáo Power ở cùng giai đoạn cho biết công suất tiêu thụ ước tính của thiết kế. Các số liệu này được ghi nhận và tổng hợp nhằm đánh giá mức độ tối ưu về diện tích và năng lượng của bộ tăng tốc.

4.2 Kết quả thực nghiệm

4.2.1 Quá trình kiểm thử chức năng và đánh giá độ chính xác

Bảng 4.1 Kết quả của các bài kiểm tra

Thứ tự	Tên bài test	Mô Tả	Kết quả
1	Fused-MB Conv 1	Kiểm tra cho cấu hình khối Fused thứ 1	Đúng
2	Fused-MB Conv 2	Kiểm tra cho cấu hình khối Fused thứ 2	Đúng
3	Fused-MB Conv 3	Kiểm tra cho cấu hình khối Fused thứ 3	Đúng
4	Fused-MB Conv 4	Kiểm tra cho cấu hình khối Fused thứ 4	Đúng
5	MB Conv 1	Kiểm tra cho cấu hình khối MB thứ 1	Đúng
6	MB Conv 2	Kiểm tra cho cấu hình khối MB thứ 2	Đúng
7	MB Conv 3	Kiểm tra cho cấu hình khối MB thứ 3	Đúng
8	MB Conv 4	Kiểm tra cho cấu hình khối MB thứ 4	Đúng
9	MB Conv 5	Kiểm tra cho cấu hình khối MB thứ 5	Đúng
10	MB Conv 6	Kiểm tra cho cấu hình khối MB thứ 6	Đúng
11	MB Conv 7	Kiểm tra cho cấu hình khối MB thứ 7	Đúng
12	MB Conv 8	Kiểm tra cho cấu hình khối MB thứ 8	Đúng
13	MB Conv 9	Kiểm tra cho cấu hình khối MB thứ 9	Đúng
14	MB Conv 10	Kiểm tra cho cấu hình khối MB thứ 10	Đúng
15	MB Conv 11	Kiểm tra cho cấu hình khối MB thứ 11	Đúng
16	MB Conv 12	Kiểm tra cho cấu hình khối MB thứ 12	Đúng
17	MB Conv 13	Kiểm tra cho cấu hình khối MB thứ 13	Đúng
18	MB Conv 14	Kiểm tra cho cấu hình khối MB thứ 14	Đúng
19	MB Conv 15	Kiểm tra cho cấu hình khối MB thứ 15	Đúng
20	MB Conv 16	Kiểm tra cho cấu hình khối MB thứ 16	Đúng
21	Padding test	Kiểm tra chức năng khối padding	Đúng
22	Controller test	Kiểm tra chức năng khối điều khiển chính	Đúng
23	AXI Master test	Kiểm tra chức năng đọc, ghi với bộ nhớ ngoài qua giao diện AXI	Đúng
24	Inference test	Kiểm tra chức năng hệ thống khi thực hiện 1 ảnh	Đúng

Bảng trên tổng hợp kết quả của 24 bài kiểm tra chức năng và độ chính xác của hệ thống. Mỗi bài kiểm tra tương ứng với các thành phần của bộ gia tốc CNN trên FPGA, với các mô-đun như Fused-MB Conv, MB Conv và các khối khác như Padding, Controller và AXI Master. Các bài kiểm tra này đều thực hiện việc kiểm tra các khối tính toán chính, chức năng hệ thống và khả năng tương tác giữa các thành phần trong hệ thống.

Nhận xét cụ thể về kết quả:

- Các khối Fused-MB Conv và MB Conv: Tất cả các bài kiểm tra cho các khối tính toán, bao gồm 4 khối Fused và 16 khối MB, đều đạt kết quả "Đúng", cho thấy rằng hệ thống hoạt động chính xác và không gặp vấn đề trong việc thực hiện các phép toán liên quan đến mạng EfficientNetV2.
- Padding Test: Bài kiểm tra này cũng đạt "Đúng", chứng tỏ hệ thống có thể xử lý các phép toán padding mà không gặp sự cố.
- Controller Test: Kiểm tra chức năng của khối điều khiển cũng cho kết quả "Đúng", cho thấy các tín hiệu điều khiển và quá trình điều phối các hoạt động trong hệ thống hoạt động ổn định.
- AXI Master Test: Kết quả cho thấy khối AXI Master có thể truyền và nhận dữ liệu từ bộ nhớ ngoài qua giao diện AXI mà không gặp lỗi, đảm bảo khả năng đọc ghi dữ liệu của khối này.
- Inference Test: Kiểm tra chức năng hệ thống thực hiện một phép suy luận khi xử lý một ảnh cũng đã đạt yêu cầu của bài kiểm tra, xác nhận rằng hệ thống có thể thực hiện đúng các tác vụ suy luận và cho kết quả dự đoán đối tượng trong ảnh.

4.2.2 Tài nguyên sử dụng

Tài nguyên sử dụng trong báo cáo Implementation của Vivado được chúng tôi tổng hợp tại bảng 4.2

Bảng 4.2 Kết quả sau khi tổng hợp bằng Vivado

Kiểu	Bộ tăng tốc	MIG-DDR4	Micro Blaze	Khác	Tổng	Phần trăm tiêu thụ
LUT	10,058	15,090	1,740	15,698	42,586	18.48%
BRAM	186.5	25.5	10	0	179	71.15%
FFs	1,582	17,969	1,367	22,682	43,600	9.46%

Bảng 4.2 trình bày kết quả tổng hợp các tài nguyên sử dụng khi triển khai hệ thống trên

Vivado, bao gồm số lượng các tài nguyên như LUT, BRAM và FFs được sử dụng cho các khối khác nhau như Bộ tăng tốc, MIG-DDR4, MicroBlaze và các thành phần khác.

- LUT (Look-Up Tables): Sử dụng tổng cộng 42,586 LUTs, chiếm 18.48% tài nguyên của FPGA.
- BRAM (Block RAM): Tổng số 222 BRAMs được sử dụng, chiếm 71.15% tổng số BRAM của FPGA.
- FFs (Flip-Flops): Sử dụng 43,600 FFs, chiếm 9.46% tổng tài nguyên.

4.2.3 Throughput và FPS

Các thông số được chúng tôi đo đạc bằng mô phỏng (các đại lượng và cách thức thực hiện đã được mô tả tại mục 4.1.4) được thể hiện trong bảng 4.3.

Bảng 4.3 Kết quả đo hiệu năng

Thông số	Giá trị	Đơn vị
T_{cal}	5,541,666	chu kỳ
Throughput	62.7	GOPS
FPS	48	khung/giây

4.3 Kết quả và nhận xét

Bảng 4.4 Tổng hợp và so sánh với các nghiên cứu tiêu biểu

	[1]	[2]	[3]	[4]	Our Design
Năm	2021	2022	2023	2024	2025
Mô hình	EfficientNet B0	MobileNetV2	EfficientNet B3	MobileNetV1	EfficientNet-V2 B0
GOPs	1.57	0.60	3.67	0.57	1.40
FPGA	XCVU37P	ZCU102	XCVU37P	XCZU9EG	ZCU104
Tần số	300 MHz	100 MHz	300 MHz	150 MHz	266 MHz
Throughput (GOPS)	95.1	29.3	255.2	108.3	62.7
LUTs	71,689	39,100	138 519	214 900	42,586
FFs	113,415	N/A	197 799	166 000	43,600
BRAMs (36Kb)	85	126.5	220.5	506	222
URAMs	56	N/A	40	N/A	0
RAMs (KB)	2398.5	569.25	2432.25	2277	999
Năng lượng (W)	7.8	3	22.9	5.95	2.225
FPS	243	49.2	69.5	190	48
GOPS/W	12.2	9.7	11.15	18.2	28.18
GOPS/RAMs	0.039	0.05	0.1	0.04	0.06

Dựa trên các kết quả thu được trong mục 4.2 cùng thông số của các công trình tiêu biểu hiện nay, chúng tôi lập bảng so sánh tại Bảng 4.4. Trong bảng này, vì mỗi thiết kế có thể dùng

nhiều loại bộ nhớ on-chip khác nhau (BRAM, URAM, ...), để so sánh thống nhất chúng tôi quy đổi tổng dung lượng bộ nhớ thành một chỉ số chung – *RAMs* (KB) – bằng cách cộng gộp kích thước của tất cả các khối nhớ.

Thiết kế của chúng tôi đạt thông lượng 62,7 GOPS, cao hơn nghiên cứu [2] nhưng vẫn thấp hơn các công trình [1], [3] và [4]. Tuy nhiên, khi xét tới hiệu năng trên năng lượng (GOPS/W) và hiệu năng trên dung lượng bộ nhớ (GOPS/RAMs), giải pháp của chúng tôi đều vượt trội so với tất cả các thiết kế còn lại. Bên cạnh đó, mức tiêu thụ tài nguyên logic — bao gồm LUT và FF — cũng thấp hơn đáng kể so với các công trình đối chiếu.

4.4 Kết luận chương

Chương này trình bày quá trình thiết lập môi trường thực nghiệm, bao gồm: Xây dựng block design trên Vivado, phân vùng địa chỉ cho bus AXI, mô phỏng chức năng để kiểm thử và đo lường thông lượng (Throughput) cùng số khung hình trên giây (FPS), thực hiện synthesis và implementation để thu thập các số liệu về tài nguyên (LUT, FF, BRAM/URAM) và công suất tiêu thụ. Cuối cùng, chúng tôi đánh giá hiệu quả của thiết kế bằng cách so sánh các kết quả đạt được với những công trình nghiên cứu tiêu biểu hiện nay.

KẾT LUẬN

Kết luận

Trong đồ án này, chúng tôi đề xuất và hiện thực một bộ tăng tốc tính toán mạng EfficientNet V2 gồm hai khối tính toán chuyên biệt: Fused FU xử lý các lớp Fused-MB Conv và MB FU xử lý các lớp MB Conv. Mỗi khối được trang bị các buffer nội (IFM Buffer và Weight Buffer) cho phép lưu trữ dữ liệu ngay cạnh phần cứng tính toán, nhờ đó giảm độ trễ truy cập bộ nhớ ngoài. Bên trong, các PE cluster chuyên dụng đảm nhiệm từng phép toán tiêu biểu—Standard Conv, Depthwise Conv và Pointwise Conv—đồng thời thiết kế còn hỗ trợ tăng tốc các phép Pooling. Kết nối tới bộ nhớ ngoài được thực hiện qua bus AXI của Vivado.

Để đánh giá, chúng tôi xây dựng một SoC hoàn chỉnh bằng block design trong Vivado và triển khai trên nền tảng Zynq ZCU104 (FPGA). Kết quả cho thấy thiết kế chạy ổn định ở 250 MHz, đạt 62,7 GOPS tổng, bao gồm 34 GOPS từ Fused FU và 28,7 GOPS từ MB FU. Mức tiêu thụ tài nguyên là 58 560 LUT (25,4 %), 52 300 FF (11,24 %) và 222 BRAM (61,67 %).

Hướng phát triển

Mặc dù bộ gia tốc EfficientNet hiện tại đã đáp ứng yêu cầu chức năng, thông lượng (GOPS) và tốc độ khung hình (FPS) vẫn còn khiêm tốn, đồng thời mức khai thác tài nguyên FPGA chưa thực sự tối ưu. Trong tương lai, chúng tôi dự kiến cải tiến kiến trúc theo các hướng sau:

- **Mở rộng số lượng khối tính toán chuyên biệt:** Tăng từ 2 lên 4–8 khối tính toán chính, khai thác tối đa LUT, DSP và BRAM sẵn có, qua đó nâng throughput và FPS.
- **Tăng khả năng tính toán PE/PE Cluster:**
 - Mở rộng số nhân–cộng trong mỗi PE.
 - Tăng số PE trong các PE Cluster để rút ngắn thời gian xử lý từng lớp.
- **Áp dụng cơ chế *ping-pong buffer*:** Hoán chuyển dữ liệu IFM/OFM giữa hai buffer xen kẽ, giảm nhu cầu mở rộng mạnh dung lượng bộ nhớ on-chip và duy trì luồng dữ liệu liên tục.
- **Chèn thêm tầng thanh ghi (flip-flop):** Bố trí thêm các pipeline register trên đường dữ liệu và giữa các bộ cộng và bộ nhân trong các PE, cho phép tăng tần số hoạt động mà không mở rộng đáng kể diện tích logic.

TÀI LIỆU THAM KHẢO

- [1] J. Gao, Y. Qian, Y. Hu, X. Fan, W.-S. Luk, W. Cao, and L. Wang, “Leta: A lightweight exchangeable-track accelerator for efficientnet based on fpga,” in *2021 International Conference on Field-Programmable Technology (ICFPT)*, pp. 1–9, 2021.
- [2] M. Sun, Z. Li, A. Lu, Y. Li, S.-E. Chang, X. Ma, X. Lin, and Z. Fang, “Film-qnn: Efficient fpga acceleration of deep neural networks with intra-layer, mixed-precision quantization,” in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, FPGA ’22, (New York, NY, USA), p. 134–145, Association for Computing Machinery, 2022.
- [3] F. An, L. Wang, and X. Zhou, “A high performance reconfigurable hardware architecture for lightweight convolutional neural network,” *Electronics*, vol. 12, no. 13, 2023.
- [4] H. Hong, D. Choi, N. Kim, and H. Kim, “Mobile-x: Dedicated fpga implementation of the mobilenet accelerator optimizing depthwise separable convolution,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 11, pp. 4668–4672, 2024.
- [5] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009.
- [6] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Commun. ACM*, vol. 60, p. 84–90, May 2017.
- [8] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” 2015.
- [9] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” 2014.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015.
- [11] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” 2017.

- [12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Advances in neural information processing systems*, vol. 25, pp. 1097–1105, 2012.
- [13] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” 2017.
- [14] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” 2020.
- [15] M. Tan and Q. V. Le, “Efficientnetv2: Smaller models and faster training,” 2021.
- [16] H. Hong, D. Choi, N. Kim, H. Lee, B. Kang, H. Kang, and H. Kim, “Survey of convolutional neural network accelerators on field-programmable gate array platforms: architectures and optimization techniques,” *Journal of Real-Time Image Processing*, vol. 21, no. 3, p. 64, 2024.
- [17] A. Ltd., *AMBA AXI and ACE Protocol Specification*, 2011. ARM IHI 0022E.
- [18] X. Inc., *AXI Reference Guide*, 2012. UG761 (v14.5).
- [19] X. Inc., *Vivado Design Suite User Guide: Getting Started*, 2023.
- [20] X. Inc., “Xilinx fpga product overview,” 2023.
- [21] X. Inc., *AXI UART Lite v2.0 Product Guide*, 2023. Xilinx PG143.
- [22] X. Inc., *UltraScale Architecture-Based Memory IP*, 2023. Xilinx User Guide UG586.

PHỤ LỤC

Chứng nhận tham gia Hội Nghị Sinh Viên Nghiên Cứu Khoa Học 2024–2025:



**THÁNG SINH VIÊN
NGHIÊN CỨU
KHOA HỌC
và
SÁNG TẠO
2025**



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Giấy chứng nhận
CERTIFICATE
CAO NGỌC THÀNH ĐÔ

Ngành học/ Field of Study: Kỹ thuật điện tử - viễn thông/ Electronics And Telecommunications Engineering

MSSV/ Student ID: 20210184 - K66

Trường Điện – Điện tử/ School of Electrical and Electronic Engineering

Đã tham gia/ Participated in

HỘI NGHỊ SINH VIÊN NGHIÊN CỨU KHOA HỌC LẦN THỨ 42
THE 42nd STUDENT RESEARCH CONFERENCE

Năm học/ Academic Year: 2024 - 2025

Mã đề tài/ Research ID: S2.2-O-06

Ngày/ Date: 03/06/2025

KT. GIÁM ĐỐC/ PP. PRESIDENT

PHÓ GIÁM ĐỐC/ VICE PRESIDENT



Số/ No.: 5584/QĐ-DHBK



PGS. TS. HUỖNH ĐĂNG CHÍNH